

Program Repair and Automated Debugging with Code Large Language Models: A Comprehensive Survey of Techniques, Applications, and Future Frontiers

1 Introduction and the Paradigm Shift from Traditional to LLM-Driven Repair

1.1 From Heuristics and Templates to Neural Machine Translation

1.1 From Heuristics and Templates to Neural Machine Translation

The historical trajectory of Automated Program Repair (APR) is characterized by a distinct evolution from rigid, rule-based methodologies to flexible, data-driven approaches. In the foundational era of APR, the prevailing paradigm relied heavily on search-based techniques, template-dependent systems, and constraint-solving methods. These early approaches were grounded in the assumption that bug fixes could be synthesized through the systematic application of predefined transformation rules or by exploring a constrained search space of code mutations. While these methods achieved notable successes in specific contexts, they were fundamentally limited by their reliance on human-engineered knowledge and their inability to generalize beyond the scope of their explicit programming.

Search-based APR techniques, often utilizing genetic programming, operated by iteratively mutating buggy code and validating the resulting candidates against a test suite. Similarly, template-based approaches, such as TBar, curated extensive libraries of fix patterns derived from historical commit data to systematically apply known repair strategies to identified faults [1]. Although TBar demonstrated unprecedented performance by correctly fixing 43 bugs on the Defects4J benchmark under standard evaluation pipelines, its effectiveness was inherently capped by the completeness and diversity of its manually crafted template library. The reliance on such templates meant that these systems struggled with novel bug types or complex logical errors that did not conform to existing patterns. Furthermore, the “plastic surgery hypothesis,” which posits that code ingredients for a fix often already exist within the project, was leveraged by redundancy-based approaches to mine donor code. However, methods like Repatt, which utilized fine-grained pattern mining to improve redundancy-based repair, still faced challenges in selecting appropriate donor code, often leading to plausible but incorrect patches when the retrieved context was inappropriate [2].

Constraint-solving methods offered a more formal approach by encoding program semantics and specifications into logical constraints to synthesize correct patches. While powerful for specific classes of bugs, such as those involving arithmetic operators or boundary conditions, these techniques often suffered from scalability issues and required precise formal specifications that are rarely available in real-world software development. The limitations of these traditional heuristics became increasingly apparent as software systems grew in complexity; the search spaces became too vast for exhaustive exploration, and the diversity of bugs outpaced the capacity of human experts to codify effective fix templates. Consequently, the community recognized the need for a paradigm shift that could learn repair strategies directly from data rather than relying on static, hand-crafted rules.

This recognition catalyzed the transition toward Neural Program Repair (NPR), a paradigm that reframed bug fixing as a sequence-to-sequence translation task akin to Neural Machine Translation (NMT) in natural language processing. In this new framework, buggy code snippets are treated as the source language, and their corresponding correct versions are treated as the target language.

This approach leverages deep learning architectures, particularly encoder-decoder models, to automatically learn the latent mappings between erroneous and corrected code from massive repositories of historical bug-fix commits. A seminal work in this domain, SequenceR, introduced an end-to-end sequence-to-sequence learning system that utilized a copy mechanism to handle the unlimited vocabulary problem inherent in source code, successfully predicting fixed lines and finding correct patches for bugs in the Defects4J benchmark without any domain-specific top-down design [3].

The shift to NPR offered significant advantages over traditional methods, primarily in terms of applicability and generalizability. Unlike template-based systems that require predefined patterns, NPR approaches do not need explicit specifications or test suites to generate candidate patches, allowing them to propose fixes for a broader range of bug types [4]. Early learning-based systems like Ratchet demonstrated the viability of this approach by training neural models on pre-correction and post-correction code pairs, achieving high syntactic validity and generating statements that developers found helpful even when not perfectly correct [5]. However, the initial generation of NPR models faced its own set of challenges. Critics noted that early NMT-based techniques often underperformed compared to the best template-based approaches because their search strategies ignored strict code syntax and their search spaces frequently failed to contain the correct fix. To address these limitations, improved models like CURE were developed, incorporating code-aware search strategies and subword tokenization to ensure generated patches were compilable and closer in length to the original buggy code, ultimately outperforming existing techniques on benchmarks like Defects4J and QuixBugs [6].

Despite these advancements, the first wave of neural repair models encountered a critical bottleneck: they remained data-hungry, requiring large numbers of high-quality bug-fixing commits which are costly to construct. Consequently, their edit variety was strictly limited to the patterns present in their training datasets, raising concerns about overfitting and the inability to generalize to unseen projects or novel bug types. This dependency on massive, task-specific datasets created a ceiling for performance that traditional NMT architectures could not easily breach. Nevertheless, the transition from heuristics and templates to neural machine translation marked a pivotal turning point, establishing the essential foundation for the subsequent integration of large pre-trained language models. It shifted the focus from manual feature engineering and rule definition to learning rich, semantic representations of code, thereby setting the stage for the zero-shot and few-shot capabilities that define the current era of LLM-driven repair [7]. This evolution underscores a fundamental change in how the research community conceptualizes program repair: from a problem of search and matching to one of generation and understanding, paving the way for the seismic shift toward pre-trained foundation models discussed next.

1.2 The Rise of Pre-trained Code Models and Zero-Shot Capabilities

1.2 The Rise of Pre-trained Code Models and Zero-Shot Capabilities

Building upon the foundational shift from heuristic rules to Neural Machine Translation (NMT) described in Section 1.1, the landscape of Automated Program Repair (APR) has undergone a second, equally seismic transformation: the migration from data-hungry, task-specific neural models to a paradigm dominated by large pre-trained language models (PLMs). While the NMT era established the viability of learning repair strategies from data, it remained fundamentally constrained by the scarcity of high-quality bug-fix pairs. As noted previously, constructing massive datasets of

bug-fix commits is notoriously costly, and models trained on limited corpora often suffered from poor generalization, effectively capped by the edit patterns present in their training sets [7]. This dependency created a performance ceiling where novel or complex semantic errors remained elusive, setting the stage for a more robust solution.

The emergence of large-scale pre-trained code models, such as CodeBERT, CodeT5, and PLBART, marked the decisive breakthrough needed to overcome these data limitations. Unlike their NMT predecessors, which were typically trained from scratch on narrow bug-fix datasets, these foundation models leverage vast corpora of source code and natural language text acquired through self-supervised learning objectives. This extensive pre-training phase imbues the models with a profound inherent knowledge of programming language syntax, semantics, and common coding patterns, independent of explicit bug-fix labels. Research has demonstrated that models like CodeT5 and PLBART, when pre-trained on both natural language and programming languages, possess an intrinsic understanding that can be directly harnessed for repair tasks without the need for extensive task-specific retraining [8]. By decoupling repair capability from the immediate necessity of large labeled datasets, this transition unlocked the potential for zero-shot and few-shot learning paradigms, directly addressing the generalization bottlenecks identified at the end of the previous section.

Zero-shot learning has emerged as a particularly powerful capability within this new era, allowing models to generate plausible patches without ever encountering a specific bug type during a fine-tuning phase. The approach known as AlphaRepair exemplifies this shift; instead of modeling what a repair edit should look like based on historical data, it directly predicts the correct code based on contextual information using a pre-trained CodeBERT model in a zero-shot setting. Empirical evaluations on standard benchmarks like Defects4J have shown that such zero-shot approaches can substantially outperform state-of-the-art traditional and learning-based APR tools that rely heavily on fine-tuning [7]. Furthermore, extensive studies applying various pre-trained language models directly to APR tasks have confirmed that these models can fix bugs across multiple languages and datasets, often surpassing existing techniques that depend on curated bug-fixing commits [9]. This ability to generalize from pre-trained knowledge rather than memorized fixes represents a qualitative leap in APR methodology.

Complementing zero-shot capabilities, few-shot learning further amplifies the efficacy of these models by leveraging in-context learning. By including a small number of examples in the prompt context, developers can guide general-purpose models to perform specific repair tasks with minimal overhead. Investigations into GPT-3.5 and similar architectures reveal that few-shot learning can significantly boost performance compared to zero-shot baselines, achieving higher exact match rates and code quality metrics in tasks such as code review automation and bug fixing [10]. The ability to learn from just a handful of examples is particularly synergistic with software engineering practices, where project-specific coding patterns and API usages can be rapidly internalized by the model through in-context learning [11]. This flexibility allows the models to adapt dynamically to diverse debugging scenarios that were previously inaccessible to rigid, template-based or narrowly trained NMT systems.

Moreover, the architectural flexibility of modern code PLMs supports a diverse array of repair strategies beyond simple sequence-to-sequence translation. Models like CodeT5+ introduce mixture-of-pretraining objectives and flexible encoder-decoder architectures that can be adapted for various downstream tasks, including code refinement and completion, often achieving state-of-the-art results in zero-shot and instruction-tuned settings [12]. The scaling effect observed in these models suggests that larger parameter counts correlate with better APR performance, enabling them to

handle more complex logical inconsistencies and long-range dependencies within codebases [9]. Additionally, the integration of instruction tuning has further refined these capabilities, allowing models to follow natural language directives for repair, thereby bridging the gap between human intent and automated code generation [13].

Despite these advancements, the transition is not without its challenges. The “black-box” nature of some large models raises concerns regarding data leakage, where evaluation benchmarks might inadvertently appear in pre-training corpora, potentially inflating performance metrics [14]. Nevertheless, the consensus in the field is clear: the move from specialized, data-intensive neural models to versatile, pre-trained foundation models represents a fundamental evolution. This shift empowers APR systems to operate effectively in low-resource settings, leveraging zero-shot and few-shot mechanisms to synthesize fixes that were previously unattainable, thus redefining the boundaries of automated debugging [15]. Crucially, while Section 1.1 highlighted the move from rules to learning, and this section establishes the power of pre-training to reduce data dependency, a critical question remains: what underlying mechanism allows these models to generalize so effectively? As these models continue to scale, they promise not just to mimic historical fixes, but to deliver robust, context-aware solutions that transcend the limitations of historical data dependencies, paving the way for the deep semantic awareness and contextual understanding explored in the following section.

1.3 Beyond Syntax: Embracing Semantic Awareness and Contextual Understanding

1.3 Beyond Syntax: Embracing Semantic Awareness and Contextual Understanding

While Section 1.2 established how pre-trained Code LLMs decouple repair capabilities from data-hungry fine-tuning through zero-shot and few-shot learning, a critical question remains: what exactly do these models learn that allows them to generalize so effectively? The answer lies in a fundamental shift from the historical “plastic surgery hypothesis” to a new paradigm of deep semantic awareness. Traditionally, Automated Program Repair (APR) was defined by the assumption that the ingredients required to fix a bug already exist within the project, leading tools to rely on syntactic pattern matching, heuristic search, and template-based transformations to transplant existing code fragments. While effective for localized defects, these approaches faltered when faced with complex semantic bugs requiring novel logic or an understanding of program intent beyond textual similarity. The emergence of Code LLMs marks a decisive transition from this syntax-centric reliance on code transplantation to a semantic-aware generation capability, where models synthesize entirely new fixes based on a profound understanding of code context, variable usage, and logical flow.

The limitations of the plastic surgery hypothesis have become increasingly apparent in the era of deep learning. Traditional methods, excelling at copying and pasting snippets, often failed when fixes required reasoning about the interplay between distant components or understanding the functional purpose of a code block rather than its syntactic structure. Recent research has revisited this hypothesis, questioning its utility when equipped with the generative power of LLMs. Studies indicate that while the hypothesis remains relevant, LLMs offer a unique opportunity to automate and extend it not just by copying, but by understanding the semantic role of code ingredients and adapting them dynamically. For instance, the work presented in [16] demonstrates that while LLMs can leverage existing project information, their true potential lies in combining direct usage with domain-specific fine-tuning and prompting strategies that go beyond simple retrieval. This suggests that the future of APR is not merely about finding the right snippet to copy, but about

understanding why that snippet works and how it can be modified to fit a new context—a capability rooted in semantic awareness.

This transition from syntax to semantics is further evidenced by the ability of modern models to comprehend the logical intent behind code changes. Unlike the earlier Neural Machine Translation (NMT) models discussed in Section 1.2, which treated repair as a surface-level sequence-to-sequence task, contemporary Code LLMs are increasingly capable of reasoning about underlying logic. The paper [17] argues that high-performing repair requires models to reason about the logic behind code changes, not just the syntactic patterns. By employing multi-objective fine-tuning that explicitly targets the logical reasons for code modifications, researchers have shown significant improvements in patch quality. This approach forces the model to internalize the semantic necessity of a fix, rather than merely mimicking the syntactic structure of historical repairs. Consequently, LLMs are becoming adept at generating patches that resolve logical inconsistencies and semantic errors that would be invisible to tools relying solely on syntactic similarity or static analysis rules.

Furthermore, the depth of contextual understanding possessed by LLMs allows them to navigate complex codebases with a level of sophistication previously unattainable, directly addressing the “small input window” problem that plagued traditional models. Where earlier approaches were restricted to local lines of code, missing critical global context, advanced LLMs can process extensive contexts to understand long-range dependencies and broader architectural intent. Research such as [18] highlights that while repair success correlates with the size and relevance of local context, LLMs push this boundary further by integrating global project knowledge. This expanded context window enables models to discern how a variable is used across different modules or how a specific function contributes to overall system behavior, facilitating fixes that are semantically consistent with the entire codebase.

However, this shift toward semantic awareness is not without its challenges, revealing a disparity between syntactic mastery and true semantic comprehension. Despite their impressive capabilities, LLMs often struggle with dynamic semantics and are susceptible to hallucinations when interpreting complex logical structures. A study titled [19] reveals that while LLMs possess capabilities similar to an Abstract Syntax Tree (AST) parser and demonstrate strong static analysis skills, they frequently fail to fully grasp runtime behavior. The tendency of LLMs to generate syntactically correct but logically flawed code underscores the need for continued advancements in training objectives and architectural designs that prioritize semantic fidelity, a gap that motivates the integration of execution feedback.

Indeed, the integration of dynamic analysis represents a critical frontier in bridging the gap between static semantic understanding and actual program behavior. Purely static analysis, even when powered by LLMs, may miss bugs that only manifest during runtime. Approaches that incorporate dynamic information, such as test case executions and fuzzing results, into the model’s representation learning are proving essential. The work in [20] proposes embedding dynamic information revealed by test cases into code representations, significantly improving the model’s ability to distinguish between functionally equivalent but behaviorally distinct code segments. This fusion of static semantic understanding with dynamic execution insights allows LLMs to generate fixes that are not only syntactically valid but also functionally correct.

In conclusion, the evolution from syntax-centric repair to semantically aware generation represents a foundational change in the philosophy of automated debugging, building upon the pre-trained capabilities outlined in Section 1.2. By moving beyond the constraints of the plastic surgery hypothesis and leveraging the deep contextual and logical reasoning capabilities of Code LLMs, the

field is poised to tackle a broader spectrum of complex bugs. This enhanced semantic grounding sets the stage for the structural redefinition of the repair workflow discussed in Section 1.4, where the decoupled “localize-then-repair” pipeline gives way to unified, end-to-end generation processes that fully exploit these holistic understanding capabilities.

1.4 Redefining the Repair Workflow: From Localize-Then-Repair to End-to-End Generation

1.4 Redefining the Repair Workflow: From Localize-Then-Repair to End-to-End Generation

Building upon the semantic awareness and contextual depth established in Section 1.3, the integration of Code Large Language Models (LLMs) necessitates a fundamental restructuring of the Automated Program Repair (APR) workflow itself. The historical trajectory of APR has been dominated by a rigid, decoupled paradigm known as “localize-then-repair.” In this traditional approach, the debugging process is strictly segmented into two distinct phases: first, a fault localization (FL) technique identifies suspicious code statements, often relying on spectrum-based analysis or mutation testing; second, a patch synthesis engine attempts to generate fixes specifically for those pre-identified locations. While this modular design facilitated early research progress, it inherently suffers from error propagation, where inaccuracies in the localization phase severely constrain subsequent repair capabilities. The emergence of Code LLMs, with their holistic reasoning and generative powers, is fundamentally challenging this dogma, prompting a paradigm shift toward unified, end-to-end generation workflows where localization and repair are no longer separate tasks but intertwined components of a single, coherent reasoning process.

A primary driver of this shift is the recognition that the traditional reliance on off-the-shelf fault localization tools creates an artificial bottleneck that limits the potential of generative models. Recent research suggests that LLMs possess an intrinsic capacity to perform direct debugging without explicit, localization steps. For instance, the work presented in [15] argues that the standard next-token prediction objective of decoder-only LLMs is misaligned with methods that force fixes at fixed locations. By allowing the model to refine the whole program directly, approaches like D4C have demonstrated the ability to surpass state-of-the-art methods that assume perfect fault localization, effectively proving that the “localize-then-repair” constraint hinders LLMs from exploring potential patches beyond given locations. This finding underscores a critical insight: replacing the traditional decoupled workflow with direct debugging is not merely an optimization but a more effective strategy for harnessing the pre-trained capabilities of modern LLMs.

Furthermore, the definition of “localization” itself is evolving within the LLM era, moving from coarse-grained external scoring to fine-grained, internal attention mechanisms. Rather than relying on statement-level suspicion scores provided by external tools, emerging frameworks empower LLMs to perform token-level localization as an integrated part of the repair generation. The [21] study introduces Toggle, a framework that uniquely employs LLMs to predict bug locations at the token level before utilizing them for bug fixing. This methodological separation within a unified model architecture allows for the effective integration of diverse contextual information and inductive biases, enabling the system to generate complete corrected functions rather than isolated line edits. Similarly, [22] proposes a seamless approach where a single pre-trained model addresses bug detection, localization, and repair through multi-task learning. By training on objectives that simultaneously determine if code is buggy, identify specific lines, and translate buggy code to fixed

versions, such unified frameworks ignore the artificial boundaries between tasks to leverage their mutual benefits, significantly outperforming stage-wise techniques.

The scope of repair has also expanded from single-line fixes to holistic function-level and even repository-level corrections, necessitating a redefined workflow that matches the expanded context windows discussed in Section 1.3. Traditional APR often struggles with complex bugs requiring changes across multiple hunks or functions. However, LLM-driven approaches are increasingly adopting function-level APR, which broadens the task scope to fix entire buggy functions using only cost-efficient function-level fault localization. As demonstrated in [23], LLMs with zero-shot learning are already powerful function-level APR techniques, capable of fixing multi-function bugs that were previously unaddressed by any APR technique. This shift reduces the dependency on costly statement-level localization and allows the model to understand the broader semantic context required for complex repairs. Moreover, the challenge of repository-level context is being addressed by frameworks that extract precise context to handle bugs spanning multiple files, as seen in [24], which highlights that providing repository-level context significantly enhances repair rates compared to function-only inputs.

The move toward end-to-end generation is further reinforced by agentic frameworks that simulate the iterative, conversational nature of human debugging, effectively bridging the gap between static semantic understanding and dynamic execution feedback highlighted in the previous section. Instead of a linear pipeline, these systems engage in dynamic loops where localization and repair inform each other iteratively. For example, [25] presents FixAgent, a framework that performs end-to-end localization, repair, and analysis through agent synergy, overcoming the upstream imperfections of traditional fault localization. Similarly, [26] introduces ChatRepair, a conversation-driven approach that interleaves patch generation with instant feedback from test failures. This method learns from both failures and successes of earlier attempts, effectively merging the localization of errors (via test feedback) with the synthesis of new patches in a continuous dialogue. Such approaches demonstrate that the rigid separation of tasks is unnecessary when the model can utilize execution feedback to guide its own search for faults and fixes simultaneously.

Even in scenarios where localization is explicitly modeled, the integration is becoming tighter and more symbiotic. The concept of retrospective fault localization, as explored in [27], reuses the outcome of failed patch validations to refine localization dynamically, creating a feedback loop that blurs the line between the two phases. Additionally, [28] explores the reverse connection, showing that patch execution results from repair attempts can significantly boost fault localization accuracy, suggesting a bidirectional relationship that traditional workflows fail to exploit. The [29] study further reveals that while LLMs like ChatGPT-4 excel at function-level localization, their performance drops at the class level, indicating that future end-to-end workflows must intelligently manage context scope to maintain efficacy.

In conclusion, the advent of Code LLMs is dismantling the century-old “localize-then-repair” silo. By enabling direct debugging, token-level precision, holistic function-level understanding, and iterative agentic workflows, LLMs are redefining the repair process as a unified, semantic-aware generation task. This transition from a decoupled, heuristic-driven pipeline to an integrated, end-to-end generative process represents a fundamental maturation in the field of automated software engineering, promising to resolve complex bugs that were previously inaccessible to traditional APR techniques. This structural evolution sets the foundation for exploring specific architectural innovations and training strategies in the subsequent sections, where we will examine how these unified workflows are technically implemented to maximize repair efficacy.

2 Core Methodologies for Fault Localization and Patch Synthesis

2.1 Advanced Prompt Engineering Strategies for Repair Reasoning

2.1 Advanced Prompt Engineering Strategies for Repair Reasoning

Before delving into parameter-efficient fine-tuning adaptations discussed in the subsequent section, it is essential to address the immediate, training-free mechanisms that unlock the reasoning potential of foundation models for Automated Program Repair (APR). The application of Large Language Models (LLMs) to APR has necessitated a departure from simple input-output generation toward sophisticated prompting paradigms that emulate human debugging cognition. Traditional approaches often treated bug fixing as a direct translation task; however, complex software defects frequently require multi-step logical deduction, hypothesis generation, and iterative verification. To address these challenges, advanced prompt engineering strategies have emerged as critical methodologies for guiding LLMs through the intricate reasoning paths required to identify fault root causes and synthesize logically sound patches. Among these, Chain-of-Thought (CoT) prompting stands as a foundational technique, fundamentally altering how models approach repair tasks by encouraging the generation of intermediate rationales before producing the final code fix.

Chain-of-Thought prompting operates on the principle that decomposing a complex problem into a series of manageable steps significantly enhances the model’s reasoning capabilities. Research indicates that explicitly instructing models to “think step by step” or providing demonstrations with intermediate reasoning chains allows LLMs to tackle multi-step reasoning tasks that would otherwise be insurmountable [30]. In the context of program repair, this translates to models first analyzing the error message, then tracing the data flow, hypothesizing the logical inconsistency, and finally generating the corrected code segment. However, the efficacy of CoT is not merely a result of adding more tokens; it relies on the structural integrity of the reasoning path. Studies have shown that the length and ordering of reasoning steps are crucial, with longer, well-structured chains often yielding superior performance even if individual steps contain minor inaccuracies, provided the overall logical flow remains coherent [31]. Furthermore, the quality of these reasoning chains can be automated and optimized without extensive human annotation, as methods like Auto-CoT demonstrate that diverse, machine-generated reasoning demonstrations can match or exceed manually crafted ones [32].

Despite the success of linear CoT, program debugging often involves exploring multiple potential fixes simultaneously, a process that linear reasoning fails to capture effectively. This limitation has led to the adoption of non-linear reasoning structures such as Tree-of-Thoughts and Graph-of-Thoughts. These frameworks allow the model to explore a search space of possible debugging actions, evaluating different branches of logic before committing to a specific patch. The concept of navigating a reasoning space is supported by frameworks like Boosting of Thoughts, which iteratively explores and self-evaluates multiple trees of thought to acquire an ensemble of trial-and-error experiences, thereby refining the prompting strategy dynamically [33]. Similarly, Graph-Guided Reasoning introduces a mechanism where the model constructs and verifies a “question/rationale graph,” filtering out irrelevant diagnostic paths and generating follow-up queries to gather missing contextual information [34]. In APR, this capability is vital for isolating faults in large codebases where the error symptom may be distant from the root cause, requiring the model to traverse dependencies and verify assumptions at various nodes of the code graph.

To further enhance the reliability of generated patches, rationale-augmented ensembles and self-correction mechanisms have become integral to advanced prompting strategies. The inherent risk

of hallucination in LLMs—where a model generates syntactically correct but logically flawed code—necessitates robust verification steps within the prompt itself. Approaches such as Evidence to Generate (E2G) emphasize grounding the reasoning process in explicit evidence extracted from the context before generating a solution, thereby reducing the incidence of unfounded claims [35]. Moreover, the integration of critique and revision loops allows models to detect and correct their own errors. For instance, methods that employ step-level debating or recursive questioning enable the model to act as both the proposer and the critic of a patch, simulating a peer-review process that significantly improves the faithfulness of the final output [36]. The Socratic Questioning approach further refines this by mimicking human recursive thinking, where the model breaks down the original debugging problem into approachable sub-problems, aggregating their answers to resolve the primary issue with greater robustness against early-stage errors [37].

Dynamic context manipulation also plays a pivotal role in these advanced strategies. Techniques like Residual Connection Prompting (RESPROMPT) reconstruct the reasoning graph within the prompt by integrating necessary connections that are often missing in linear flows, effectively capturing the complex dependencies inherent in multi-hunk bugs [38]. Additionally, handling noisy or incomplete context—a common scenario in real-world debugging—is addressed by strategies such as R³ prompting (Review, Rephrase, Resolve), which guides the model through iterative interactions to extract key variables and resolve ambiguities before attempting a fix [39]. The synthesis of these diverse prompting paradigms—linear decomposition, non-linear exploration, evidence grounding, and iterative self-correction—represents a significant leap forward in APR. By structuring the model’s internal reasoning process to mirror the deliberate, hypothesis-driven nature of human debugging, these advanced prompt engineering strategies enable LLMs to move beyond pattern matching toward genuine semantic understanding and reliable patch synthesis. While these strategies maximize the utility of pre-trained models without weight updates, the following section explores how Parameter-Efficient Fine-Tuning (PEFT) and sample design can further specialize these models for domain-specific adaptation when computational resources and data availability permit.

2.2 Parameter-Efficient Fine-Tuning and Sample Design for Domain Adaptation

2.2 Parameter-Efficient Fine-Tuning and Sample Design for Domain Adaptation

While the advanced prompt engineering strategies discussed in the preceding section effectively unlock the reasoning potential of foundation models without modifying their weights, they are ultimately constrained by the static nature of pre-trained knowledge. To achieve deeper domain specialization for Automated Program Repair (APR), particularly when dealing with nuanced bug patterns or proprietary codebases, it becomes necessary to adapt the model’s internal representations. However, full-parameter fine-tuning is often prohibitively expensive and risks catastrophic forgetting of general programming knowledge. Consequently, Parameter-Efficient Fine-Tuning (PEFT) has emerged as the critical bridge between training-free prompting and full model adaptation, enabling the customization of pre-trained models with minimal trainable parameters. Among these techniques, Low-Rank Adaptation (LoRA) has established itself as a cornerstone methodology. The fundamental premise of LoRA involves freezing the pre-trained model weights and injecting trainable rank decomposition matrices into each layer of the Transformer architecture, thereby drastically reducing the number of trainable parameters without introducing additional inference latency [40]. This approach is particularly advantageous for code repair tasks, where the underlying logic of programming languages often requires only subtle adjustments to the model’s existing representations rather than a complete restructuring of its knowledge base.

However, the application of standard LoRA to code debugging is not without limitations, prompting the development of advanced variants that address issues such as overfitting, rank selection, and task interference. For instance, while LoRA assumes that the adaptation process is intrinsically low-dimensional, a fixed intrinsic rank may not be optimal for all types of bugs or programming languages. To mitigate this, researchers have proposed dynamic adjustment mechanisms. Sparse Low-rank Adaptation (SoRA) introduces a gate unit optimized via proximal gradient methods to dynamically control the cardinality of the rank during training, effectively pruning redundant parameters and enhancing representation power [41]. Similarly, Allocating Low-Rank Adaptation (ALoRA) utilizes importance scoring to estimate the relevance of each LoRA rank, gradually pruning negative ranks and reallocating budgets to critical Transformer modules, thus outperforming static baselines with comparable tunable parameters [42]. These adaptive strategies are crucial for APR, where the complexity of a fix can vary wildly from a simple syntax correction to a deep logical refactor.

Beyond rank optimization, addressing overfitting remains a paramount concern, especially given the often limited size of curated bug-fix datasets. BiLoRA introduces a bi-level optimization framework that splits the training of pseudo singular vectors and values across different data subsets, significantly alleviating the risk of overfitting to specific bug patterns while maintaining generalization capabilities [43]. Furthermore, the integration of residual learning pathways, as seen in ResLoRA, allows for more effective weight updates by adding residual paths during training that are merged away during inference, accelerating convergence for few-shot debugging scenarios [44]. In the specific context of source code, empirical studies have shown that PEFT methods like LoRA and Adapter Tuning can achieve state-of-the-art results in Just-In-Time Defect Prediction and Commit Message Generation, often surpassing full-model fine-tuning in cross-lingual and low-resource settings due to their ability to mitigate catastrophic forgetting [45].

Complementing weight-based adaptations, prompt tuning offers an alternative route for domain adaptation by optimizing continuous prompt vectors while keeping the backbone model entirely frozen. While traditional prompt tuning can struggle with hard sequence labeling tasks common in fault localization, advanced iterations like P-Tuning v2 have demonstrated universality across scales, matching full fine-tuning performance with only a fraction of the parameters [46]. To further enhance efficiency, Decomposed Prompt Tuning (DePT) breaks down soft prompts into shorter vectors and low-rank matrices, optimizing them with distinct learning rates to reduce memory consumption and training time, a vital consideration for processing long code contexts [47]. Moreover, the concept of decomposing prompts via low-rank reparameterization allows for the initialization of soft prompts that capture the low intrinsic rank characteristics of code repair instructions, proving effective in both high and low-resource scenarios [48].

The efficacy of these PEFT techniques is inextricably linked to the principles of sample design engineering used to construct fine-tuning datasets. The quality, diversity, and structural integrity of the training samples dictate the upper bound of model performance, serving as the fuel for the adapted weights. Effective sample design for APR involves not merely collecting bug-fix pairs but curating datasets that cover a broad spectrum of fault types, languages, and complexity levels. Techniques such as Submodular data Mixture strAtegy for instRuction Tuning (SMART) utilize submodular functions to assign importance scores to tasks, redistributing the training budget to select non-redundant samples that maximize coverage and minimize redundancy [49]. This is particularly relevant when combining data from multiple sources, as naïve mixing can lead to performance degradation. Furthermore, in few-shot settings where data is scarce, the strategic composition of LoRA modules trained on diverse tasks can enhance transferability. LoraHub demonstrates that

fluidly combining multiple LoRA modules without additional training can achieve adaptable performance on unseen debugging tasks, suggesting that dataset construction should also consider the modularity of the resulting adapters [50]. Additionally, data augmentation strategies must be carefully evaluated; while techniques like back-translation can boost performance, they may degrade results if the augmented data alters the semantic meaning of the code, necessitating contrastive loss functions to maintain representation fidelity [51]. Ultimately, the synergy between sophisticated PEFT architectures like MoELoRA, which leverages mixture-of-experts to handle diverse computational modules [52], and rigorously engineered datasets forms the bedrock of modern, efficient automated debugging systems. Yet, even with optimized weights and high-quality samples, adapted models remain bound by their parametric memory, unable to access repository-specific dependencies or real-time API changes—a limitation that necessitates the shift toward Retrieval-Augmented Generation explored in the following section.

2.3 Retrieval-Augmented Generation for Contextual Code Enrichment

2.3 Retrieval-Augmented Generation for Contextual Code Enrichment

While Parameter-Efficient Fine-Tuning (PEFT) and sophisticated sample design, as discussed in the preceding section, effectively adapt model weights to the domain of program repair, they remain constrained by the static nature of parametric knowledge. Even the most finely tuned models struggle with repository-specific dependencies, rapidly evolving API landscapes, and the intricacies of multi-hunk bugs that exceed their fixed context windows. To overcome these limitations, Retrieval-Augmented Generation (RAG) has emerged as a complementary paradigm, shifting the focus from solely relying on internalized parameters to leveraging dynamic, external context for patch synthesis. This approach transforms the repair task from a closed-world generation problem into an open-world reasoning challenge, grounded in actual project history and real-time semantics.

A primary application of RAG in Automated Program Repair (APR) involves the retrieval of historical fix patterns to guide the synthesis of new patches. Traditional learning-based approaches often suffer from a fixed parameter set that cannot fully model the expansive search space of possible repairs. To mitigate this, frameworks like RAP-Gen have been developed to leverage a hybrid patch retriever that accounts for both lexical and semantic matching across a codebase of previous bug-fix pairs [53]. By retrieving semantically equivalent bugs and their corresponding fixes, these systems augment the input prompt with concrete examples of how similar logical errors were resolved in the past. This approach significantly outperforms state-of-the-art baselines on benchmarks such as Defects4J, demonstrating that explicit retrieval of fix patterns allows the generator to synthesize more accurate and context-aware patches than relying on implicit memory alone [53]. Furthermore, the concept of the “plastic surgery hypothesis,” which posits that code ingredients for fixes often exist within the same project, has been revitalized in the LLM era through retrieval mechanisms. Methods like FitRepair combine direct LLM usage with domain-specific fine-tuning and prompting strategies that effectively automate the retrieval of project-specific code ingredients, substantially improving repair rates on diverse datasets [16].

However, the efficacy of RAG-driven repair is heavily contingent upon the precision of the retrieval component and the ability of the LLM to utilize the retrieved context correctly. A significant challenge arises when the retrieved context is redundant, imprecise, or exceeds the model’s context limit, particularly in repository-level repair tasks where the scope of relevant code is vast. Research indicates that providing raw repository-level context without refinement can lead to poor performance, as the noise overwhelms the model’s attention mechanisms. To address this, specialized

context extraction methods, such as the Repository-Level Context Extraction (RLCE), have been proposed to filter and provide precise context, resulting in repair rate improvements of up to 160% compared to naive context inclusion [24]. This highlights that optimal configuration of retrievers is not merely about fetching more data but about fetching the *right* data that aligns with the specific fault localization scope, a prerequisite for the hybrid localization architectures discussed in the subsequent section.

Beyond simple retrieval, the interaction between the LLM’s internal prior knowledge—shaped by the fine-tuning processes described earlier—and the retrieved external information presents a complex dynamic known as the “tug-of-war.” Studies show that while correct retrieved information can fix most model mistakes, LLMs may still recapitulate errors if the retrieved content contradicts their strong internal priors or if the retrieved documents contain noise [54]. In the context of code repair, where precision is paramount, this tension necessitates advanced strategies for noise handling and information refinement. Recent advancements propose viewing the LLM as an “Information Refiner,” capable of integrating knowledge from retrieved texts of varied quality to generate concise and accurate fixes, even when the retrieved data is imperfect [55]. Techniques such as active retrieval, where the model iteratively refines its query based on a “knowledge soup” of web searches, documentation, and execution feedback, further enhance the robustness of the generated patches by ensuring the context remains relevant and up-to-date [56].

Moreover, the architecture of the RAG pipeline itself requires careful tuning to balance retrieval quality with generation efficiency. Research into RAG configurations reveals that different model architectures exhibit distinct behaviors regarding context utilization; for instance, decoder-only models may struggle to effectively process large numbers of retrieved documents compared to encoder-decoder models, necessitating tailored retrieval strategies for code-specific tasks [57]. Additionally, the integration of static analysis tools can refine the retrieval process by limiting the LLM’s attention to code portions strictly necessary for the fix, thereby reducing the data requirements and improving the signal-to-noise ratio in the context [58]. As the field evolves, addressing challenges such as data contamination, retrieval bias, and the scalability of knowledge bases remains critical. The development of comprehensive benchmarks and the exploration of hybrid retrievers that blend sparse and dense indexing methods are essential steps toward creating robust, context-enriched APR systems capable of handling the intricacies of modern software development [59]. Ultimately, the successful deployment of RAG in automated debugging hinges on a synergistic design where retrieval precision, noise resilience, and generative capacity are co-optimized. This sets the stage for the next evolution in the field: hybrid architectures that deeply integrate fine-tuned semantic encoders with retrieval mechanisms to achieve project-level fault localization, as explored in the following subsection.

2.4 Hybrid Architectures: Synergizing Fine-Tuning and Retrieval for Localization

2.4 Hybrid Architectures: Synergizing Fine-Tuning and Retrieval for Localization

Building upon the context-enrichment capabilities of Retrieval-Augmented Generation (RAG) discussed in the preceding section, the next evolutionary step in automated fault localization involves the construction of hybrid architectures that deeply integrate fine-tuned semantic encoders with dynamic retrieval mechanisms. While the previous section highlighted how RAG overcomes the static limitations of parametric knowledge by injecting external context, it also revealed critical challenges: the “tug-of-war” between internal priors and retrieved data, and the sensitivity of gen-

eration quality to retrieval noise. Hybrid architectures address these limitations by moving beyond a simple pipeline of “retrieve-then-generate” toward a synergistic design where fine-tuning and retrieval are co-optimized to balance precision, scalability, and context awareness. This subsection evaluates how such integration offers a robust solution for identifying defective code spans within complex, large-scale codebases, effectively bridging the gap between the deep semantic understanding of specialized models and the expansive, up-to-date knowledge access granted by retrieval.

Fine-tuning remains a cornerstone for adapting General Purpose Large Language Models (LLMs) to the specific nuances of software defect detection, serving as the semantic engine within these hybrid systems. While pre-trained models possess broad knowledge, they often lack the specialized sensitivity required to distinguish subtle logical errors from correct variations in code. Research demonstrates that discriminative fine-tuning of LLMs can significantly outperform existing learning-based methods for Automated Vulnerability Localization (AVL), particularly when addressing specific Common Weakness Enumerations (CWEs) [60]. By optimizing model parameters on domain-specific datasets, fine-tuning enhances the model’s ability to capture intricate syntactic and semantic patterns associated with bugs. Furthermore, parameter-efficient techniques, such as adapter tuning, have shown consistent improvements in code intelligence tasks while mitigating catastrophic forgetting, allowing models to retain general versatility while gaining specialization [61]. In a hybrid architecture, these fine-tuned components act not merely as standalone classifiers but as powerful encoders that generate high-quality embeddings, ensuring that the semantic representation of code is sufficiently nuanced to guide subsequent retrieval processes.

Conversely, retrieval augmentation addresses the inherent limitations of fixed model parameters, specifically regarding context length and the need for external, project-specific knowledge, thereby complementing the fine-tuned encoder. LLMs often struggle with long-context inputs, leading to performance degradation when analyzing entire software systems rather than isolated methods [62]. Retrieval-Augmented Generation (RAG) mitigates this by dynamically injecting relevant code snippets, historical bug fixes, or documentation into the model’s context window. Crucially, the efficacy of this retrieval is heavily dependent on the quality of the embedding model used to index and search the codebase. Recent advancements suggest that fine-tuning the embedding model itself, rather than relying on generic pre-trained embeddings, yields superior performance in domain-specific retrieval tasks [63]. This creates a symbiotic relationship: a fine-tuned retriever ensures that the most pertinent context is fetched with high precision, which in turn empowers the generator or classifier to make more accurate localization decisions, directly addressing the noise sensitivity issues identified in pure RAG approaches.

Hybrid architectures leverage this synergy by employing fine-tuned embedding models to enhance the identification of defective code spans through sophisticated fusion strategies. For instance, approaches that integrate global data from open-source projects into Information Retrieval-based Fault Localization (IRFL) frameworks have demonstrated significant improvements over local-data-only baselines [64]. By combining a fine-tuned model’s ability to interpret semantic similarity with a retrieval mechanism’s capacity to access vast historical data, these systems can effectively bridge the gap between bug reports and source code even when lexical mismatches exist [65]. Moreover, the integration of retrieval allows for the incorporation of dynamic execution information or static analysis results as retrievable documents, enriching the context beyond mere source code text and providing the multi-modal signals necessary for robust fault localization.

The complementary nature of these approaches is further evident in handling data scarcity and distribution shifts, scenarios where methodologies often falter. In contexts with limited labeled data, fine-tuning alone may lead to overfitting, whereas pure retrieval might suffer from noise

if the database is not curated. Hybrid strategies, such as those utilizing noise-tolerant training alongside retrieval-based filtering of pseudo-labeled data, offer a resilient framework for leveraging unlabeled data effectively [66]. Additionally, for tasks requiring fine-grained localization, such as line-level defect prediction, combining bilinear attention mechanisms with enriched contextual features retrieved from similar historical defects can significantly boost accuracy [67]. These methods illustrate how hybrid systems can adaptively weigh internal semantic cues against external evidence to achieve superior localization granularity.

Furthermore, the necessity of hybrid approaches extends to localizing faults in declarative models or non-imperative languages, domains where traditional spectrum-based methods often fail. Tools like FLACK for Alloy models demonstrate that analyzing differences between counterexamples and valid instances—a form of logical retrieval—combined with ranking mechanisms can precisely localize errors in declarative specifications [68]. Similarly, in microservice architectures, hybrid frameworks that combine gradient-based causal structure learning with retrieval of historical metric patterns enable fine-grained root cause localization that pure statistical methods cannot achieve [69]. These examples underscore that the fusion of learned representations with retrieved logical or temporal constraints is essential for tackling diverse and complex debugging scenarios.

In conclusion, the future of fault localization lies not in choosing between fine-tuning and retrieval, but in architecting systems where they reinforce each other to overcome the individual limitations highlighted in previous sections. Fine-tuned models provide the semantic depth necessary to understand complex bug patterns and filter noise, while retrieval mechanisms supply the breadth of context and project-specific knowledge required for scalable application. As evidenced by the superior performance of hybrid setups in vulnerability detection, bug report matching, and multi-agent localization systems, this synergistic approach represents a critical frontier in developing autonomous debugging agents capable of operating effectively in real-world, large-scale software environments [62]. Future research should focus on optimizing the interaction protocols between retrievers and generators, exploring adaptive retrieval strategies that evolve based on the fine-tuned model’s confidence, and developing unified training objectives that jointly optimize both components for the specific goal of precise fault localization.

3 Agentic Frameworks, Tool Augmentation, and Interactive Debugging

3.1 Architectural Paradigms for Autonomous and Collaborative Debugging Agents

3.1 Architectural Paradigms for Autonomous and Collaborative Debugging Agents

The landscape of automated program repair has undergone a profound structural evolution, transitioning from monolithic, single-agent models that attempt to solve debugging tasks in isolation to sophisticated multi-agent architectures capable of complex, collaborative reasoning. While early neural approaches treated bug fixing as a direct sequence-to-sequence translation, the inherent limitations of large language models (LLMs) in handling long-context dependencies and logical consistency have necessitated a shift toward societies of agents. These modern frameworks leverage the principles of role specialization, standardized operating procedures (SOPs), and cognitive synergy to decompose intricate debugging workflows into manageable subtasks, thereby mitigating the cascading hallucinations often observed in naive chaining of LLMs. This architectural foundation is critical, as it establishes the organizational logic that subsequent tool-augmented agents will execute within an actionable environment.

A foundational shift in this architectural paradigm is the move from generic problem-solving agents

to systems that encode explicit human workflows. The introduction of Standardized Operating Procedures (SOPs) into prompt sequences has proven critical for streamlining collaborative efforts. By encoding these SOPs, frameworks can assign diverse, domain-specific roles to various agents, effectively creating an assembly line paradigm where each agent verifies intermediate results before passing them downstream. This approach, exemplified by frameworks like MetaGPT, allows agents to emulate human-like domain expertise, significantly reducing logic inconsistencies and ensuring that complex software engineering tasks are broken down into coherent, verifiable subtasks [70]. Furthermore, the integration of SOPs has been shown to enhance agent comprehension of complex task execution in dynamic environments, enabling state-of-the-art performance in multi-step operations without incurring additional inference costs [71]. Such structured workflows provide the necessary scaffolding for agents to effectively utilize external tools, a capability explored in depth in the following section.

Beyond static role assignment, the concept of cognitive synergy has emerged as a pivotal mechanism for enhancing problem-solving capabilities in debugging contexts. Cognitive synergy refers to the dynamic where multiple cognitive processes cooperate to overcome bottlenecks that isolated processes cannot resolve. In the realm of LLMs, this is operationalized through multi-persona self-collaboration, where a single model or a group of models simulates diverse personas to engage in multi-turn reasoning. Research indicates that assigning multiple fine-grained personas enables the emergence of cognitive synergy, which effectively reduces factual hallucinations and maintains strong reasoning capabilities, particularly in models with advanced capacities like GPT-4 [72]. This synergistic interaction is not merely additive; it represents a qualitative shift where the collaborative whole exceeds the sum of its parts, allowing for the rigorous cross-examination of code patches and the identification of subtle logical flaws that a single agent might overlook. Formally, this synergy can be understood as a relationship where cognitive processes assist one another when encountering “stuckness,” a concept that has been formalized using category theory to describe how agents map into each other to resolve impasses [73].

To support these collaborative structures, architectural designs have increasingly adopted hierarchical and self-organizing principles. Hierarchical architectures, often inspired by biological and social systems, facilitate the decomposition of high-level strategic planning from low-level task execution. For instance, frameworks utilizing hierarchical learning allow for the allocation of agents to specific subtasks, reducing observation spaces and encouraging the reuse of subtask-specific policies [74]. This decomposition is resource-rational, mirroring human behavior where tasks are broken down to minimize planning costs while maintaining performance [75]. In the context of code generation and optimization, self-organized multi-agent frameworks have demonstrated the ability to scale indefinitely by automatically multiplying agents based on problem complexity. These systems, such as Self-Organized Agents (SoA), enable agents to operate independently on code components while seamlessly collaborating to construct large-scale codebases, surpassing single-agent baselines in accuracy and scalability [76].

Furthermore, the adaptability of these architectures is enhanced through mechanisms for dynamic role selection and self-evolution. Agents are no longer static entities but can evolve their strategies through inter-task learning, consolidating successful planning trajectories into simplified workflows for future exploitation. Strategies like Investigate-Consolidate-Exploit (ICE) promote this self-evolution, allowing agents to transfer knowledge between tasks and significantly reduce the computational demand required for complex debugging scenarios [77]. Additionally, frameworks that incorporate adaptive role selection enable agents to switch between exploration and coverage modes based on environmental states, ensuring robust performance in open-ended debugging en-

vironments [78]. The emergence of self-organizing agent systems with structures like the “tree of agents” further supports dynamic workflow adjustments, allowing for non-obstructive collaboration where agents asynchronously execute tasks to address the challenges of open and dynamic software ecosystems [79].

Finally, the balance between autonomy and alignment remains a critical design consideration. Effective multi-agent debugging systems must strike a delicate equilibrium where agents possess sufficient autonomy to explore novel repair strategies while remaining aligned with the overall debugging goals and safety constraints. Taxonomies for autonomous LLM-powered multi-agent architectures highlight the necessity of managing this interplay across goal-driven task management and agent composition to prevent divergence and ensure reliable operation [80]. By integrating these architectural paradigms—role specialization via SOPs, cognitive synergy through multi-persona collaboration, hierarchical decomposition, and self-organizing scalability—modern debugging agents are better equipped to handle the nuanced and multifaceted nature of software defects. Having established the structural organization of these agents, the discussion now turns to how these architectures are empowered to interact with the external world through tool-augmented reasoning and executable action spaces.

3.2 Tool-Augmented Reasoning and Executable Action Spaces

3.2 Tool-Augmented Reasoning and Executable Action Spaces

Building upon the architectural paradigms of role specialization and cognitive synergy established in Section 3.1, this subsection examines the mechanistic enablers that allow autonomous agents to interact with the software development environment. While Section 3.1 defined *how* agents are organized to collaborate, Section 3.2 focuses on *what* capabilities these agents possess to execute their strategies. The evolution of Large Language Models (LLMs) from passive text generators to active debugging agents marks a critical paradigm shift in automated program repair. Traditional approaches often generate static code snippets that require external human intervention for validation. In contrast, modern agentic frameworks empower LLMs to transcend mere text generation by actively wielding external tools within a unified, executable action space. This transition allows agents to not only propose fixes but also to execute scripts, interact with compilers, invoke static analyzers, and manipulate debuggers in a closed-loop process, thereby grounding their reasoning in runtime reality.

A foundational advancement in this domain is the concept of “code-first” action spaces, where the agent’s actions are formulated as executable code rather than restricted JSON objects or natural language commands. The [81] paper introduces CodeAct, a unified action space that consolidates diverse agent capabilities into executable Python code. By integrating a Python interpreter, CodeAct enables agents to dynamically revise prior actions and emit new ones based on multi-turn observations. This approach significantly outperforms traditional methods with constrained action spaces, demonstrating that the flexibility to compose multiple tools via code is essential for sophisticated debugging tasks. Similarly, [82] reinforces this architecture by treating user-defined plugins as callable functions within a code-first framework. TaskWeaver converts natural language requests directly into executable code, allowing for rich data structure manipulation and dynamic plugin selection, which is crucial for handling the complex logic inherent in repository-level debugging.

The efficacy of tool-augmented reasoning relies heavily on the agent’s ability to interpret and act upon execution feedback. Static code generation often fails to account for runtime errors or logical inconsistencies that only manifest during execution. To address this, frameworks like

[83] establish environments where code acts as the action and execution feedback serves as the observation in a reinforcement learning setting. This interactive loop allows agents to correct errors, resolve ambiguities, and decompose tasks based on real-time compiler outputs and test results. Furthermore, [84] takes this further by enabling LLMs to refine programs using granular runtime information. LDB segments code into basic blocks and tracks intermediate variable values, allowing the agent to pinpoint errors block-by-block rather than treating the program as an indivisible entity. This step-by-step verification mimics human debugging practices, such as setting breakpoints, and has been shown to consistently enhance baseline performance across various benchmarks. While LDB introduces the concept of runtime verification here, Section 3.3 will delve deeper into the specific cognitive strategies and iterative refinement loops that leverage such feedback for fault localization.

Beyond simple execution, advanced agents leverage specialized tools for static analysis and dependency resolution to ensure code correctness within larger contexts. [85] presents ToolGen, which integrates autocompletion tools directly into the generation process. By triggering these tools at specific mark tokens, the agent can resolve repository-level dependencies, such as undefined variables or missing members, which are common pitfalls in standalone code generation. Complementing this, [86] proposes monitor-guided decoding, where a monitor uses static analysis to guide the model’s token generation. This ensures type consistency and adherence to API protocols, effectively preventing hallucinations that arise from a lack of global context awareness. These mechanisms illustrate how tool augmentation extends the agent’s perceptual horizon beyond the immediate prompt window, supporting the hierarchical and self-organizing structures discussed previously.

The orchestration of these tools often requires sophisticated planning and action space navigation. [87] addresses the challenge of expansive action spaces by formulating tool usage as a decision tree. By employing an A* search algorithm with task-specific cost functions, agents can efficiently prune invalid action branches and identify optimal paths for tool invocation. This is particularly vital in debugging scenarios where the sequence of tool calls—such as running a linter before a unit test, or invoking a debugger after a compilation failure—determines the success of the repair. Additionally, [88] introduces a “Summary to Action” pipeline that mirrors human problem-solving by summarizing achieved results before determining the next tool invocation, thereby improving performance on complex, multi-step tasks.

Security and safety remain paramount when granting LLMs the ability to execute code, a concern that aligns with the autonomy-alignment balance highlighted in Section 3.1. [89] argues for the necessity of “post-facto validation” mechanisms, such as intuitive undo features and damage confinement, to mitigate the risks associated with autonomous code execution. This ensures that even if an agent generates a detrimental patch, the system can revert the changes or bound the potential harm. Moreover, the vulnerability of these tool-integrated agents to indirect prompt injections is highlighted in [90], which demonstrates that malicious instructions embedded in external content can manipulate agents into executing harmful actions. This underscores the need for robust sandboxing and input validation within the executable action space.

Finally, the synergy between tool use and self-improvement is evident in frameworks where agents generate their own tools. [91] describes a closed-loop framework where an LLM acts as a tool maker to craft reusable tools for specific tasks, which are then utilized by a lighter tool-user model. This capability allows debugging agents to adapt to novel environments by creating custom analysis scripts or test generators on the fly. Similarly, [92] shows that agents can systematically generate software augmentations, such as retrieval systems or web navigators, to extend their own capabilities without human engineering. By integrating these diverse tool-augmentation strategies,

modern LLM agents are evolving into autonomous debugging entities capable of navigating the full software development lifecycle, from fault localization to verified patch synthesis. Having established the tools and action spaces that empower agents, the discussion now proceeds to Section 3.3, which details how these agents utilize runtime feedback for iterative refinement and step-by-step verification.

3.3 Iterative Refinement via Runtime Feedback and Step-by-Step Verification

3.3 Iterative Refinement via Runtime Feedback and Step-by-Step Verification

While Section 3.2 established the architectural foundation for tool-augmented agents capable of executing code and interacting with debuggers, this subsection delves into the specific cognitive strategies these agents employ to leverage runtime feedback for iterative refinement. The transition from static code analysis to dynamic, execution-aware debugging represents a pivotal evolution in Large Language Model (LLM) capabilities. Whereas early LLM-based repair systems often treated code generation as a one-pass translation task, modern agentic frameworks increasingly adopt closed-loop workflows that utilize runtime feedback to iteratively localize faults and refine patches. This paradigm shift acknowledges that syntactic correctness is insufficient for functional validity; instead, agents must verify the semantic behavior of code through intermediate variable states and execution traces. By integrating step-by-step verification mechanisms, these agents decompose complex debugging tasks into manageable units, significantly enhancing the precision of fault localization and the robustness of generated repairs.

A fundamental limitation of traditional LLM debugging approaches is their tendency to view generated programs as indivisible entities, making it difficult to pinpoint logical errors within complex control flows. To address this, recent frameworks have introduced techniques that segment programs into basic blocks and track the values of intermediate variables after each execution step. For instance, the Large Language Model Debugger (LDB) exemplifies this approach by enabling LLMs to refine generated programs using detailed runtime execution information. By concentrating on simpler code units within the overall execution flow and verifying their correctness against task descriptions block by block, LDB allows agents to efficiently identify potential errors that would otherwise remain obscured in a monolithic view. Empirical evaluations demonstrate that such block-level verification consistently enhances baseline performance across diverse benchmarks, establishing new state-of-the-art results in code debugging. This methodology effectively mimics the human developer’s practice of setting breakpoints and inspecting variable states, thereby bridging the gap between textual code comprehension and semantic execution understanding [84].

Building upon the concept of execution awareness, other research has focused on teaching LLMs to explicitly reason about program execution through chain-of-thought rationales derived from execution traces. The NExT framework illustrates this by training models to inspect execution traces—specifically variable states of executed lines—and reason about run-time behavior. Through a self-training process that bootstraps a synthetic dataset of execution-aware rationales, NExT enables models to generate high-quality fixes without requiring laborious manual annotation. This approach not only improves the fix rate significantly but also ensures that the generated rationales possess higher quality, as verified by both automated metrics and human raters. Crucially, models trained with this methodology exhibit the ability to generalize to scenarios where program traces are absent at test time, suggesting that the internalization of execution logic enhances the model’s intrinsic reasoning capabilities [93].

The integration of runtime feedback extends beyond mere observation to active prediction and

correction based on divergent execution states. TraceFixer introduces a novel technique where neural program repair models are trained to exploit excerpts of runtime traces alongside source code edits. The core insight is that developers often identify the precise point where actual behavior diverges from desired behavior, such as a variable being assigned an incorrect value. By providing the model with a partial execution trace of the incorrect code and the correct state it should reach, TraceFixer predicts the necessary edits to eliminate the divergence. This approach fundamentally differs from standard techniques that ignore execution traces, resulting in a substantial improvement in bug-fixing ability. Experimental results indicate that exploiting execution traces can improve performance by 13% to 20% compared to baselines relying solely on source code edits, highlighting the critical value of incorporating dynamic state information into the repair loop [94].

Furthermore, the efficacy of iterative refinement is heavily dependent on the quality and granularity of the test suites used to generate runtime feedback. Poorly structured tests can obscure the root cause of failures by mixing multiple execution paths. To mitigate this, techniques like B-Refactoring propose splitting test cases into smaller fragments that cover simpler parts of the control flow. By refactoring the test suite to provide purer execution traces, dynamic analysis tasks are significantly enhanced. This method has been shown to enable the fixing of real-world bugs that were previously unfixable with original test suites, demonstrating that the preparation of the verification environment is as crucial as the repair algorithm itself. Simplifying execution traces allows agents to isolate specific logical errors more effectively, facilitating a more targeted and successful iterative refinement process [95].

In the context of fault localization, which serves as the precursor to patch synthesis, the synergy between program repair and localization creates a powerful feedback loop. The ProFL approach demonstrates that patch execution results from automated program repair can be utilized as feedback to boost fault localization accuracy. By analyzing the outcomes of executing candidate patches, the system can refine its suspicion scores for different code locations, effectively creating a self-boosting cycle. This unified debugging perspective suggests that localization and repair should not be treated as decoupled stages but as interdependent processes within an agentic workflow. The ability to leverage earlier patch execution information to guide subsequent localization efforts reduces the search space and accelerates the convergence toward a correct solution [28]. Additionally, the Self-Boosted Automated Program Repair (SeAPR) technique further exploits this concept by using patch execution information on-the-fly to promote high-quality patches and degrade low-quality ones, substantially reducing the computational cost of validation while improving repair efficiency [96].

Finally, the application of these iterative refinement strategies is particularly vital in safety-critical domains such as Cyber-Physical Systems (CPS), where runtime behavior must adhere to strict temporal and logical constraints. In these environments, agents must not only verify functional correctness but also ensure that repairs do not introduce timing violations or unsafe states. Approaches like FlowRepair utilize search-based methods combined with specific repair objectives, such as minimizing the time a fault remains active, to iteratively refine models until they satisfy all safety requirements. The integration of runtime monitoring and formal specifications into the agent’s feedback loop ensures that the generated patches are not only logically sound but also operationally safe [97]. Similarly, in distributed systems, efficient interaction-based offline runtime verification allows agents to analyze multi-traces from different subsystems to detect and localize faults that arise from complex communication patterns, ensuring that iterative repairs maintain system-wide consistency [98]. Through these advanced mechanisms, agentic frameworks evolve from passive code generators into active, reasoning entities capable of sophisticated, step-by-step

debugging. However, even with robust iterative refinement, single agents may still suffer from confirmation bias or logical blind spots, motivating the shift toward synergistic multi-agent collaboration discussed in the following section.

3.4 Synergistic Collaboration through Multi-Agent Debate and Peer Review

3.4 Synergistic Collaboration through Multi-Agent Debate and Peer Review

While Section 3.3 demonstrated how single agents can leverage runtime feedback and step-by-step verification to refine patches, these iterative loops often remain susceptible to the inherent limitations of individual models, particularly confirmation bias and logical blind spots. When an agent repeatedly validates its own reasoning against execution traces, it may reinforce initial misconceptions rather than correcting them. To address these vulnerabilities, the research community has increasingly turned toward synergistic collaboration strategies that orchestrate interactions between diverse agent personas. By moving beyond single-agent generation, these systems harness “cognitive synergy,” leveraging multi-agent debate frameworks and peer review mechanisms to simulate rigorous software engineering practices such as code review and rubber duck debugging. This approach posits that the collective problem-solving ability of multiple simulated minds can surpass that of isolated individuals, effectively mitigating code hallucinations where models generate syntactically correct but logically flawed or non-existent API calls [72].

Multi-Agent Debate (MAD) has emerged as a prominent paradigm for enhancing the truthfulness and reliability of generated patches by introducing adversarial scrutiny. In these frameworks, multiple agents are assigned distinct personas or viewpoints to argue over the root cause of a bug and the validity of potential fixes. The underlying hypothesis is that adversarial interaction forces agents to critically examine their own logic and expose flaws that a single pass might miss. For instance, frameworks have been developed where agents engage in “tit-for-tat” arguments managed by a judge agent, a process designed to encourage divergent thinking and prevent the “Degeneration-of-Thought” often seen in self-reflection methods where a model reinforces its own initial incorrect confidence [99]. However, the efficacy of these debate structures is not universal; empirical studies suggest that without careful hyperparameter tuning and specific protocol designs, multi-agent debating systems may not reliably outperform simpler prompting strategies like self-consistency [100]. Consequently, advanced implementations now incorporate retrieval-augmented mechanisms to break cognitive constraints, allowing agents to access external documentation during debates to rectify obstinate adherence to incorrect viewpoints [101].

Parallel to debate frameworks, the concept of peer review collaboration has been adapted to mimic the academic and industrial code review processes, providing a structured alternative to adversarial arguing. In this setting, agents operate as independent reviewers who construct solutions, critique the outputs of others, and assign confidence levels to their assessments. This “step outside the box” approach allows a single underlying model to leverage diverse reasoning paths by simulating a committee of experts. Research indicates that such multi-agent peer review strategies can deliver superior accuracy across complex reasoning tasks compared to existing single-agent methods, particularly when the system integrates confidence-weighted feedback exchanges rather than mere solution sharing [102]. The diversity of the participating agents is a critical factor; frameworks like ReConcile utilize a round-table conference structure where diverse LLMs learn to convince one another, employing confidence-weighted voting to reach a consensus that often surpasses the

performance of even the strongest individual model in the group [103].

A specific application of these collaborative principles in debugging is the simulation of “rubber duck debugging,” a technique where a programmer explains code line-by-line to an inanimate object to clarify their own thinking. While traditional implementations rely on human articulation, LLM-based agents can automate this by acting as attentive listeners or active questioners. Although some studies investigating embodied entities in rubber duck scenarios have found mixed results regarding performance improvements compared to inanimate objects, the theoretical foundation remains strong for using conversational agents to force explicit reasoning [104]. When extended to multi-agent settings, this evolves into a dynamic where one agent articulates the bug hypothesis while another challenges the assumptions, effectively grounding the reasoning process and reducing the likelihood of hallucinated fixes, thus complementing the runtime verification strategies discussed previously.

Furthermore, the architecture of these collaborative systems is evolving to include more sophisticated coordination mechanisms inspired by biological and social theories. Concepts such as the Theory of Collective Mind are being applied to predict the mental states and intentions of fellow agents, thereby improving cooperative efficiency in complex decision-making environments [105]. This is particularly relevant in debugging scenarios where agents must coordinate to localize faults across distributed codebases. Techniques that disentangle interaction patterns and filter noisy communications between entities have shown promise in improving the generalizability and interpretability of multi-agent systems, ensuring that the collaboration remains focused on relevant code segments [106].

Despite the promise of these synergistic approaches, challenges remain regarding computational cost and the complexity of managing long-horizon contexts. Distilling the reasoning interactions from multiple large agents into smaller, more efficient models has been proposed as a viable solution to maintain high reasoning quality while reducing inference overhead [107]. Additionally, the robustness of these systems against adversarial attacks is an area of active investigation, with findings suggesting that multi-agent debates can reduce model toxicity and susceptibility to jailbreaks when capable agents debate with less capable or compromised ones [108]. As the field progresses, the focus is shifting from simple debate protocols to structured, goal-oriented collaborations that combine the creativity of LLMs with the rigor of formal verification and peer inspection. However, while these internal multi-agent dynamics significantly enhance autonomous reasoning, the ultimate deployment of debugging assistants requires seamless integration with human developers. This necessity drives the evolution toward the human-in-the-loop interaction patterns and conversational debugging assistants explored in the following section, which bridge the gap between autonomous agent collaboration and human-AI co-creative problem solving.

3.5 Human-in-the-Loop Interaction Patterns and Conversational Debugging Assistants

3.5 Human-in-the-Loop Interaction Patterns and Conversational Debugging Assistants

While the synergistic collaboration strategies discussed in Section 3.4 leverage multi-agent debates to mitigate hallucinations and refine reasoning internally, the integration of Large Language Models (LLMs) into Interactive Development Environments (IDEs) necessitates a parallel evolution in how these models interact with human developers. This shift moves the paradigm from static, one-off code generation to dynamic, conversational collaborations where the effectiveness of a debugging assistant is measured not just by isolated patch synthesis, but by its capacity to engage in sustained, context-aware dialogue for natural language root cause analysis. Early implementations

often treated model invocations as independent events, lacking memory of previous interactions or the broader codebase context, which led to implicit assumptions and inaccurate responses. To address these deficiencies, modern conversational designs emphasize multi-turn interactions where the developer and AI collaboratively navigate the debugging workflow. Research indicates that equipping LLMs with structured interaction patterns, such as insert expansion and facilitated turn-taking, significantly lowers conversation barriers and can lead to substantial improvements in bug resolution rates, with some studies reporting up to a fivefold increase in success when these patterns are effectively implemented [109].

A critical component of effective human-in-the-loop debugging is the management of agency, balancing autonomous agent actions with targeted human oversight. While LLMs possess impressive generative capabilities, they often struggle with proactive management of interaction direction unless explicitly designed to exhibit agency features such as intentionality, motivation, and self-regulation. In a debugging context, an agent must not only respond to queries but also proactively suggest diagnostic steps, clarify ambiguous error messages, and articulate its reasoning process to the human user. Frameworks that measure and enhance these agency features have shown that models manifesting high levels of self-efficacy and self-regulation are perceived as more strongly agentive, thereby fostering a more productive collaborative environment [110]. This shift transforms the LLM from a passive code completer into an active partner capable of co-creative problem solving, a role that participants in user studies have found to unlock capabilities beyond simple code generation, including deep knowledge retrieval and complex architectural reasoning [111].

However, the transition to conversational debugging introduces significant challenges regarding safety, reliability, and the potential for hallucination, particularly in multi-turn scenarios. Unlike single-turn interactions, multi-turn dialogues offer adversaries or even unintentional user behaviors the opportunity to decompose unsafe or complex queries into sub-queries that bypass safety guardrails, potentially leading the agent to generate harmful code or reveal sensitive system information. Recent investigations have exposed vulnerabilities where LLMs fail to reject borderline unsafe queries when they are part of a longer, incremental dialogue sequence, highlighting the urgent need for robust safety mechanisms specifically tailored for multi-turn conversational contexts [112]. Furthermore, the “black box” nature of these models can obscure the decision-making process, making it difficult for developers to trust suggested fixes without transparent explanation. To mitigate this, metacognitive approaches have been proposed to equip LLMs with self-aware error identification and correction capabilities. These frameworks enable the model to construct concept-specific subnetworks that illuminate decision pathways, allowing for user-friendly intervention and rectification at inference time without the need for additional parameter tuning [113].

The design of these conversational interfaces also plays a pivotal role in how users customize and experience the debugging assistant. Fixed personality and functionality constraints in standard LLM-driven agents can limit their adaptability to specific debugging styles or domain expertise required by different developers. Systems that support easy and accurate customization of agent personas allow users to tailor the assistant’s behavior, leading to stronger emotional bonds, more dynamic dialogues, and a greater willingness to sustain interactions over time [114]. Moreover, the interaction flow in debugging tasks often involves distinct phases such as planning, facilitating, iterating, and testing. A comprehensive taxonomy of human-LLM interaction modes suggests that moving beyond standard prompting to context-based and agent-facilitator modes is essential for handling the complexity of iterative debugging tasks [115].

To ensure that these conversational assistants remain helpful and honest while avoiding harmful outputs, grounding responses in commonsense social rules and human norms is increasingly rec-

ognized as a vital strategy. Approaches that utilize human-norm-guided decoding can achieve quantitatively safer responses without the substantial costs associated with fine-tuning on large annotated datasets [116]. Additionally, the concept of “proactivity” in dialogue systems is crucial for debugging; agents must be capable of clarification, target-guided questioning, and even non-collaborative refusal when user requests are ambiguous or potentially dangerous. Proactive Chain-of-Thought prompting schemes have been developed to augment LLMs with goal planning capabilities, enabling them to navigate these complex conversational landscapes more effectively [117]. While current human-in-the-loop systems rely on static or pre-defined interaction protocols, the increasing complexity of software defects demands more flexible coordination. This necessity drives the evolution toward the dynamic orchestration and adaptive team management mechanisms explored in the following section, which enable agent teams to self-configure and optimize their collective behavior in real-time.

3.6 Dynamic Orchestration and Adaptive Team Management

3.6 Dynamic Orchestration and Adaptive Team Management

While Section 3.5 established the importance of structured human-in-the-loop interaction patterns for conversational debugging, the increasing complexity of software defects often exceeds the capacity of static or pre-defined interaction protocols. Consequently, the field is evolving toward dynamic orchestration mechanisms that enable agent teams to self-configure and optimize their collective behavior in real-time. This shift moves beyond rigid, sequential workflows to adaptive systems capable of responding to the varying unpredictability of software failures. In complex debugging scenarios involving multifaceted bugs—spanning syntax errors, logical flaws, and architectural inconsistencies—advanced coordination mechanisms are required to allow agent teams to dynamically reconfigure based on task complexity. By leveraging self-organizing structures, hierarchical leadership models, and optimization algorithms, these systems can autonomously select the most capable agents for specific debugging stages, ensuring resilience and efficiency where static pipelines fail.

Self-organizing structures represent a paradigmatic shift in how debugging agents collaborate, moving away from centralized control toward emergent, decentralized coordination. Inspired by biological and complex systems, self-organization allows agent teams to autonomously establish, maintain, and reconfigure dynamic multi-level system architectures in response to environmental changes or task failures [118]. In the context of software repair, this means that a group of specialized agents—such as those dedicated to fault localization, patch synthesis, and verification—can spontaneously form coalitions to address a specific bug without explicit human intervention. This capability is critical for handling open-ended debugging environments where the nature of the defect is unknown a priori. Research into self-organizing agents in open-ended environments demonstrates that structures like the “tree of agents” and “hourglass agent architecture” enable dynamic workflow adjustments, balancing information priorities and allowing asynchronous task execution [79]. Such systems can autonomously coordinate to address challenges in dynamic environments, ensuring that the debugging process remains efficient even as the codebase or error landscape evolves. Furthermore, formal frameworks for reasoning about agents’ independence in self-organizing systems provide the theoretical underpinning for verifying these dynamic coalitions, ensuring that local agent behaviors contribute effectively to the global goal of bug resolution [119].

Complementing decentralized self-organization, hierarchical leadership models offer a structured approach to dynamic orchestration, particularly when debugging tasks require a clear separation

between strategic planning and tactical execution. In these models, a high-level “manager” or “coach” agent decomposes complex debugging objectives into subtasks and allocates them to specialized subordinate agents. This hierarchical decomposition significantly reduces the observation space for individual agents and encourages the reuse of subtask-specific policies, which is essential for scaling debugging efforts across large codebases [74]. The effectiveness of such hierarchies is further enhanced by mechanisms that facilitate inter-level cooperation, ensuring that goal-assigning and goal-achieving levels within the hierarchy are aligned through modified objective functions and gradient adjustments [120]. For instance, in heterogeneous teams involving both human developers and AI agents, a hierarchical decision model can leverage attention mechanisms to discern nuances in the state space, facilitating efficient initial task assignment and subsequent reassignment as the debugging process unfolds [121]. Moreover, the concept of “Objects and Key Results” (OKR) driven agent systems introduces a hierarchical self-collaboration mechanism where high-level strategic planning is recursively decomposed into detailed task executions, with multi-level evaluation modules refining solutions at each step [122].

To maximize the efficacy of both self-organizing and hierarchical structures, optimization algorithms play a pivotal role in selecting the most capable agents for specific debugging stages, ensuring that the right expertise is applied at the right time. This selection process can be formulated as a combinatorial-hybrid optimization problem, where both the discrete modes of collaboration (i.e., which agents form a team) and the continuous control parameters (i.e., how they execute the repair) are optimized simultaneously [123]. In dynamic settings where agent capabilities or task requirements may change, algorithms such as Dynamic Iterative Task Allocation Graph Search (D-ITAGS) enable targeted repair by identifying and repairing parts of the existing solution impacted by disruptions while conserving the rest of the valid workflow [124]. Additionally, learning dynamic subtask assignment (LDSA) frameworks allow agents to be grouped based on their evolving abilities, ensuring that agents dealing with similar subtasks share specific knowledge while maintaining diversity across different debugging stages [125]. The integration of graph-based coordination strategies further enhances this selection process by capturing underlying dynamic decision structures, allowing the system to generate directed acyclic graphs that optimize the flow of information and actions among agents [126].

The synergy between self-organization, hierarchy, and optimization is essential for building resilient debugging agents that can handle the inherent uncertainty of software defects. For example, in scenarios requiring resilience against agent failures or unexpected code behaviors, hybrid automation modes can switch between homogeneous deformation (where all agents coordinate uniformly) and containment exclusion (where failed agents are isolated and tasks are rerouted) [127]. This adaptability ensures that the debugging process continues uninterrupted even when individual components of the agent team fail. Furthermore, the application of structured prediction approaches allows for zero-shot generalization, enabling agent teams to scale to larger problems with more agents and tasks than seen during training, a crucial feature for adapting to the vast diversity of real-world software projects [128]. By combining these advanced coordination mechanisms, dynamic orchestration systems achieve a level of flexibility and robustness that static workflows cannot, ultimately leading to more effective and autonomous program repair capabilities. However, realizing the full potential of these adaptive teams requires rigorous assessment. As these systems evolve from simple coordinators into autonomous, self-organizing problem solvers, traditional evaluation metrics become insufficient, necessitating the development of specialized benchmarks and performance metrics to measure their progress, collaboration efficiency, and reasoning capacity, as explored in the following section.

3.7 Evaluation Benchmarks and Performance Metrics for Agentic Debugging Systems

3.7 Evaluation Benchmarks and Performance Metrics for Agentic Debugging Systems

The transition from the dynamic orchestration mechanisms discussed in Section 3.6 to practical deployment necessitates a fundamental rethinking of evaluation paradigms. As Large Language Model (LLM) agents evolve from single-turn text generators into autonomous, multi-step problem solvers capable of self-organization and hierarchical coordination, traditional metrics focusing solely on final task success rates prove increasingly insufficient. These binary outcomes fail to capture the nuances of agentic debugging, where the iterative processes of fault localization, hypothesis generation, and refinement are as critical as the final patch. To address this, the research community is shifting towards modular, extensible frameworks designed to dissect agent behavior across diverse dimensions. A prime example is AgentQuest, which advocates for modular benchmarks and introduces specific metrics to reliably track an agent’s progress throughout a multi-step task rather than just recording binary outcomes [129]. This granularity is essential for debugging agents, as it allows researchers to pinpoint whether a failure stems from incorrect fault localization, flawed reasoning during repair synthesis, or an inability to effectively utilize external tools—issues that dynamic orchestration strategies aim to mitigate but must be rigorously measured.

A significant challenge in evaluating these sophisticated agentic debugging systems lies in the complexity of the environments they must navigate. Real-world debugging rarely occurs in isolated, static codebases; instead, it involves dynamic interactions with compilers, test runners, file systems, and distributed infrastructure, mirroring the unpredictable conditions that adaptive agent teams are designed to handle. Consequently, there is a pressing need for benchmarks that emulate these realistic, interactive settings. WebArena exemplifies this trend by providing a highly realistic web environment where agents must perform long-horizon tasks, revealing that even state-of-the-art models struggle significantly compared to human performance in complex, open-ended scenarios [130]. Similarly, OSWorld extends this concept to general computer environments, offering a scalable platform for assessing agents on open-ended tasks across different operating systems, highlighting deficiencies in GUI grounding and operational knowledge that are directly relevant to interactive debugging assistants [131]. These environments move beyond synthetic datasets to test an agent’s ability to handle the unpredictability and tool diversity inherent in actual software development workflows, providing a true stress test for the resilience mechanisms described previously.

Furthermore, given the emphasis on collaborative frameworks in Section 3.6, evaluating modern debugging systems requires metrics that specifically assess social intelligence and coordination efficiency. The effectiveness of an agent team cannot be measured merely by the sum of individual capabilities; it depends on how well agents communicate, delegate sub-tasks, and resolve conflicts within self-organizing or hierarchical structures. Benchmarks like LLMArena address this by evaluating LLM agents in dynamic multi-agent environments, utilizing scoring systems to assess crucial abilities such as opponent modeling, team collaboration, and strategic planning [132]. In the context of debugging, this translates to evaluating how well a “localizer” agent communicates root causes to a “repairer” agent, or how a team of agents debates potential fixes to mitigate hallucinations. The concept of the “Harmony Index” further contributes to this domain by providing a utilitarian metric to quantify the effectiveness of mixed-skill teams, classifying team members based on their contribution to overall effectiveness [133]. Such metrics are vital for understanding the synergy in multi-agent debugging frameworks where role specialization and dynamic reconfiguration are key.

Beyond collaboration, the ability of an agent to adapt its strategy based on feedback and self-assess

its competency is a critical dimension of robust debugging, directly reflecting the adaptive nature of the systems proposed in prior sections. Dynamic environments often present novel bugs or changing constraints that require agents to adjust their reasoning on the fly. Frameworks like AgentBoard introduce fine-grained progress rate metrics that capture incremental advancements, offering a more analytical view of agent performance over multi-turn interactions [134]. This is complemented by research into dynamic competency self-assessment, where agents use statistical tests to evaluate their own confidence during task execution, a capability essential for knowing when to request human intervention or switch strategies [135]. Additionally, the “Investigate-Consolidate-Exploit” (ICE) strategy highlights the importance of inter-task self-evolution, where agents consolidate planning trajectories into simplified workflows, a process that can be benchmarked by measuring reductions in API calls and improvements in execution efficiency [77].

However, the current landscape of evaluation faces significant hurdles, particularly regarding benchmark saturation and the lack of standardization in measuring reasoning capacity. Studies mapping the global dynamics of benchmark creation warn that many existing benchmarks quickly trend toward saturation, failing to distinguish between progressively smarter models [136]. This necessitates the development of self-evolving benchmarks that can dynamically generate new, challenging instances to prevent overfitting, ensuring that evaluation keeps pace with the rapid advancements in agent orchestration. Moreover, the concept of “reasoning capacity” has been proposed as a unifying criterion to evaluate multi-agent systems under real-world constraints like budget and time, arguing for a holistic approach that connects different system components rather than optimizing narrow objectives [137]. Finally, specialized frameworks like MAgIC provide quantitative metrics for cognition, adaptability, rationality, and collaboration in multi-agent settings, using game-theoretic scenarios to expose capability gaps between different model architectures [138]. Together, these emerging benchmarks and metrics form a comprehensive toolkit for rigorously assessing the next generation of autonomous debugging agents, ensuring they are not only capable of fixing code but doing so efficiently, collaboratively, and reliably in complex, real-world ecosystems.

4 Ensuring Correctness: Verification, Self-Correction, and Neuro-Symbolic Hybrids

4.1 Neuro-Symbolic Frameworks for Semantic Validation and Formal Guarantees

4.1 Neuro-Symbolic Frameworks for Semantic Validation and Formal Guarantees

While Large Language Models (LLMs) have ushered in an era of unprecedented generative capability for automated program repair, they remain fundamentally constrained by the issue of hallucination, often producing syntactically correct but logically flawed code. To bridge the gap between the probabilistic nature of neural generation and the deterministic requirements of software correctness, research has increasingly pivoted toward neuro-symbolic frameworks. Unlike the intrinsic self-correction mechanisms discussed in the following section, which rely on the model’s internal ability to critique its own outputs, neuro-symbolic approaches externalize logical validation to rigorous symbolic solvers. These hybrid architectures leverage the semantic understanding and code synthesis strengths of LLMs while delegating formal proof generation and constraint satisfaction to deterministic engines, ensuring that generated patches are not merely plausible but causally reliable and formally verified.

At the core of this paradigm is a strict division of labor: the LLM acts as a translator or hypothesis generator, converting natural language bug descriptions or informal code comments into precise symbolic formulations, while the symbolic solver performs deductive reasoning to verify their validity.

For instance, the Logic-LM framework exemplifies this approach by utilizing an LLM to translate natural language problems into symbolic logic, which is subsequently processed by a deterministic solver. This method significantly enhances logical problem-solving capabilities, demonstrating substantial performance improvements over standard prompting and chain-of-thought techniques by ensuring that the reasoning process is faithful to logical constraints [139]. Similarly, in the domain of planning and formal artifact synthesis, researchers have proposed using Satisfiability Modulo Theories (SMT) solvers as deductive engines. In this setup, the LLM generates potential solutions or plans, which are then analyzed by the SMT solver; if a solution is incorrect, the solver produces counterexamples that are fed back to the LLM, creating an iterative refinement loop that steers the model toward provably correct responses [140].

A critical advantage of neuro-symbolic integration is the ability to generate reasoning proofs that are guaranteed to be causal and reliable. Unlike pure neural approaches where reasoning traces may be post-hoc rationalizations lacking true causal links, neuro-symbolic methods utilize custom meta-interpreters and symbolic execution to produce verifiable proofs. The CaRing method, for example, employs a neural LLM to represent problem knowledge while relying on an LLM-free symbolic solver for deliberative reasoning. Empirical results indicate that this approach surpasses chain-of-thought baselines by nearly double in accuracy and more than triple in proof similarity, underscoring the efficacy of deterministic execution in ensuring the reliability of reasoning proofs [141]. This is particularly vital for program repair, where the “why” behind a fix is as important as the fix itself, enabling developers to trust the automated suggestions.

In the specific context of formal theorem proving and complex code verification, frameworks like Lyra have introduced dual correction mechanisms to mitigate hallucinations effectively. Lyra employs Tool Correction to replace incorrect prover tools in post-processing and Conjecture Correction to refine formal proof conjectures based on error messages from the prover. This interactive feedback mechanism allows the system to adjust subgoals and refine generations without requiring paired training data of errors and refinements, achieving state-of-the-art performance on rigorous benchmarks like miniF2F [142]. Furthermore, the application of neuro-symbolic reasoning extends to ethical and logical inference, where hybrid frameworks integrate LLMs with backward-chaining solvers to refine step-wise explanations. The Logic-Explainer framework demonstrates how such integration can enhance the logical validity and alignment of explanations, producing formal proofs that support the model’s reasoning and reduce inconsistencies [143].

The versatility of neuro-symbolic approaches is further highlighted by their application to specialized reasoning domains such as Theory of Mind (ToM). The ToM-LM approach externalizes the reasoning component of ToM tasks to a symbolic model checker, allowing the LLM to focus on generating the symbolic formulation while the executor handles the transparent and verifiable reasoning logic. This separation of concerns significantly improves performance on complex reasoning tasks that pure LLMs struggle with, suggesting a generalizable pattern for enhancing reliability in diverse debugging contexts [144]. Additionally, for arithmetic and logical reasoning within code, the SYRELM architecture demonstrates that even smaller, parameter-efficient LMs can achieve high accuracy when tasked with formalizing problems for a symbolic solver, rather than attempting to solve them end-to-end. This “formalize-then-solve” strategy, trained via reinforcement learning with feedback from the non-differentiable solver, offers a robust alternative to scaling model size alone [145].

Moreover, the concept of transforming indeterminate premises into determinate ones through iterative reasoning, as seen in the DetermLR framework, provides a structural blueprint for debugging agents. By categorizing premises and prioritizing exploration based on relevance, such systems can

navigate complex codebases more efficiently, reducing the search space for potential bugs and ensuring that each reasoning step contributes deterministically toward a solution [146]. The cognitive architecture perspective further reinforces this, arguing that hybrid neuro-symbolic approaches are essential for overcoming the limitations of current LLM-centered systems, providing the necessary dual-process architecture for reliable, human-like reasoning in software engineering tasks [147].

Ultimately, neuro-symbolic frameworks represent a decisive move away from relying solely on the statistical likelihood of token sequences toward ensuring semantic correctness through external formal guarantees. By anchoring the creative generation of LLMs in the rigid logic of symbolic solvers, these hybrid systems offer a pathway to autonomous debugging agents that can not only propose fixes but also prove their correctness. However, while these external solvers provide strong validation, they require the LLM to correctly formulate the problem in a symbolic language—a non-trivial task. This limitation naturally leads to the complementary exploration of intrinsic self-correction mechanisms, where the focus shifts to enhancing the model’s internal capacity to verify and refine its own reasoning without explicit reliance on external symbolic tools, a topic examined in the subsequent subsection.

4.2 Intrinsic Self-Correction Mechanisms and Critique Abilities

4.2 Intrinsic Self-Correction Mechanisms and Critique Abilities

While the neuro-symbolic frameworks discussed in the preceding section externalize logical validation to rigorous symbolic solvers to ensure formal guarantees, a parallel line of inquiry focuses on enhancing the model’s internal capacity to verify and refine its own outputs. This paradigm of intrinsic self-correction aims to emulate the human developer’s iterative process of writing, reviewing, and refactoring code without immediate reliance on external oracles or explicit symbolic translation. The core hypothesis driving this research is that Large Language Models (LLMs), given their vast pre-training on code corpora, possess latent reasoning capabilities that can be unlocked through structured prompting and iterative feedback loops. However, the efficacy of these intrinsic mechanisms remains a subject of intense debate. While early optimism suggested that scaling model parameters would naturally yield robust self-verification skills, recent empirical studies indicate that LLMs often struggle to identify logical flaws in their own reasoning traces, sometimes even degrading performance when attempting naive self-refinement [148]. This subsection analyzes the mechanisms designed to overcome these limitations, focusing on step-by-step verification, confidence-aware prompting, and ensemble-based criticism, while highlighting the inherent constraints that necessitate the hybrid approaches explored later in this survey.

A primary strategy for enhancing intrinsic correction is the decomposition of the verification task into granular, step-by-step processes. The assumption that verifying a solution is computationally easier than generating one does not always hold for LLMs, particularly when errors accumulate across long reasoning chains. To address this, frameworks like SelfCheck propose zero-shot verification schemas where the model explicitly checks its own step-by-step reasoning to recognize errors before finalizing an output [149]. Similarly, the concept of “Deductive Verification of Chain-of-Thought Reasoning” advocates for decomposing the verification process into a series of subprocesses, ensuring that each deductive step is rigorously grounded on prior premises, thereby mitigating the propagation of hallucinations [150]. These approaches suggest that forcing the model to articulate its critique at a fine-grained level can unlock latent verification capabilities that are obscured in holistic, single-pass generation, offering a neural alternative to the symbolic step-wise validation seen in neuro-symbolic systems.

Despite these structural improvements, a significant barrier to effective self-correction is the model’s inability to accurately assess its own uncertainty. LLMs frequently exhibit overconfidence, leading them to stubbornly maintain incorrect solutions or aggressively “correct” valid ones based on noisy internal feedback. Research highlights that overlooking the factor of “confidence” is a key reason for the instability of intrinsic reflection [151]. To remedy this, novel prompting frameworks such as “If-or-Else” (IoE) have been developed to guide LLMs in assessing their confidence levels, facilitating more reliable self-correction by only triggering refinement when the model detects low confidence in its initial response [151]. Furthermore, methods like Fact-and-Reflection (FaR) prompting improve calibration by first eliciting known facts relevant to the problem and then asking the model to reflect on them, which helps the model verbally express concerns in less confident scenarios [152]. These confidence-aware mechanisms are crucial for preventing the “hallucination-reliability paradox,” where syntactically correct but logically flawed patches are generated with high certainty, a risk that symbolic solvers in the previous section mitigate through deterministic false-positive rejection.

When individual self-critique proves insufficient, ensemble-based strategies offer a powerful alternative by leveraging diverse perspectives to identify discrepancies. The “Self-Contrast” method operates on the insight that LLMs often provide inconsistent feedback when evaluating their own work; by adaptively exploring diverse solving perspectives and contrasting the differences, the model can summarize discrepancies into a checklist to re-examine and eliminate errors [153]. This approach effectively mitigates stubborn biases by exposing the model to its own conflicting outputs. Extending this to a multi-agent context, the “N-Critics” framework employs an ensemble of critics alongside the model’s own feedback to refine outputs, drawing inspiration from human behavior where individuals seek input from others to refine their understanding of complex topics [154]. Similarly, multi-agent peer review collaboration allows multiple model instances to independently construct solutions, critique one another, and revise their outputs based on peer feedback, demonstrating superior accuracy compared to single-model self-correction [102]. These methods represent an attempt to simulate the robustness of external validation using only the model’s internal knowledge distribution.

However, it is imperative to acknowledge the fundamental limitations of these intrinsic mechanisms, which distinguish them sharply from the formal guarantees of neuro-symbolic approaches and the statistical filtering of the ranking methods to follow. Empirical evidence suggests that while LLMs may possess the capacity to correct errors if the location of the mistake is explicitly provided, they frequently fail at the preceding step of mistake finding [155]. Studies on planning tasks reveal that self-critiquing can diminish performance due to a high rate of false positives in the verifier’s judgments, casting doubt on the reliability of purely iterative, self-contained frameworks for complex debugging [156]. Furthermore, the content of the criticism generated by the model often matters less than the presence of a sound external verifier, implying that intrinsic feedback loops may be inherently noisy [157]. Consequently, while techniques like “Learning to Check” demonstrate that meticulous dataset design and tailored prompts (e.g., “Step CoT Check”) can enhance self-checking capabilities [158], the consensus is shifting toward hybrid approaches. These approaches combine intrinsic critique with stronger external signals or specialized verifier models, recognizing that small language models, and even some large ones, often require strong verifiers to effectively self-correct reasoning tasks [159]. Thus, while intrinsic self-correction provides a valuable first line of defense for refining candidate patches, its practical deployment in program repair is increasingly viewed as a precursor to, rather than a replacement for, the robust external validation and sophisticated ranking mechanisms detailed in the subsequent sections.

4.3 Neural Patch Ranking and Representation Learning for Correctness Prediction

4.3 Neural Patch Ranking and Representation Learning for Correctness Prediction

Building upon the recognition in the previous section that intrinsic self-correction often lacks the robustness to guarantee semantic correctness without external validation, a critical bottleneck emerges in the subsequent stage of Automated Program Repair (APR): distinguishing between genuinely correct patches and those that merely overfit the available test suite. Overfitting occurs when a generated patch passes existing tests but fails to address the underlying semantic defect, potentially introducing new errors or masking symptoms. While intrinsic mechanisms attempt to refine code internally, and as will be discussed in the following section, tool-interactive approaches leverage explicit execution feedback, neural patch ranking and representation learning serve as a vital intermediate layer. These techniques employ deep learned code embeddings, entropy-based naturalness metrics, and semantic correlation models to predict patch correctness with high fidelity, effectively acting as a sophisticated secondary filter to enhance the reliability of APR workflows before costly dynamic verification is applied.

The foundation of modern patch correctness prediction lies in the ability to learn robust vector representations of code changes. Early approaches relied heavily on manually engineered features or dynamic execution traces, which often lacked generalizability across diverse contexts. However, recent advancements have demonstrated the efficacy of using pre-trained transformer models to derive deep embeddings that capture the syntactic and semantic nuances of patches. For instance, research has shown that BERT transformer-based embeddings, when associated with machine learning classifiers such as XGBoost or logistic regression, can achieve Area Under the Curve (AUC) values approaching 0.8 in predicting patch correctness [160; 161]. These learned embeddings offer a significant advantage over traditional methods by encoding complex patterns of code evolution that are difficult to capture through manual feature engineering. Furthermore, studies indicate that these representations are complementary to existing dynamic approaches; when combined with engineered features, they form hybrid predictors like “Panther” that outperform models relying solely on either learned or hand-crafted attributes [160].

Beyond static embeddings, the concept of code “naturalness” has emerged as a powerful metric for ranking candidate patches, addressing the limitation where models generate syntactically valid but statistically anomalous code. The hypothesis underpinning this approach is that buggy code tends to be statistically unnatural or high in entropy, whereas correct code adheres to the probabilistic patterns of professional software development. By leveraging the entropy scores emitted by LLMs, researchers can re-rank patch candidates before they undergo expensive test suite execution. A notable application of this principle involves using “entropy-delta” to measure the change in naturalness between the buggy and patched states. Empirical results suggest that such entropy-guided re-ranking can improve Top-1 accuracy by nearly 50% compared to traditional spectrum-based fault localization techniques, effectively filtering out unnatural, likely incorrect patches early in the pipeline [162]. This aligns with earlier findings that code with bugs exhibits higher entropy and that focusing on highly entropic lines can significantly improve the cost-effectiveness of bug detection and repair [163]. Moreover, the integration of entropy metrics helps mitigate the risk of test-suite overfitting by prioritizing patches that not only pass tests but also align with the statistical norms of correct code [164].

Another innovative direction in correctness prediction involves framing the assessment task as a Question Answering (QA) problem, thereby bridging the gap between natural language intent and code implementation. The intuition here is that a correct patch essentially provides the “answer” to

the “question” posed by the bug report or the failing test behavior. Approaches like Quatrain utilize neural architectures to learn the semantic correlation between natural language bug descriptions and the commit messages or code changes associated with patches. By treating patch correctness assessment as a semantic matching task, these models can achieve high recall rates, identifying over 90% of correct patches while filtering out a majority of incorrect ones [165]. This method underscores the importance of multimodal reasoning, where the alignment between natural language intent and code implementation serves as a strong indicator of validity, complementing the purely code-centric views of embedding and entropy-based methods.

To further refine ranking capabilities without relying solely on massive labeled datasets, researchers have developed unsupervised methods that exploit the similarity between failing test cases and historical fixes. The system BATS, for example, operates on the hypothesis that similar failing test cases require similar patches. By embedding failing test specifications and searching for historically similar cases, BATS can assess the correctness of new patches without requiring large labeled datasets for training [166]. Similarly, lightweight tools like ObjSim prioritize patches by comparing the object state similarity between the original and patched programs, effectively promoting patches that preserve the intended system state on passing tests while diverging appropriately on failing ones [167]. These approaches provide a computationally efficient means of triaging candidates, setting the stage for more rigorous verification.

Despite these advancements, the field continues to grapple with the limitations of purely neural approaches, particularly regarding data leakage and the need for explainability. To address this, hybrid frameworks like INVALIDATOR combine semantic reasoning via program invariant inference with syntactic reasoning using pre-trained language models. This dual approach allows for the detection of overfitting patches that maintain erroneous behaviors or violate correct specifications, achieving higher accuracy than methods relying on a single reasoning modality [168]. Additionally, zero-shot approaches leveraging large code LLMs, such as PatchZero, have demonstrated the ability to assess patch correctness without task-specific fine-tuning, offering a scalable solution for evaluating patches from unseen APR tools [169]. Collectively, these neural ranking and representation learning techniques represent a paradigm shift in ensuring APR correctness. However, while they provide a strong probabilistic filter, they ultimately serve as a precursor to the definitive validation provided by external tools. As the field moves toward more robust systems, these neural predictors are increasingly integrated into pipelines that culminate in the tool-interactive, feedback-driven synthesis detailed in the next section, ensuring that final patches are not only statistically probable but semantically verified.

4.4 Tool-Interactive Critiquing and External Feedback Integration

4.4 Tool-Interactive Critiquing and External Feedback Integration

While neural patch ranking and representation learning, as discussed in the previous section, provide a sophisticated probabilistic filter to prioritize candidate patches, they ultimately lack the definitive ground truth required to guarantee semantic correctness. Purely learned metrics, whether based on embeddings, entropy, or semantic matching, cannot fully eliminate the risk of overfitting or hallucination without empirical validation. To bridge this gap between high-probability candidates and verified solutions, contemporary research has pivoted toward tool-interactive critiquing and external feedback integration. This paradigm shifts the Large Language Model (LLM) from a passive generator relying on internal knowledge to an active participant in a closed-loop feedback system. By integrating external tools—such as compilers, test runners, and static analyzers—directly into

the repair loop, these frameworks leverage executable signals to interpret error messages, validate logical consistency, and guide iterative refinement with a precision that static prediction alone cannot achieve.

A foundational aspect of this approach is the capacity to execute code and interpret runtime feedback as a direct observation signal. The [83] framework exemplifies this by establishing a standardized environment where code generation acts as an action and execution results serve as the observation. This setup enables LLMs to engage in a reinforcement learning-like process, correcting errors based on concrete execution traces rather than abstract reasoning. Similarly, the [170] framework explicitly mirrors human debugging practices, where developers utilize external interpreters to cross-check outputs. CRITIC demonstrates that enabling LLMs to interact with tools to evaluate specific aspects of their generated text significantly enhances performance across diverse tasks, highlighting the critical role of external feedback in fostering continuous self-improvement and reducing reliance on intrinsic model confidence.

Beyond holistic execution, advanced frameworks have begun to decompose the debugging process to leverage granular runtime information, addressing the limitations of treating programs as indivisible blocks. The [84] framework segments programs into basic blocks, tracking intermediate variable values after each segment during runtime. This step-by-step verification allows the LLM to pinpoint errors within complex logic flows more effectively than whole-program verification methods. By focusing on simpler code units and verifying their correctness against task descriptions block by block, LDB achieves state-of-the-art performance, demonstrating that fine-grained runtime feedback is essential for diagnosing and repairing intricate software defects that evade coarse-grained checks.

Complementing dynamic execution, the integration of static analysis tools provides a pre-execution safety net to catch structural and logical errors early. The [171] system employs a dual-LLM architecture wherein a proposer generates revisions based on static analysis recommendations, and a ranker evaluates these candidates against a rubric aligned with developer acceptance criteria. This ensures that revisions satisfy both static quality constraints and functional requirements, significantly reducing false positives. Furthermore, the [172] approach leverages LLMs to simulate pseudo-code execution for static analysis tasks. By incorporating a verification process for this simulated execution, E&V mitigates hallucinations and improves the accuracy of identifying faulty functions in complex systems like the Linux kernel, proving that LLMs can effectively emulate static analysis when guided by structured verification protocols.

Compiler feedback serves as another vital source of external guidance, particularly for resolving syntax and type errors that prevent code execution entirely. The [173] system emulates human interactive repair by assigning distinct roles: a Code Learner that generates code and a Code Teacher that crafts a Chain-of-Repair. The Code Teacher utilizes compiler error feedback to reassess bugs and provide precise repair instructions, leading to substantial improvements in generation and translation tasks. Similarly, [174] synergizes an LLM with a completion engine to prune infeasible tokens and proactively complete code based on semantic constraints. This interaction ensures generated patches are statically valid, addressing the common issue where LLMs produce syntactically correct but semantically flawed code due to a lack of awareness regarding underlying language constraints.

Iterative refinement driven by tool feedback is also central to frameworks designed for complex, multi-step reasoning and long-term memory retention. The [175] framework reinforces agents through linguistic feedback derived from external environments, such as compilers or test suites. Agents verbally reflect on task feedback signals and store these reflections in an episodic memory

buffer, enabling improved decision-making in subsequent trials without updating model weights. Additionally, the [176] framework focuses specifically on learning to self-refine faulty generations using execution results from test suites. CYCLE demonstrates that code language models can significantly enhance their self-refinement capabilities when provided with explicit feedback on execution failures, outperforming larger models that lack such iterative training mechanisms.

The effectiveness of these tool-interactive approaches underscores a broader shift in automated program repair: the transition from open-ended generation to constrained, feedback-driven synthesis. Whether through the dynamic orchestration of agents in [177], which modularizes workflows into grounding, execution, and observing roles, or through the risk-aware calibration in [178], which uses programmatic supervision to assign risk scores to LLM responses, the consensus is clear. External tools provide the necessary ground truth that LLMs lack internally. By interpreting compiler errors, validating against unit tests, and simulating static analysis, these frameworks create a robust safety net that guides LLMs toward semantically correct and reliable patches. However, the efficacy of this feedback loop remains intrinsically bound to the quality of the oracle itself; if the test suite is incomplete or the specifications are weak, even the most sophisticated interactive agent may converge on an overfitted solution. This limitation necessitates the next critical evolution in the field: the augmentation of test suites and the synthesis of formal specifications to ensure the verification ecosystem itself is robust enough to support these advanced repair loops.

4.5 Test Suite Augmentation and Specification Synthesis for Robust Verification

4.5 Test Suite Augmentation and Specification Synthesis for Robust Verification

While the tool-interactive frameworks discussed in the previous subsection leverage existing compilers and test runners to guide repair, the efficacy of any Automated Program Repair (APR) system remains intrinsically bound to the quality and completeness of the validation oracle itself. In many real-world scenarios, available test suites are insufficient to fully specify the expected behavior of the program under repair, leading to the pervasive issue of patch overfitting. Overfitting occurs when a generated patch passes all existing test cases but fails to generalize to other inputs or violates undisclosed specifications. To mitigate this reliance on imperfect oracles, recent research has increasingly focused on test suite augmentation and the synthesis of formal specifications as critical mechanisms for robust verification. These approaches aim to strengthen the repair constraints by automatically generating stronger test cases, deriving formal properties from natural language descriptions, and employing invariant-guided searches to filter out semantically incorrect patches, thereby providing the rigorous ground truth necessary for the verification processes analyzed in the following section.

A primary strategy for combating overfitting involves the automatic generation of additional test cases to expand the validation scope. The rationale is that a denser set of test inputs reduces the probability of a spurious patch passing verification by chance. Early investigations into this approach, such as the study on the Nopol repair system, proposed methods like UnsatGuided, which utilizes automatically generated tests to strengthen repair constraints. However, empirical evaluations revealed a nuanced reality: while automatic test generation proved effective in alleviating regression introduction (one form of overfitting), it had minimal impact on preventing incomplete fixing due to the inherent “oracle problem” where the correctness of new test outputs is difficult to ascertain [179]. Further large-scale studies confirmed these limitations, noting that while test case generation can alter the resulting patch, it is not universally effective at transforming incorrect patches into correct ones without sophisticated guidance [180].

To address the limitations of random or heuristic-based test generation, researchers have turned to Large Language Models (LLMs) and differential testing to create more potent test oracles. Traditional automated tools often struggle to generate tricky bug-revealing inputs and precise oracles for complex scenarios. Recent advancements propose frameworks like AID, which combines LLMs with differential testing to target plausibly correct programs. By selecting inputs that yield diverse outputs across program variants generated by LLMs, these systems construct robust test oracles that significantly outperform state-of-the-art baselines in recall and precision [181]. Similarly, techniques like TELPA enhance LLM-based test generation for hard-to-cover branches by integrating program analysis results and using counter-examples as feedback. This iterative refinement allows the model to learn complex object construction and inter-procedural dependencies, achieving substantial improvements in branch coverage compared to search-based software testing [182]. Furthermore, the integration of metamorphic relations has shown promise in enhancing the fault detection effectiveness of automatically generated test cases, providing a mechanism to verify correctness even in the absence of a perfect ground truth [183].

Beyond generating new test inputs, another vital avenue is the synthesis of test oracles themselves, particularly assertions that define expected behaviors. The difficulty of manually constructing functional oracles has spurred the development of neural methods for Test Oracle Generation (NTOG). Approaches like TOGA leverage transformer-based models to infer both exception and assertion oracles based on the context of the focal method, demonstrating the ability to find real-world bugs that other methods miss [184]. However, critical re-evaluations of such neural methods have highlighted significant challenges, including high false positive rates and misclassification of oracle types, suggesting that current models often lack the confidence or accuracy required for widespread deployment without human oversight [185]. To improve oracle quality, researchers have also explored retrieval-augmented generation, such as EditAS, which retrieves similar focal-test pairs and edits the prototype assertions to fit the current context, outperforming previous integration approaches in accuracy [186]. Additionally, leveraging developer-written summarizations as guidance has proven effective; the SAGA approach fine-tunes pre-trained language models to generate assert statements guided by natural language summaries, capturing intended program behavior that pure code context might miss [187].

Parallel to test generation, the synthesis of formal specifications and invariants offers a rigorous path to verification, serving as a deterministic counterpart to probabilistic test generation. By inferring likely invariants from the program’s execution or static analysis, tools can detect patches that maintain erroneous behaviors or violate correct specifications. The INVALIDATOR tool exemplifies this by combining semantic reasoning via program invariants with syntactic reasoning from pre-trained language models to assess patch correctness without requiring new test cases, successfully identifying a significant portion of overfitting patches [168]. Similarly, static analysis feedback, specifically using Incorrectness Separation Logic, can guide the repair process by evaluating how close a patch is to fixing a bug based on symbolic heap properties, thereby reducing reliance on test suites for memory safety issues [188]. The integration of static bug detection patterns into specifications allows repair systems to narrow the search space and prioritize patches that satisfy logical constraints, further enhancing reliability [189].

Finally, the concept of specification synthesis extends to translating natural language requirements into executable constraints. In domains where formal specs are missing, generating specification instructions from natural language can guide models to adhere to desired functionalities. Research indicates that larger language models can benefit from such specification-augmented prompts, generalizing desirable behaviors across unseen functionalities [190]. Moreover, the use of tailored

mutation operators, derived from project history and codebase characteristics, can create mutants that are more closely coupled to real faults, effectively simulating a stronger test suite for evaluating patch robustness [191]. Collectively, these methods of test suite augmentation and specification synthesis represent a paradigm shift from passive validation against static test suites to active, dynamic, and semantic-rich verification ecosystems. However, the success of these augmented oracles ultimately depends on the reliability of the verification mechanism itself, a challenge that leads directly to the limitations of intrinsic self-verification and the critical need for strong external verifiers discussed in the next subsection.

4.6 Limitations of Intrinsic Self-Verification and the Role of Strong Verifiers

4.6 Limitations of Intrinsic Self-Verification and the Role of Strong Verifiers

Building upon the active verification ecosystems established in the previous subsection—where test suite augmentation and specification synthesis provide robust external constraints—it is crucial to evaluate the reliability of the mechanisms tasked with utilizing these constraints. The promise of autonomous program repair often rests on the assumption that Large Language Models (LLMs) can effectively identify and rectify their own logical errors through intrinsic self-verification. However, a critical examination of recent empirical studies reveals significant boundaries to this capability, suggesting that relying solely on a model’s internal reasoning for self-correction is often insufficient and, in many cases, detrimental. While Section 4.5 detailed how to strengthen the *oracle*, this subsection addresses the limitations of the *critic*: the asymmetry between generation and verification. Although LLMs excel at producing plausible code sequences, their ability to accurately critique the logical validity of those sequences without external grounding remains severely limited. Here, we critically evaluate these limitations, highlighting empirical findings where intrinsic self-correction fails and demonstrating the necessity of coupling models with stronger verifiers or the external ground truth mechanisms discussed previously.

A primary limitation of intrinsic self-verification is the prevalence of “self-bias,” where models exhibit a tendency to favor their own initial generations even when they are erroneous. Research indicates that when LLMs attempt to refine their outputs based on self-generated feedback, they often amplify their own biases rather than correcting mistakes. For instance, studies have shown that self-feedback loops can worsen performance on specific tasks because the model struggles to objectively distinguish between correct and incorrect reasoning paths without an external anchor [192]. This phenomenon is particularly pronounced in complex reasoning domains such as mathematics and code generation, where the logical steps required for verification are as demanding as those for generation. Consequently, without the rigorous test cases or formal specifications described in Section 4.5, the model may confidently reaffirm a flawed patch, leading to a degradation in overall repair success rates.

Furthermore, the efficacy of self-correction is highly dependent on the model’s ability to detect logical fallacies, a skill that current LLMs often lack. Empirical evaluations suggest that while models can sometimes correct surface-level syntax errors or stylistic inconsistencies, they struggle profoundly with identifying deep logical mistakes in their own reasoning traces. In the context of logical reasoning, models frequently fail to pinpoint the specific step where a derivation goes wrong, rendering the subsequent correction attempt ineffective or misdirected [155]. This inability to localize errors internally means that intrinsic self-correction often operates as a blind search, lacking the precise diagnostic information needed to guide effective repairs. The situation is exacerbated by the fact that models often generate plausible but unfaithful explanations for their decisions,

creating an illusion of understanding that masks underlying reasoning failures [193]. Without external signals—such as the counter-examples or invariant violations generated by the methods in Section 4.5—the model remains trapped in its own hallucinated logic.

The limitations of intrinsic verification are not uniform across all model scales or task types, but the consensus indicates that smaller models, in particular, lack the emergent capabilities required for reliable self-critique. Research focusing on models with fewer than 13 billion parameters demonstrates that these systems generally cannot self-correct reasoning tasks effectively without guidance from stronger models [159]. Even for larger models, the process of self-reflection can be counterproductive if the model’s initial confidence is misplaced. Studies have observed that self-reflection can adversely affect performance on complex query tasks, especially when the model’s initial response is already close to correct, leading to unnecessary and harmful modifications [194]. Moreover, the content of the self-critique often matters less than the mere presence of a verification signal, suggesting that the internal reasoning of the critic model is not the driving force behind any observed improvements [157].

Given these inherent limitations, the role of strong verifiers becomes paramount in constructing robust automated debugging systems. The integration of external, sound verifiers—whether they are larger, more capable LLMs, formal solvers, or the execution-based test suites and synthesized specifications detailed in Section 4.5—provides the necessary ground truth that intrinsic methods lack. Empirical evidence strongly supports the hypothesis that verification is easier than generation only when the verifier possesses superior capabilities or access to external tools. For example, systems that employ a strong external verifier to judge the correctness of candidate solutions significantly outperform those relying on self-critique, with the verifier’s accuracy being a critical determinant of the system’s overall success [157]. The use of a strong verifier allows the system to filter out hallucinated patches and select only those that satisfy rigorous logical or execution constraints, thereby mitigating the risk of overfitting or introducing new bugs.

Moreover, the synergy between generator and verifier models can be optimized through iterative training frameworks that leverage both correct and incorrect solutions. Approaches such as V-STaR demonstrate that training a dedicated verifier model using data from the self-improvement process can lead to progressively better reasoners and verifiers, effectively closing the gap between generation and verification capabilities [195]. This decoupling of roles allows the generator to focus on creative patch synthesis while the verifier specializes in rigorous validation, a division of labor that mirrors effective human debugging practices. Additionally, techniques that incorporate confidence estimation and calibration help determine when a model should defer to a stronger verifier, preventing the amplification of errors during the self-correction loop [151].

In conclusion, while intrinsic self-verification offers a theoretically appealing path toward autonomous repair, current empirical findings underscore its fragility and unreliability in isolation. The tendency of LLMs to hallucinate, exhibit self-bias, and fail in logical error detection necessitates a paradigm shift toward hybrid architectures. Future advancements in automated program repair must prioritize the integration of strong, external verifiers and neuro-symbolic hybrids that combine the generative flexibility of LLMs with the deterministic rigor of the formal verification and execution-based testing strategies outlined in the preceding section. Only by anchoring the self-correction process in external ground truth can we hope to achieve the levels of reliability and correctness required for deploying autonomous debugging agents in real-world software engineering contexts.

5 Specialized Applications Across Domains and Languages

5.1 Hardware Security and Verilog Repair

5.1 Hardware Security and Verilog Repair

While the following section addresses the unique immutability of blockchain bytecode, the domain of hardware design presents an even more stringent constraint: the physical permanence of silicon. Unlike software, where bugs can often be patched post-deployment via updates, hardware vulnerabilities discovered after fabrication can lead to costly product recalls or permanent security breaches. Consequently, the application of Large Language Models (LLMs) to Register Transfer Level (RTL) code represents a paradigm shift focused almost exclusively on pre-silicon detection and remediation. Recent research underscores that while general-purpose LLMs possess foundational coding capabilities, they require specialized adaptation to effectively handle the nuances of Hardware Description Languages (HDLs) like Verilog and SystemVerilog, particularly concerning security vulnerabilities.

A primary challenge in this domain is the scarcity of high-quality, security-focused hardware datasets compared to software repositories. To address this, researchers have developed frameworks that leverage version control data to compile datasets of open-source hardware design defects and their corresponding remediation steps. For instance, the LLM4SecHW framework utilizes such datasets to fine-tune medium-sized LLMs, enabling them to accurately identify and rectify bugs in hardware designs [196]. This approach highlights the necessity of domain-specific fine-tuning, as generic models often lack the contextual understanding required for hardware security logic. Similarly, other studies have constructed corpora of domain-representative hardware security bugs to quantitatively evaluate LLM performance, showing that an ensemble of LLMs can successfully repair benchmarks that stump traditional automated repair tools [197].

The efficacy of these models is significantly enhanced through advanced prompt engineering and iterative feedback loops, mirroring the “tool-in-the-loop” strategies seen in other automated debugging contexts. Simple zero-shot prompting often yields syntactically correct but functionally insecure code. To mitigate this, frameworks like AutoChip combine the interactive capabilities of LLMs with output from Verilog simulations and compiler error messages. By feeding compilation errors and debugging context back into the model, AutoChip guides the LLM to generate functionally accurate Verilog modules without human intervention, achieving a substantial increase in accuracy over native generation [198]. Furthermore, the RTLFixer framework employs Retrieval-Augmented Generation (RAG) and ReAct prompting to act as an autonomous agent that interactively debugs syntax errors, successfully correcting approximately 98.5% of compilation errors in debugging datasets [199]. These methodologies underscore a critical transition from static code generation to dynamic, feedback-driven repair cycles essential for complex hardware designs.

Security verification in hardware also extends beyond simple bug fixing to the proactive detection of Common Weakness Enumerations (CWEs). Studies indicate that standard LLMs are often unaware of specific hardware CWEs, leading to the generation of vulnerable designs in approximately 60% of cases [200]. To counter this, researchers have proposed frameworks like DIVAS, which leverages LLMs to identify security vulnerabilities from System-on-Chip (SoC) specifications, map them to relevant CWEs, and generate equivalent assertions for formal verification [201]. Additionally, the generation of hardware assertions—a critical component of security verification—has been automated using LLMs, where natural language prompts derived from code comments are translated into SystemVerilog assertions, thereby strengthening the verification backbone of hardware designs

[202].

The landscape of hardware security also involves addressing sophisticated threats such as Hardware Trojans (HTs) and side-channel leakages. While HT insertion during manufacturing remains a critical threat, LLMs are being explored to analyze layout susceptibility and understand the complex interplay of placement and routing resources that facilitate covert trojan insertion [203]. In the realm of side-channel attacks, LLMs have shown promise in generating patches for microarchitectural leakages. The ZeroLeak framework demonstrates that LLMs, guided by carefully crafted zero-shot prompts and dynamic analysis tools, can generate code replacements that are resilient to information leakage at a fraction of the cost of manual expert remediation [204]. This cost-effectiveness is crucial for scaling security practices across large codebases where expert resources are limited.

Despite these advancements, significant gaps remain. The creativity and optimization capabilities of LLMs in hardware design are still being quantified, with frameworks like CreativeEval attempting to measure originality and flexibility beyond mere functional correctness [205]. Moreover, the reliability of LLMs in security contexts is not absolute; evaluations reveal that models can struggle with vulnerability localization and may produce non-deterministic responses when faced with varying code structures [206]. Consequently, the future of hardware security with LLMs lies in hybrid approaches that combine the generative power of fine-tuned models with rigorous formal verification and tool-in-the-loop feedback, ensuring that the transition from natural language specifications to secure silicon is both efficient and trustworthy [207]. As the field evolves from manual, error-prone design processes toward automated, self-healing hardware ecosystems, these specialized LLM applications provide a foundational counterpart to the bytecode-level innovations discussed in the subsequent section on smart contracts.

5.2 Smart Contract Auditing and Bytecode Patching

5.2 Smart Contract Auditing and Bytecode Patching

While the preceding section highlighted the physical permanence of silicon in hardware design, the blockchain ecosystem presents a distinct form of immutability: the cryptographic finality of deployed bytecode. In this domain, the cost of failure is not merely a product recall but often irreversible financial loss, with smart contract vulnerabilities leading to billions of dollars in damages. Consequently, automated program repair in blockchain has bifurcated into two critical streams: source-code auditing for pre-deployment verification and bytecode-level patching for post-deployment mitigation. This duality is essential because, unlike the hardware domain where pre-silicon verification is the sole defense, blockchain faces the unique constraint that a vast majority of deployed contracts lack publicly available source code, necessitating repair strategies that operate directly on binary representations.

In the realm of source-code auditing, traditional static analysis and symbolic execution tools have long served as the first line of defense. Comprehensive studies have evaluated numerous vulnerability scanners, revealing significant discrepancies in their findings and often poor performance when validated against ground truth datasets [208]. While tools like Slither, Mythril, and Oyente have become industry standards, empirical assessments indicate that they frequently miss elusive faults or generate high false-positive rates, leaving many critical vulnerabilities undetected [209]. To address these limitations, recent research has increasingly turned to Large Language Models (LLMs) to enhance auditing capabilities. However, the application of off-the-shelf LLMs such as GPT-4 has shown mixed results; while they excel at code parsing and generating Proof of Concepts (PoCs),

their raw performance in vulnerability detection often suffers from low recall, missing a substantial portion of existing bugs [210].

To bridge the gap between general-purpose LLMs and the specialized requirements of smart contract security, novel frameworks have emerged that combine fine-tuning with agentic reasoning. For instance, the TrustLLM framework introduces a two-stage approach where a detector model identifies potential issues, followed by a reasoner model that generates justifications, further refined by ranker and critic agents to debate and select the optimal cause of vulnerabilities [211]. Similarly, other approaches leverage Chain-of-Thought prompting to simulate professional auditor workflows, significantly improving the precision of vulnerability identification compared to standard prompting techniques [210]. Furthermore, LLMs are being integrated into fuzzing campaigns to intelligently guide input generation towards high-value code regions, as demonstrated by LLM4Fuzz, which uncovered critical vulnerabilities in DeFi projects that traditional fuzzers missed [212]. These advancements signify a paradigm shift from purely rule-based detection to semantic-aware, reasoning-driven auditing, mirroring the move toward agent-based debugging seen in hardware contexts.

Despite improvements in pre-deployment auditing, the inability to patch deployed contracts directly remains a unique challenge, as the bytecode on the blockchain is immutable. With only a fraction of deployed contracts having their source code publicly verified, relying solely on source-level fixes is insufficient for the vast majority of the ecosystem. This has necessitated the development of bytecode-level repair strategies, a task even more constrained than the RTL repair discussed previously due to the loss of high-level semantic information during compilation. Early approaches to this problem relied heavily on hard-coded templates to rewrite specific vulnerability patterns, such as integer overflows or access control errors, directly into the EVM bytecode. Tools like EVMPatch demonstrated the feasibility of automatically transforming vulnerable contracts into upgradable versions, successfully blocking attack transactions while preserving functionality [213]. However, template-based methods often lack the flexibility to adapt to complex, context-dependent bytecode structures.

Recent innovations have sought to overcome the rigidity of template-based systems by incorporating semantic analysis into bytecode patching. The Elysium framework represents a significant advancement in this area by combining template-based and semantic-based patching techniques. By inferring context information directly from the bytecode, Elysium can dynamically adapt its repair strategies, outperforming existing tools in correctly patching a wider variety of vulnerability types while minimizing runtime transaction costs [214]. Another notable approach, Two Timin', proposes a two-layered framework that first classifies malicious contracts using machine learning and then employs fine-tuned LLMs to directly repair the identified vulnerabilities, achieving a drastic reduction in vulnerability counts for both source and bytecode scenarios [215].

The scarcity of open-source code has also spurred research into detection and repair methods that operate exclusively on bytecode or binary representations. Techniques utilizing structural code embeddings, such as SmartEmbed, allow for the detection of clone-related bugs by comparing vector representations of code elements, effectively identifying vulnerabilities even without access to the original source logic [216]. Additionally, dynamic analysis tools like ÆGIS provide runtime protection by monitoring execution flows against defined attack patterns, offering a shield for vulnerable contracts that cannot be statically patched [217]. The evolution from static, rule-based scanners to adaptive, LLM-driven auditors and semantic-aware bytecode patchers illustrates a maturing field capable of addressing the dual challenges of pre-deployment correctness and post-deployment resilience. As the complexity of decentralized systems grows, the integration of neuro-symbolic

hybrids and autonomous agents will likely become essential, setting a precedent for handling immutability that contrasts with the proactive cryptographic migration strategies discussed in the subsequent section on quantum software engineering.

5.3 Quantum Software Engineering and Post-Quantum Cryptography

5.3 Quantum Software Engineering and Post-Quantum Cryptography

While Section 5.2 addressed the immediate immutability challenges of blockchain bytecode, the horizon of software security faces an even more fundamental shift: the advent of quantum computing. The intersection of artificial intelligence, specifically Code Large Language Models (LLMs), and quantum computing represents a critical frontier in software engineering. As the industry transitions from the theoretical potential of quantum mechanics to the practical realities of the Noisy Intermediate-Scale Quantum (NISQ) era, the role of LLMs has emerged as dual-faceted: acting as both accelerators for migrating classical infrastructure to post-quantum cryptographic (PQC) standards and as intelligent assistants for the development and debugging of fragile quantum algorithms. This subsection investigates these two pivotal domains, analyzing how LLM-driven program repair and automated debugging techniques are being adapted to address the unique constraints of quantum hardware and the urgent security demands of cryptanalytic threats.

The first major application lies in the migration of classical systems to post-quantum cryptographic standards, a task that parallels the proactive auditing strategies discussed previously but operates on a global cryptographic scale. The advent of sufficiently powerful quantum computers poses an existential threat to widely deployed public-key cryptosystems such as RSA and ECC, necessitating a proactive shift toward quantum-resistant algorithms [218]. Code LLMs are uniquely positioned to facilitate this massive engineering undertaking by automating the refactoring of legacy codebases. These models can be tasked with identifying vulnerable cryptographic primitives and synthesizing replacements based on newly standardized algorithms like ML-KEM (formerly Kyber). Recent advancements demonstrate that optimizing these new algorithms for performance is non-trivial; for instance, leveraging advanced instruction sets like AVX-512 to parallelize polynomial multiplication in ML-KEM can yield significant speedups, a complex optimization task that LLMs can assist in generating and verifying [219]. Furthermore, the integration of PQC into existing protocols like TLS 1.3 requires careful handling of hybrid modes to ensure backward compatibility while maintaining security against “steal now, crack later” attacks [220]. LLMs can automate the synthesis of these hybrid protocol implementations, ensuring that the nested modes of operation comply with rigorous standards such as FIPS while minimizing performance overheads in handshake phases.

Beyond cryptographic migration, the second critical role of LLMs involves the software engineering lifecycle of quantum algorithms themselves, where debugging transcends logical errors to encompass physical hardware limitations. Developing software for NISQ devices is notoriously difficult due to hardware noise, decoherence, and severe resource constraints [221]. Unlike classical code, where bugs often result in crashes or logical errors, bugs in quantum programs may manifest as subtle fidelity drops or complete loss of quantum advantage due to environmental interference. Code LLMs, trained on quantum assembly languages like OpenQASM and high-level frameworks like Qiskit, can assist developers in debugging these issues by suggesting error-mitigation strategies and optimizing circuit topologies. For example, LLMs can recommend specific compilation passes that account for dynamic hardware calibration data, effectively mapping logical qubits to physical qubits with lower error rates to maximize program success [222].

The debugging process in quantum software engineering also extends to the optimization of varia-

tional quantum algorithms (VQAs), which are central to many NISQ applications. These algorithms rely on parameterized quantum circuits where the parameters are tuned via classical optimization loops. LLMs can analyze the structure of these circuits to propose ansatz improvements or identify redundancies that contribute to decoherence. Research indicates that techniques such as delaying qubits into superposition can significantly reduce error rates by minimizing qubit lifetime, a strategy that automated tools powered by LLMs could identify and implement within the source code [223]. Moreover, as quantum machine learning models become more complex, ensuring their robustness against adversarial attacks becomes paramount. Experimental demonstrations have shown that quantum classifiers are vulnerable to adversarial perturbations, similar to their classical counterparts [224]. LLMs can be employed to generate adversarial examples for testing purposes or to synthesize defensive code modifications, such as adversarial training routines, thereby enhancing the security and reliability of quantum AI systems.

Furthermore, the verification of quantum code presents a unique challenge that LLMs can help address through the generation of test suites and the interpretation of simulation results, bridging the gap between abstract algorithm design and concrete hardware execution. Given that classical simulation of large quantum circuits is computationally prohibitive, hybrid approaches that cut large circuits into smaller subcircuits for evaluation are essential [225]. LLMs can automate the decomposition logic required for such hybrid evaluations, ensuring that the reconstructed outputs maintain high fidelity. Additionally, the explainability of quantum circuits is an emerging concern; LLMs can assist in generating natural language explanations for why specific gate configurations were chosen or how certain entanglement patterns contribute to the overall algorithmic goal, bridging the gap between low-level quantum operations and high-level software intent [226].

In conclusion, the synergy between Code LLMs and quantum software engineering is transforming how developers approach both the defensive hardening of classical systems against quantum threats and the constructive creation of native quantum applications. By automating the migration to post-quantum cryptography and providing intelligent support for debugging noise-sensitive quantum algorithms on NISQ devices, LLMs are becoming indispensable tools in the quest for quantum advantage. This evolution from cryptographic migration to low-level circuit optimization sets the stage for the subsequent discussion on microarchitectural vulnerabilities, as the principles of automated remediation extend further into the hardware-software interface to address side-channel leakages and access control flaws in Section 5.4.

5.4 Side-Channel Mitigation and Access Control Remediation

5.4 Side-Channel Mitigation and Access Control Remediation

Building upon the discussion of cryptographic migration and quantum algorithm debugging in Section 5.3, the focus now shifts to the immediate hardware-software interface and the logical governance of decentralized systems. While quantum concerns address future existential threats, microarchitectural side-channel leakages and complex Role-Based Access Control (RBAC) flaws represent persistent, high-stakes vulnerabilities in current infrastructure. These domains have historically demanded deep, specialized human expertise for diagnosis and remediation, often resulting in a critical backlog of unpatched systems due to resource constraints. Recent advancements demonstrate that automated, cost-effective approaches leveraging Large Language Models (LLMs) can now bridge this gap, offering scalable solutions for both physical implementation vulnerabilities and logical access control failures. This evolution from quantum-resistant cryptography to low-level leakage mitigation sets a foundation for understanding the broader security challenges in

Cyber-Physical Systems discussed in Section 5.5.

Microarchitectural side-channel attacks exploit the physical implementation details of processors—such as cache timing, branch prediction, and power consumption—to extract secret information like cryptographic keys. The complexity of mitigating these issues arises from the subtle interplay between software logic and hardware behavior, where even “constant-time” code can leak information through unintended microarchitectural interactions. Traditional mitigation strategies often involve manual code rewriting or the use of domain-specific languages, which are labor-intensive and prone to error. However, emerging frameworks leverage LLMs to automate the detection and patching of these leakages with remarkable efficiency. For instance, the [204] framework illustrates a paradigm shift by utilizing powerful LLMs in a zero-shot learning configuration to generate patches for vulnerable code. By carefully crafting prompts that guide the model to replace vulnerable instruction sequences with leakage-resilient alternatives, [204] demonstrates that LLM-based patching can be achieved at a fraction of the cost of traditional expert review, costing only a few cents per vulnerability fixed via API calls. This approach is particularly vital given that security-critical software often contains numerous unpatched side-channel leakages due to a lack of specialized resources.

Furthermore, the automation of side-channel mitigation extends beyond simple text generation to include rigorous formal verification and statistical analysis, creating a robust pipeline for securing cryptographic implementations. Tools like [227] and its predecessor [228] exemplify the integration of automated code rewrite engines with leakage emulators. These systems do not merely suggest fixes; they iteratively detect higher-order leakages in masked implementations and automatically apply code transformations to eliminate them, ensuring that the resulting binaries are secure against sophisticated power analysis attacks. Similarly, [229] introduces a pre-silicon analysis framework that combines Graph Neural Networks with LLMs to not only localize potential leakages in hardware designs but also to automatically generate and insert fortification code. This synergy between deep learning for localization and LLMs for code synthesis represents a significant advancement in securing hardware descriptions before fabrication, addressing vulnerabilities in algorithms ranging from AES to post-quantum cryptography standards.

Parallel to the challenges in hardware security, the domain of decentralized applications faces critical challenges regarding access control, echoing the immutability concerns raised in earlier sections on blockchain. In blockchain ecosystems, smart contracts manage vast financial assets, making flaws in Role-Based Access Control (RBAC) policies particularly devastating. Unlike reentrancy bugs which can often be fixed with templates, RBAC vulnerabilities require a nuanced understanding of the contract’s business logic to assign the correct roles and permissions. The [230] system addresses this by mining a taxonomy of common RBAC practices from hundreds of thousands of on-chain contracts. By integrating this mined knowledge with a Chain-of-Thought reasoning pipeline, [230] guides LLMs to identify the appropriate role-permission pairs for specific contexts, achieving a repair success rate of nearly 95% on real-world vulnerabilities, a substantial improvement over baseline models. This context-aware repair is essential because generic LLMs often fail to grasp the intricate permission hierarchies required in decentralized finance (DeFi) protocols.

The necessity for such advanced automated remediation is underscored by the limitations of current security tooling. Studies such as [231] reveal that existing automated scanners detect only a small fraction of high-impact attacks, particularly those involving complex logic flaws rather than simple pattern matches. Consequently, the integration of LLM-driven repair mechanisms offers a path forward where the gap between detection and remediation is closed. Moreover, the principles of RBAC are being reimaged through deep learning, as seen in proposals like [232] and [233], which suggest moving away from static, manually engineered policies toward dynamic, neural-network-

driven access control systems that can adapt to evolving threat landscapes.

In summary, the convergence of LLMs, formal methods, and domain-specific mining techniques is transforming the landscape of vulnerability remediation. Whether it is the automated elimination of microarchitectural leakages in cryptographic libraries as demonstrated by [204] and [227], or the context-sensitive repair of RBAC flaws in smart contracts via [230], these approaches provide cost-effective, scalable, and highly accurate solutions. As software systems grow in complexity and the attack surface expands, these automated paradigms become indispensable for maintaining the integrity of critical infrastructure. This transition from code-level patching and logical access repair naturally leads to the next frontier: securing the physical world itself, where software faults in Cyber-Physical Systems can result in catastrophic real-world consequences, as explored in Section 5.5.

5.5 Cyber-Physical Systems and Distributed Infrastructure Defense

5.5 Cyber-Physical Systems and Distributed Infrastructure Defense

While the preceding sections addressed vulnerabilities rooted in microarchitectural leakages and logical access control flaws, the security landscape expands further when considering Cyber-Physical Systems (CPS) and distributed infrastructure. In these domains, the stakes are elevated beyond data confidentiality or financial loss; software faults can precipitate catastrophic physical consequences. The integration of Large Language Models (LLMs) into the security paradigm of CPS—encompassing Networked-Control Systems (NCS), smart grids, and autonomous vehicle fleets—represents a critical frontier in automated debugging and program repair. Unlike traditional software environments, CPS operate within tightly coupled physical and cyber realms where the adaptation of LLMs necessitates a shift from mere syntax correction to the synthesis of resilient control policies. These policies must withstand autonomous cyber-attacks and ensure system stability under adversarial conditions, demanding specialized debugging agents capable of reasoning about both logical code correctness and complex physical system dynamics, including real-time constraints and heterogeneous subsystem interactions.

A primary application of LLM-driven repair in this domain involves the hardening of NCS against sophisticated attack vectors such as Denial-of-Service (DoS) and False Data Injection (FDI). Traditional defense mechanisms often rely on static models that struggle to adapt to the evolving tactics of adversaries. However, recent research envisions a paradigm shift where defenders leverage machine learning and quantum-inspired strategies to anticipate and recover from attacks, ensuring that an increase in adversarial resources does not translate into a higher likelihood of disruptions [234]. LLMs can be deployed to analyze complex attack graphs and synthesize adaptive control policies that dynamically reconfigure network topologies or switch control modes in response to detected anomalies. For instance, in scenarios involving DoS attacks, LLM agents could assist in designing switching LPV (Linear Parameter-Varying) controllers or determining optimal trade-offs between jamming actions and network delays to maintain exponential stability [235]. Furthermore, the ability of LLMs to process vast amounts of historical incident data allows them to identify subtle patterns indicative of impending failures, facilitating proactive patching before physical safety is compromised.

The distributed nature of modern infrastructure introduces the challenge of cascading failures, where a fault in one subsystem can trigger a domino effect across the entire network. Ensuring reliability in such interdependent systems requires debugging tools that understand the topology

and coupling dynamics of the infrastructure. LLMs are increasingly being explored for their potential to coordinate resilient consensus algorithms and optimize the allocation of security resources across distributed nodes. Game-theoretic approaches, such as Colonel Blotto games, have been proposed to model the interaction between attackers and defenders in smart cities, and LLMs can serve as strategic agents within these frameworks to compute optimal defense distributions [236]. By simulating various attack scenarios, LLM-based agents can generate “moving target” defenses that continuously shift system configurations, thereby reducing the attack surface and confusing autonomous adversaries [237]. This capability is particularly vital for protecting critical assets in industrial control systems (ICS) and energy grids, where the cost of failure is exceptionally high.

Moreover, the deployment of LLMs in CPS extends to the realm of autonomous cyber-defense, where agents must operate with a degree of independence to counter attacks occurring at machine speed. While autonomous agents offer the promise of real-time response, they also introduce risks of unintended harm if their decision-making processes are not rigorously verified [238]. Consequently, the integration of LLMs into CPS defense architectures must be accompanied by robust verification mechanisms, such as control barrier certificates, to guarantee that generated patches or control actions do not violate safety constraints [239]. The concept of “graceful degradation” is central to this effort; LLMs can be trained to recognize when a system is under severe stress and automatically synthesize code that weakens non-essential requirements to maintain core safety functions, subsequently coordinating recovery once the threat subsides [240].

Despite these promising avenues, significant challenges remain in ensuring the reliability of LLM-driven repairs in distributed infrastructure. The opacity of deep learning models poses a barrier to trust, especially in high-stakes environments where explainability is mandated by safety standards. Additionally, the potential for LLMs to be exploited by adversaries to automate post-breach attacks or synthesize novel exploits underscores the dual-use nature of these technologies [241]. Research has shown that LLM agents can autonomously exploit vulnerabilities in real-world systems, highlighting the urgent need for defensive LLMs that are robust against prompt injection and manipulation [242]. To address these vulnerabilities, future research must focus on developing neuro-symbolic hybrids that combine the generative creativity of LLMs with the formal guarantees of symbolic verification, ensuring that automated repairs in CPS are not only effective but also provably safe. The evolution of testbeds like ICSSIM, which allow for the safe simulation of cyber-attacks on industrial systems, will be instrumental in training and evaluating these next-generation debugging agents without risking physical infrastructure [243]. Ultimately, the successful adaptation of LLMs to CPS defense hinges on a holistic approach that integrates advanced threat modeling, resilient control theory, and rigorous safety validation to create self-healing ecosystems capable of surviving in an era of autonomous cyber-warfare, setting the stage for securing the data-intensive layers discussed in subsequent database systems.

5.6 Database Security and Query Optimization

5.6 Database Security and Query Optimization

Bridging the gap between the physical resilience required in Cyber-Physical Systems and the broader architectural vulnerabilities of LLM-integrated applications, the domain of database management presents a unique intersection of data integrity, query optimization, and security. While the preceding section highlighted the catastrophic physical consequences of faults in distributed infrastructure, this section focuses on the data-intensive layer where software faults manifest as privacy leaks, injection attacks, and resource inefficiencies. The integration of Large Language Models (LLMs) into

database management systems has ushered in a new era of intelligent data interaction, fundamentally altering how developers and analysts engage with complex data repositories. This convergence offers transformative potential for query optimization and natural language interfaces, yet it simultaneously introduces a sophisticated attack surface characterized by novel injection vectors and privacy concerns. As LLMs are increasingly deployed as middleware to translate natural language into structured query languages like SQL, the imperative to secure these pipelines against adversarial manipulation while maintaining high performance has become a critical research frontier.

A primary application of LLMs in this domain is the automation of Text-to-SQL translation, enabling users to query databases using natural language without requiring deep syntactic knowledge of SQL. However, the reliance on LLMs for this task introduces significant security risks, particularly regarding prompt injection attacks that can escalate into SQL injection vulnerabilities. Research has demonstrated that unsanitized user prompts, when processed by LLM-integrated middleware such as Langchain, can be manipulated to generate malicious SQL queries, effectively compromising the underlying database security [244]. These “Prompt-to-SQL” (P₂SQL) attacks exploit the model’s instruction-following capabilities to bypass traditional input sanitization mechanisms, highlighting the pervasiveness of this threat across various state-of-the-art language models. To mitigate these risks, recent studies have proposed structured defense mechanisms, such as separating prompts and data into distinct channels through structured queries, which significantly improves resistance to injection attacks with minimal impact on utility [245]. Furthermore, innovative prompt engineering techniques like “spotlighting” have been developed to help models distinguish between trusted instructions and untrusted data sources, reducing attack success rates from over 50% to below 2% in experimental settings [246].

Beyond immediate injection threats, the architectural integration of LLMs into database systems raises profound concerns regarding data privacy and leakage, especially in Retrieval-Augmented Generation (RAG) scenarios where models access external knowledge bases. Adversaries can exploit the instruction-following nature of LLMs to extract sensitive data verbatim from the datastore of RAG systems, a vulnerability that exacerbates as model scale increases [247]. This risk is compounded by the finding that LLM-based embeddings can inadvertently leak private information, allowing for the reconstruction of original text or the prediction of entity attributes from vector representations [248]. Consequently, securing LLM-integrated applications requires a holistic approach that addresses not only the model but also the surrounding ecosystem, including the frontend, web tools, and sandbox environments, to prevent illicit acquisition of user chat history and sensitive data [249]. To counteract these threats, researchers have defined key properties for safe LLM-integrated applications, including integrity, source identification, and attack detectability, proposing lightweight, threat-agnostic defenses that mitigate both insider and outsider attacks [250].

In parallel with security enhancements, significant efforts have been directed toward optimizing the performance and cost-efficiency of LLM-driven database interactions. The computational overhead of invoking LLMs for every query poses a substantial barrier to adoption, with studies indicating that using LLMs for database queries can incur significant energy overhead, making them environmentally unfavorable compared to traditional relational database engines for simple tasks [251]. To address this, novel optimization strategies have been proposed to accelerate LLM inference within analytical workloads. These include reordering rows and columns to maximize key-value cache reuse and deduplicating redundant inference requests, techniques that have achieved up to a 4.4x improvement in end-to-end latency when implemented in systems like Apache Spark [252]. Additionally, advanced frameworks like PURPLE (Pre-trained models Utilized to Retrieve Prompts

for Logical Enhancement) have been developed to improve the accuracy of NL2SQL translation by retrieving demonstrations containing requisite logical operator compositions, thereby guiding LLMs to produce more complex and correct SQL queries without extensive fine-tuning [253].

The pursuit of efficient and secure database management has also led to the development of specialized, domain-adapted models. Systems like DB-GPT integrate private LLMs fine-tuned on domain-specific corpora to ensure data security while providing context-aware responses and high-accuracy SQL generation [254]. Similarly, approaches like LlamaTune leverage domain knowledge and automated dimensionality reduction to improve the sample efficiency of database configuration tuning, identifying optimal configurations with significantly fewer workload runs [255]. Despite these advancements, the challenge of hallucination remains a persistent obstacle, where LLMs may generate plausible but incorrect queries or analyses. To combat this, paradigms like LLMDB have been designed to embed domain-specific knowledge and utilize vector databases for semantic search, effectively reducing hallucinations and improving task accuracy in database diagnosis and query rewriting [256]. Furthermore, the exploration of smaller, cognitively enhanced models offers a pathway to deploying secure, local database agents on resource-constrained devices, balancing performance with the need for data privacy [257].

Ultimately, the future of database security and query optimization lies in the synergistic development of robust defense mechanisms and efficient inference architectures. As the landscape evolves, the community must remain vigilant against emerging threats such as knowledge poisoning in RAG systems, where attackers inject poisoned texts to manipulate model outputs [258]. By integrating rigorous security protocols with performance optimization techniques, the field can move toward realizing trustworthy, autonomous database agents capable of navigating the complexities of modern data ecosystems. However, as database interactions become just one component of larger software stacks, the scope of vulnerability expands further. The specific risks identified here—such as prompt injection and data poisoning—serve as foundational elements for understanding the broader, more complex security challenges inherent in fully LLM-integrated applications, which are discussed in the subsequent section.

5.7 Vulnerabilities Within LLM-Integrated Applications

5.7 Vulnerabilities Within LLM-Integrated Applications

Building upon the specific challenges of database security and query optimization discussed in the previous section, the integration of Large Language Models (LLMs) into broader software ecosystems introduces a more expansive architectural paradigm known as LLM-integrated applications. In this setup, the LLM functions not merely as a standalone interface but as critical middleware that refines user queries, accesses domain-specific knowledge, and orchestrates dynamic responses for downstream services. While this evolution unlocks unprecedented capabilities in code completion, automated search, and conversational interfaces, it simultaneously expands the attack surface beyond isolated database interactions. Unlike traditional software vulnerabilities rooted in memory safety or deterministic logic errors, the security challenges in LLM-integrated applications stem from the probabilistic nature of language generation and the increasingly blurred boundaries between data and executable instructions. This subsection analyzes the unique security threats inherent to these compositional systems, with a particular focus on prompt injection, data poisoning, and the fragility of query-response protocols across the entire application stack.

A primary vulnerability in this landscape is the susceptibility to prompt injection attacks, where adversaries manipulate the model’s input to override original system instructions. This threat is

particularly acute when LLMs serve as middleware processing untrusted data retrieved from external sources alongside user commands. Research has demonstrated that attackers can exploit this by embedding malicious instructions within data fields, a technique known as indirect prompt injection. For instance, [259] reveals how adversaries can remotely exploit applications by injecting prompts into data likely to be retrieved, effectively causing the LLM to execute arbitrary code or manipulate API calls without direct user interaction. This erosion of the distinction between data and instructions allows attackers to bypass safety guardrails, leading to severe outcomes such as data theft and information ecosystem contamination. Furthermore, [260] introduces “HouYi,” a novel black-box attack technique that leverages context partitioning to achieve unrestricted arbitrary LLM usage and prompt theft, validating these risks across dozens of real-world commercial applications.

The integrity of the query-response protocol in LLM-integrated systems represents another critical concern, extending the “Prompt-to-SQL” risks previously identified to a wider range of backend interactions. These applications often rely on a communication round-trip where user queries are enriched by the middleware before reaching the backend LLM. [250] highlights that this protocol is vulnerable to both insider threats from malicious developers and outsider threats capable of poisoning the database access. Successful exploits can result in users receiving responses tailored to an attacker’s intent, including biased, toxic, or disinformation-laden content that bypasses the moderation policies of major providers like OpenAI. The complexity is further compounded when LLMs are integrated with web frameworks that translate natural language into various executable actions. [244] specifically examines the “prompt-to-SQL” injection vector, demonstrating how unsanitized prompts in frameworks like Langchain can lead to traditional SQL injection attacks, thereby compromising the underlying database security. This convergence of natural language vulnerabilities with classical web exploits creates a multifaceted threat landscape that traditional security tools are ill-equipped to handle.

Beyond direct interaction attacks, the supply chain and training integrity of LLM-integrated applications face significant risks from data poisoning, a threat that permeates the Retrieval-Augmented Generation (RAG) mechanisms discussed earlier. As these systems increasingly rely on RAG to access up-to-date information, the trustworthiness of the retrieved content becomes paramount. [261] describes a novel threat where attackers craft documents that are visually indistinguishable from benign ones but contain hidden triggers that mislead the RAG process, causing the application to generate malicious responses with high success rates. Additionally, the fine-tuning and instruction-tuning phases of model deployment present opportunities for backdoor insertion. [262] illustrates how adversaries can insert gradient-guided backdoor triggers into a small fraction of instruction tuning samples, causing the model to malfunction or produce compromised outputs when specific triggers are present, all while maintaining normal performance on clean inputs. Similarly, [263] extends this analysis to agentic systems, showing how backdoors can be hidden in user queries or intermediate observations to manipulate the reasoning processes of autonomous agents.

The systemic nature of these vulnerabilities suggests that securing LLM-integrated applications requires a holistic approach that goes beyond patching individual models or optimizing specific database queries. [249] argues that LLM systems are inherently compositional, involving layers such as frontends, sandboxes, and plugins, each introducing potential constraint failures. Their multi-layer security analysis of systems like GPT-4 exposes vulnerabilities not just within the model but in the alignment of information flow between the LLM and its integrated components. Moreover, the ethical implications of these security failures are profound. [264] underscores that threats like PII exposure and hate speech generation are not merely technical glitches but ethical crises that demand

robust defensive strategies aligned with societal values. To address these challenges, researchers propose various mitigation frameworks, such as structured queries that separate prompts from data [245] and instruction hierarchies that prioritize privileged system commands over user input [265]. However, as noted in [266], a comprehensive risk assessment process is essential for stakeholders to understand the severity of consequences and deploy effective mitigation strategies across the development lifecycle. Ultimately, the security of LLM-integrated applications hinges on redefining trust boundaries in an era where natural language serves as both the interface and the potential weapon, setting the stage for exploring automated debugging techniques to detect and repair these complex vulnerabilities.

6 Evaluation Landscapes: Benchmarks, Metrics, and Data Integrity

6.1 The Evolution and Limitations of Standard Benchmarks

6.1 The Evolution and Limitations of Standard Benchmarks

The trajectory of research in Automated Program Repair (APR) and fault localization has been inextricably linked to the availability and quality of benchmark datasets. For over a decade, the field has been dominated by a small set of legacy benchmarks, most notably Defects4J and QuixBugs, which have served as the de facto standards for evaluating the efficacy of repair techniques. While these datasets were instrumental in catalyzing early breakthroughs and enabling fair comparisons between competing approaches, their continued dominance has begun to reveal significant limitations. As software engineering practices evolve rapidly with the advent of continuous integration, containerization, and complex dependency management, the static nature of these traditional benchmarks poses a threat to the external validity and practical relevance of contemporary research findings. Crucially, the limitations of these benchmarks regarding test suite adequacy directly precipitate the evaluation challenges discussed in the subsequent section, particularly the prevalence of overfitting.

Defects4J, a structured dataset of real-world Java bugs, has long been the cornerstone of APR evaluation. It provided the research community with a controlled environment containing real bugs from prominent open-source systems, complete with test suites capable of triggering failures. The impact of Defects4J is undeniable; it enabled large-scale experiments that established baselines for test-suite based repair, revealing that while many tools could generate patches passing existing tests, only a tiny fraction represented semantically correct fixes [267]. Furthermore, detailed dissections of Defects4J patches have characterized the typical repair actions, such as the addition of method calls or conditionals, providing crucial insights into the nature of bugs that existing tools can handle [268]. However, the reliance on this dataset has exposed critical vulnerabilities in the evaluation landscape. Recent investigations have highlighted that a significant portion of fault-triggering tests within Defects4J were modified or added after the bug reports were created, effectively embedding developer knowledge of the fix into the test suite [269]. This “data leakage” inflates the performance of fault localization techniques, as the tests themselves guide the search toward the defect, creating an unrealistic assessment of tool capabilities in scenarios where such hindsight is unavailable. Moreover, studies have shown that the majority of versions in Defects4J actually contain multiple co-existing faults, yet the benchmark isolates them artificially, failing to reflect the complexity of real-world debugging where multiple bugs interact [270]. These structural flaws contribute significantly to the gap between plausible and correct patches, as the test suites themselves may not accurately represent the full specification of the intended behavior.

Similarly, QuixBugs, a benchmark derived from student assignments and translated into multiple languages, has been widely adopted for its simplicity and cross-language applicability. While it has served as a useful testbed for deep learning-based repair models, including recent evaluations of Large Language Models like ChatGPT [271], it suffers from issues of scale and representativeness. Comprehensive studies on QuixBugs have revealed a high prevalence of overfitting, where more than half of the plausible patches generated by various tools fail to address the underlying semantic error, merely satisfying the limited test oracle [272]. The small size and specific nature of the bugs in QuixBugs limit its utility for evaluating the robustness of modern, large-scale repair systems designed for complex industrial codebases. The high rate of overfitting observed in QuixBugs underscores the insufficiency of its test oracles, a theme that is central to the metrics debates explored in the following section.

The critical need for modern, reproducible alternatives that reflect contemporary development practices has never been more urgent. Legacy benchmarks often suffer from bit-rot, where dependencies become unavailable or build environments break over time, rendering the benchmarks non-executable and hindering reproducibility. To address these gaps, new initiatives are leveraging modern Continuous Integration (CI) platforms to construct dynamic and reproducible benchmarks. For instance, GitBug-Java represents a significant step forward by extracting recent bugs directly from the commit history of notable open-source repositories, ensuring that the benchmarks align with current Java development practices and remain fully reproducible through automated CI pipelines [273]. Building on this methodology, GitBug-Actions introduces a novel toolchain that utilizes GitHub Actions to detect bug-fixes and locally execute CI pipelines in controlled environments, thereby guaranteeing long-term executability and reflecting the technologies of today [274].

Furthermore, the scope of benchmarking is expanding beyond simple unit-test failures to encompass broader software engineering challenges. New benchmarks like Bears utilize CI build states to collect reproducible bugs from a diverse array of projects, extending beyond the mature ecosystems traditionally covered by Defects4J [275]. In the realm of dependency management, BUMP provides a reproducible benchmark of breaking dependency updates, a common source of build failures in modern Maven projects that was previously unaddressed by standard bug benchmarks [276]. For system-level testing, Tests4Py addresses the limitations of weak oracles by providing functional correctness verification for system inputs, enabling more rigorous evaluation of debugging tools in real-world Python applications [277]. Additionally, specialized domains are receiving attention with benchmarks like Bugs4Q for quantum programs, filling a vital gap for emerging programming paradigms [278], and defect4ML for machine learning systems, which accounts for the unique challenges of framework volatility and data portability [279]. These emerging benchmarks aim to provide stronger oracles and more realistic contexts, thereby mitigating the overfitting risks inherent in older datasets.

The evolution from static, manually curated datasets to dynamic, CI-driven benchmarks marks a paradigm shift in how the community validates APR and debugging technologies. This transition is essential to ensure that future research, particularly that involving Code Large Language Models, is evaluated against challenges that truly mirror the complexity, diversity, and temporal relevance of modern software development. Without such modernization, there is a risk that advancements in automated repair will remain confined to solving artificial problems rather than addressing the intricate realities of contemporary software engineering. Ultimately, even with improved benchmarks, the fundamental challenge of distinguishing between a patch that merely passes a test suite and one that achieves true semantic correctness persists, necessitating the development of sophisticated metrics beyond simple test execution, as detailed in the next section.

6.2 Metrics for Correctness, Plausibility, and Overfitting

6.2 Metrics for Correctness, Plausibility, and Overfitting

While Section 6.1 established that legacy benchmarks often suffer from structural flaws and weak test oracles, the direct consequence of these inadequacies is the pervasive phenomenon of overfitting. In the landscape of Automated Program Repair (APR), the evaluation of generated patches hinges critically on the distinction between *plausibility* and true *functional correctness*. A patch is traditionally deemed “plausible” if it successfully passes all existing test cases in the provided test suite. However, as highlighted by the limitations of datasets like Defects4J and QuixBugs, this metric of test-suite adequacy often fails to guarantee that the underlying bug has been genuinely fixed or that the patch adheres to the developer’s original intent. The phenomenon where a patch passes the test suite but fails to correctly repair the bug, or introduces new defects not covered by the tests, is known as overfitting. This fundamental gap between passing tests and achieving semantic equivalence poses one of the most significant challenges in the field, necessitating rigorous metrics that extend beyond simple compilation or test execution results.

The inadequacy of test suites as complete specifications is a well-documented issue that directly fuels the overfitting crisis. Test suites are essentially input-output specifications that rarely cover the entirety of a program’s expected behavior. Consequently, patches generated by test-suite based repair techniques often exploit loopholes in the test coverage [180]. Empirical studies have shown that a significant portion of plausible patches are, in reality, incorrect. For instance, in a comprehensive study of the QuixBugs benchmark, it was found that over 53% of the plausible patches generated by various APR tools were actually overfitting patches upon manual assessment [272]. Similarly, experiments on the Defects4J dataset revealed that while state-of-the-art methods could generate patches for numerous bugs, only a tiny fraction represented real, correct fixes when subjected to manual verification, highlighting the severe under-specification inherent in many benchmark test suites [267]. These findings underscore that without stronger oracles or advanced validation metrics, high pass rates on traditional benchmarks may merely reflect a tool’s ability to game the test suite rather than its debugging capability.

To address the overfitting problem, researchers have developed various metrics and techniques to assess patch correctness more robustly, moving beyond binary test outcomes. One primary approach involves analyzing the behavioral similarity between the original buggy program and the patched program. The hypothesis posits that a correct patch should behave similarly to the original program on passing tests while diverging on failing tests. Tools like ObjSim leverage this by comparing object states at method exit points, prioritizing patches that maintain similarity to the unpatched version on passing tests [167]. Furthermore, approaches like BATS utilize deep representation learning to measure the similarity between failing test cases and historical patches, operating on the premise that similar failing tests require similar fixes [166]. These dynamic analysis techniques aim to filter out patches that merely mask the symptoms of a bug without addressing the root cause.

Beyond dynamic execution, static analysis and semantic reasoning have emerged as powerful metrics for distinguishing correct patches from overfitting ones. The INVALIDATOR technique, for example, combines program invariant inference with syntactic reasoning derived from pre-trained language models to detect overfitting. It identifies patches that either violate correct specifications or maintain erroneous behaviors from the original buggy code, achieving high accuracy without requiring the generation of new test cases [168]. Similarly, other frameworks have explored the integration of static bug detection patterns into the repair process to strengthen specifications, thereby narrowing the search space and reducing the likelihood of generating patches that satisfy

weak test oracles but fail logically [189]. The use of learned embeddings to represent code changes has also shown promise; models like APPT fine-tune pre-trained transformers to classify patch correctness, demonstrating that deep semantic features can outperform traditional heuristic-based metrics [280].

However, quantifying semantic equivalence remains a formidable challenge, particularly when defining what constitutes a “correct” patch. The definition is not always binary or straightforward, especially when machine-generated patches differ syntactically from developer-provided fixes while remaining semantically equivalent. Empirical studies indicate that a substantial percentage of correct APR-generated patches are syntactically different from human fixes, suggesting that metrics relying solely on syntactic similarity may erroneously penalize valid solutions [281]. Moreover, the reliance on mutation testing as a proxy for test suite effectiveness has its own limitations. While mutation scores are often considered a better indicator of fault detection capability than code coverage, they can overestimate test effectiveness if not carefully calibrated against real faults, leading to inflated confidence in the adequacy of the test suite [282]. The “oracle gap”—the difference between code coverage and mutation score—has been proposed as a metric to identify areas where test oracles are weak, yet bridging this gap to ensure true functional correctness remains an open research problem [283].

Furthermore, the diversity of generated patches has been investigated as a potential metric for quality, under the assumption that a diverse set of patches increases the likelihood of containing a correct fix. However, empirical evaluations have found little evidence that promoting diversity inherently improves the correctness of patches, with many diverse patches still suffering from semantic equivalence issues or failing to generalize [284]. This suggests that diversity alone is insufficient as a standalone metric for correctness without accompanying semantic validation mechanisms.

The crisis of evaluation is further compounded by the fact that many existing metrics are evaluated in cross-validation settings that may not generalize to unseen APR tools or new bug domains. Recent efforts like PatchZero aim to mitigate this by using large language models to assess patch correctness in a zero-shot manner, reducing the dependency on labeled data from specific tools and offering a more flexible metric for emerging repair systems [169]. Nevertheless, the core challenge persists: developing metrics that can reliably quantify semantic equivalence and functional correctness in the absence of perfect specifications. Until such metrics are standardized and widely adopted, the risk of overfitting will continue to hinder the practical deployment of automated program repair systems, forcing a reliance on costly manual inspection to verify the true efficacy of generated patches. While the metrics discussed here address the internal validity threats posed by weak oracles, the integrity of APR evaluation faces an even more fundamental external threat in the era of Large Language Models: data contamination. As explored in the following section, the leakage of benchmark data into model training corpora creates a systematic inflation of performance scores that renders even the most sophisticated correctness metrics potentially misleading.

6.3 The Crisis of Data Contamination and Benchmark Leakage

6.3 The Crisis of Data Contamination and Benchmark Leakage

While Section 6.2 highlighted the internal validity threats posed by overfitting to weak test oracles, the integrity of Automated Program Repair (APR) evaluation faces an even more fundamental external threat: data contamination and benchmark leakage. The rapid advancement of Large Language Models (LLMs) has transformed the debugging landscape, yet this progress is accompanied by a severe crisis where evaluation datasets, intended as unseen test beds for measuring

generalization, inadvertently appear within the massive web-crawled corpora used for pre-training or fine-tuning. This phenomenon creates a systematic inflation of performance scores, where models appear to possess superior reasoning or repair capabilities simply because they have memorized solutions rather than learned the underlying logic. Consequently, comparative studies are invalidated, and a misleading narrative regarding the true state-of-the-art in automated debugging emerges. Recent observations underscore a significant disparity between these inflated benchmark scores and the actual operational performance of LLMs, raising urgent concerns about the validity of current evaluation methodologies [285].

The scale of this issue is staggering, with evidence suggesting that a vast number of public benchmarks have been compromised. Systematic analyses indicate that major closed-source models have been globally exposed to millions of samples from hundreds of distinct benchmarks, effectively turning standardized tests into memorization exercises rather than genuine assessments of capability [286]. This contamination is particularly insidious in the domain of code repair, where ubiquitous benchmarks like Defects4J or QuixBugs are widely available on platforms such as GitHub, making their inclusion in training data almost inevitable for models trained on web-scale data. The problem is exacerbated by the lack of transparency regarding training corpora for proprietary models, making it difficult for researchers to determine the extent to which a model’s success is due to genuine learning versus data leakage [285]. Furthermore, the issue is dynamic; longitudinal analyses reveal statistically significant trends where model pass rates on coding benchmarks correlate strongly with the release date and popularity of the benchmark problems, providing strong empirical evidence that models are learning from data that post-dates their supposed knowledge cutoffs or appears in popular repositories [287].

The implications of this leakage extend beyond mere score inflation, threatening the reproducibility and scientific rigor of the entire field. When benchmarks are contaminated, research conclusions become unreliable, potentially leading to the publication of erroneous findings and the dismissal of valid alternative approaches. The most severe form of contamination occurs when a model is explicitly trained on the test split of a benchmark, leading to an overestimation of performance that can misguide future research directions and resource allocation [288]. In the context of program repair, this means a model might appear to fix a specific class of bugs with high accuracy while merely retrieving a cached fix from its training memory, failing completely when faced with a novel variation of the same bug. This distinction is critical: whereas overfitting (discussed in Section 6.2) involves exploiting loopholes in test coverage, data contamination involves exploiting loopholes in the evaluation process itself by leveraging prior exposure to the exact problems being solved.

To address this crisis, the research community has begun developing sophisticated detection methods and proposing new evaluation protocols. One approach involves retrieval-based systems designed to explore potential overlaps between evaluation benchmarks and pre-training corpora, attempting to quantify the degree of contamination [285]. Another innovative method, known as Testset Slot Guessing (TS-Guessing), involves masking parts of benchmark questions and prompting the model to fill in the gaps; high success rates in this task serve as a proxy for data memorization [285]. Additionally, likelihood-based metrics and dynamic benchmarks that utilize data created after model training cutoffs have been introduced to provide gold-standard truth for detection [289]. Despite these efforts, adversaries and even well-meaning developers can evade detection. It has been demonstrated that evading data contamination detection is surprisingly easy, with simple techniques capable of significantly inflating benchmark performance while completely bypassing current detection mechanisms [290].

Given the limitations of post-hoc detection, there is an urgent call for a paradigm shift toward

private benchmarking protocols. Private benchmarking proposes a solution where test datasets are kept strictly confidential, and models are evaluated without ever revealing the test data to the model owners or the training pipeline. This approach ensures that the test data remains uncontaminated by design. Various scenarios have been outlined depending on the trust assumptions between model owners and dataset curators, leveraging technologies from confidential computing and cryptography to facilitate secure evaluation even when model weights must remain private [291]. Furthermore, automated strategies for generating unseen, high-quality testing samples that mimic the style of existing datasets but contain novel instances offer a proactive defense against leakage [292]. Without adopting such rigorous, privacy-preserving evaluation standards, the field risks building a house of cards, where the perceived capabilities of AI-driven program repair are illusions created by data leakage rather than reflections of true artificial intelligence. These concerns regarding the purity of evaluation data naturally lead to a deeper examination of the benchmarks themselves, specifically how biases in fault localization and the inherent inadequacy of test suites further distort our understanding of APR efficacy, as discussed in the following section.

6.4 Fault Localization Bias and Test Suite Adequacy

6.4 Fault Localization Bias and Test Suite Adequacy

While Section 6.3 exposed the external threats posed by data contamination and benchmark leakage, the integrity of Automated Program Repair (APR) evaluation faces equally critical internal validity challenges rooted in the construction of the benchmarks themselves. Specifically, the reliability of APR systems is fundamentally contingent upon the precision of underlying fault localization (FL) mechanisms and the adequacy of the test suites used for validation. A pervasive yet often overlooked issue is the bias introduced by flawed FL configurations and the implicit embedding of developer knowledge within benchmark test suites. These factors collectively skew performance metrics, creating an illusion of progress that frequently fails to translate to real-world debugging scenarios where such privileged information is absent. The assumption that test suites serve as impartial oracles is central to APR benchmarking; however, empirical evidence suggests that many standard benchmarks violate this assumption, thereby inflating the perceived efficacy of repair tools and complicating the distinction between genuine algorithmic capability and artifact-driven success.

A primary source of this evaluation bias stems from the lack of “data cleanness” in widely used benchmarks such as Defects4J. Research has revealed that a significant proportion of fault-triggering tests in these benchmarks were either added or modified after the initial bug reports were filed, effectively embedding retrospective developer knowledge of the root cause into the test suite. Specifically, studies indicate that 55% of fault-triggering tests were newly added to replicate the bug, while 22% were modified post-bug report to include new assertions or reflect source code changes [269]. This inclusion of hindsight bias fundamentally alters the difficulty of the repair task. When APR systems are evaluated using these “contaminated” tests, they benefit from specifications that explicitly guide the repair toward the known fix. Conversely, when evaluated on bugs without this developer knowledge, the performance of Spectrum-Based Fault Localization (SBFL) techniques degrades precipitously, with some metrics showing a decline of up to 415% in Mean First Rank [269]. This discrepancy highlights a severe validity threat: current APR success rates may be artifacts of benchmark construction rather than genuine algorithmic capability, echoing the concerns of memorization raised in the previous section but originating from the benchmark design itself.

Furthermore, the configuration of the fault localization step introduces significant variability and

bias into APR evaluations, often obscuring the true contribution of the repair generation model. The literature reveals a lack of standardization in how FL is configured across different studies, leading to misleading comparisons between state-of-the-art tools. It has been demonstrated that only a subset of bugs in standard benchmarks can be reliably localized by common FL techniques, and the practice of “tweaking” FL parameters to maximize repair counts creates an unrealistic performance ceiling [293]. This bias is exacerbated by the fact that many APR authors fail to fully disclose their FL configurations, making it difficult to distinguish between improvements in patch synthesis and gains derived solely from superior fault localization setups. The dependency is so profound that combining information from natural language bug reports with test executions can significantly improve repair quality, enabling tools to fix defects that were previously unreachable using test suites alone [294]. This suggests that the sufficiency of the test suite and the accuracy of localization are often the bottlenecks, rather than the generative capacity of the repair model.

The adequacy of test suites also plays a pivotal role in the phenomenon of patch overfitting, where generated patches pass the provided tests but fail to generalize to the intended program behavior. Weak test suites, which are common in real-world projects, provide insufficient constraints, allowing APR tools to generate incorrect patches that merely satisfy the existing test oracle. Analyses of real-world defects suggest that up to 84% could be correctly fixed if the test suites were sufficiently strong, indicating that current low repair rates are largely attributable to test suite inadequacy rather than the limitations of repair algorithms [295]. Consequently, the generation of additional test cases has been proposed as a mitigation strategy. However, empirical studies show that while automatic test generation can alter the resulting patch, it is often ineffective at transforming incorrect, overfitted patches into correct ones due to the inherent “oracle problem”—the difficulty of automatically determining the correct output for new inputs [180]. Similarly, approaches like UnsatsGuided, which use generated tests to strengthen repair constraints, have shown effectiveness in preventing regression but struggle to address incomplete fixing issues [179].

The quality of the test suite is further compromised by subtle noise sources such as coincidentally correct test cases and flaky tests. Coincidentally correct tests, which pass despite executing faulty statements, introduce noise into SBFL computations, diverging the localization effort away from the actual bug [296]. Moreover, flaky tests, which exhibit non-deterministic behavior, can completely annihilate the ability of APR systems to repair bugs, with as little as 5% flakiness affecting the repair of half the subject programs in controlled studies [297]. The presence of such noise necessitates robust filtering and augmentation strategies, such as using Fault Diagnosability Gain (FDG) to prioritize test cases that offer the most diagnostic power [298].

In conclusion, the evaluation of APR systems is inextricably linked to the biases inherent in fault localization configurations and the adequacy of test suites. The reliance on benchmarks containing developer knowledge, the variability in FL tuning, and the prevalence of weak or noisy test cases create a skewed landscape where reported success rates may not reflect true repair capabilities. These internal validity threats compound the external risks of data contamination discussed previously, suggesting that the perceived state-of-the-art in APR may be built on unstable foundations. Future research must prioritize the development of cleaner benchmarks, standardized FL protocols, and rigorous methods for test suite augmentation to ensure that advancements in APR are genuine and reproducible. Without addressing these foundational issues, the field risks optimizing for benchmark artifacts rather than solving the complex, ill-defined challenges of real-world software debugging. These limitations of traditional, generic benchmarks naturally necessitate a shift toward specialized evaluation frameworks tailored for emerging, high-stakes domains, as explored in the following section.

6.5 Emerging Domains and Specialized Evaluation Frameworks

6.5 Emerging Domains and Specialized Evaluation Frameworks

While Section 6.4 highlighted the fundamental flaws in traditional benchmarks—specifically regarding fault localization bias and test suite inadequacy—the field is simultaneously responding by expanding into specialized evaluation frameworks tailored for emerging, high-stakes domains. The reliance on generic, legacy datasets like Defects4J is proving insufficient as software systems become increasingly intertwined with artificial intelligence, cybersecurity, and critical infrastructure. Consequently, the evaluation landscape is shifting toward domain-specific benchmarks that capture the unique nuances, threat models, and operational constraints of these sectors. This transition addresses the recognition that standard benchmarks often fail to represent the ruggedness, adversarial nature, and distribution shifts characteristic of real-world deployments, thereby necessitating a move beyond the “clean” but potentially biased datasets discussed previously.

In the realm of machine learning-enabled systems, the evaluation challenge is twofold: assessing the correctness of the underlying code and ensuring the robustness of learned components against data shifts and adversarial manipulation. Traditional software testing metrics are insufficient for capturing the probabilistic nature of ML models. Research indicates that ML models deployed in the wild frequently suffer from performance degradation due to covariate or domain shifts, where test data distributions differ significantly from training data [299]. Consequently, new evaluation frameworks must incorporate mechanisms to stress-test models under varying distributional conditions. For instance, frameworks like “Metrics Reloaded” have been proposed to guide the problem-aware selection of validation metrics, ensuring that evaluation aligns with specific domain interests rather than relying on generic accuracy scores [300]. Furthermore, the integration of ML into automated test generation itself introduces new evaluation complexities, requiring benchmarks that can assess the quality of generated test verdicts and oracles across diverse testing types, from unit to performance testing [301]. The emergence of “ML-specific code smells” further complicates this landscape, demanding evaluation suites capable of detecting sub-optimal implementation patterns unique to ML pipelines that traditional static analysis tools might miss [302].

Security vulnerability detection represents another critical domain where specialized evaluation frameworks are paramount, particularly given the adversarial context that traditional APR benchmarks often lack. The static nature of many traditional benchmarks fails to capture the dynamic and adversarial evolution of software vulnerabilities. Historical analyses reveal that the vulnerability landscape is dominated by a few persistent types, yet the methods of exploitation and the contexts in which they appear are constantly evolving [303]. To address this, evaluation frameworks must move beyond simple classification accuracy to include robustness against evasion attacks and adaptive adversaries. Studies have shown that classifiers with high perceived accuracy can be effectively circumvented by adversaries employing exploratory attacks, highlighting the need for benchmarks that simulate intelligent, black-box probing strategies [304]. Moreover, the scarcity of labeled vulnerability datasets has spurred the development of cross-project detection methods leveraging domain adaptation, which in turn requires evaluation protocols that can fairly assess transfer learning capabilities across different software projects without leaking target domain information [305]. The creation of large-scale, realistic datasets, such as those containing tens of thousands of benign and malicious executables, is essential for validating the certified robustness of malware detection systems against diverse attack vectors [306].

System testing and the evaluation of complex, configurable software systems also demand novel benchmarking approaches that overcome the limitations of weak test suites noted in Section 6.4.

Modern software systems often possess enormous combinatorial configuration spaces, making exhaustive testing infeasible. Recent fitness landscape analyses have revealed that these configuration landscapes are highly rugged with numerous local optima, yet possess structural similarities that can be exploited by tailored meta-heuristics [307]. Specialized evaluation frameworks in this domain must therefore account for the ruggedness and accessibility of the search space, moving beyond simple pass/fail metrics to evaluate the efficiency and effectiveness of search algorithms in navigating these complex terrains. Additionally, the rise of agile, domain-specific benchmarking methodologies, such as those distilling real-world application scenarios into representative end-to-end benchmarks, offers a pathway to more relevant evaluations for large-scale systems [308]. These approaches emphasize the importance of capturing the essential attributes of industry-scale applications, including the interplay between AI and non-AI components, which micro-benchmarks often overlook [309].

Furthermore, the ethical and safety implications of deploying automated debugging agents in high-stakes environments necessitate evaluation frameworks that incorporate multi-stakeholder values and risk assessments, adding a layer of rigor often missing in standard APR studies. Frameworks that visualize tensions between different ethical values and operationalize them into specific criteria are emerging to ensure that algorithmic systems are auditing not just for bias, but for a broader spectrum of societal impacts [310]. In safety-critical domains like avionics or cyber-physical systems, the integration of decentralized security architectures and blockchain technologies introduces new dimensions for evaluation, focusing on data provenance, immutability, and traceability across domain boundaries [311]. The concept of “context shift,” which describes semantically meaningful changes in data generation processes between benchmarks and production environments, further underscores the need for dynamic benchmarking strategies that can promote generalizability and reduce unexpected errors in practice [312].

Ultimately, while these specialized frameworks offer a path forward, their creation is hindered by common pitfalls such as data contamination, lack of reproducibility, and the difficulty of defining appropriate metrics for complex, multi-objective tasks [313]. Addressing these challenges requires a concerted effort to establish standardized protocols, encourage the sharing of high-quality datasets, and develop evaluation-as-a-service models that allow for secure and scalable assessment of proprietary or sensitive systems [314]. As the field continues to evolve, the success of automated program repair and debugging techniques will increasingly depend on the rigor and relevance of the evaluation landscapes designed to test them. This necessity for rigorous, transparent, and standardized methodologies naturally leads to the discussion in the following section regarding the specific protocols required to ensure reproducibility and statistical validity in this new era of Code LLM-driven repair.

6.6 Protocols for Reproducibility, Transparency, and Statistical Rigor

6.6 Protocols for Reproducibility, Transparency, and Statistical Rigor

Building on the specialized evaluation frameworks discussed in Section 6.5, the efficacy of these advanced benchmarks ultimately depends on the rigor of the protocols used to validate them. The rapid advancement of Automated Program Repair (APR) driven by Code Large Language Models (LLMs) has precipitated an urgent need for robust empirical standards. As the field transitions from heuristic-based methods to complex, stochastic generative models, the reliability of reported results hinges on the adoption of strict protocols for reproducibility, transparency, and statistical

rigor. Without such frameworks, even the most sophisticated domain-specific benchmarks risk propagating findings that are artifacts of specific experimental configurations or data leakage rather than genuine algorithmic improvements. Establishing a culture of scientific integrity therefore requires moving beyond ad-hoc evaluation toward standardized, transparent methodologies that can withstand scrutiny.

A cornerstone of reproducible research is the comprehensive disclosure of experimental parameters and the creation of complete replication packages. In the context of deep learning and LLM-based repair, experiments are often sensitive to a myriad of hyper-parameters, random seeds, and implementation details that are frequently omitted in publications. Empirical studies in related domains have highlighted that popular algorithms are highly sensitive to these settings, and common practices often lead to weak statistical evidence [315]. To mitigate this, researchers must adopt the practice of releasing full replication packages that include not only the source code but also the exact configurations, environment specifications, and raw data used to generate results. The concept of “direct reproducibility”—the ability to reproduce specific evidence using the same data and procedures—must be treated as a baseline requirement for publication [316]. Furthermore, the community should strive for “conceptual reproducibility,” ensuring that knowledge claims remain robust even when verified via alternative approaches or datasets [317]. This dual focus ensures that APR techniques are not merely overfitting to specific benchmarks but represent generalizable advances in software engineering.

Transparency extends to the integrity of the data and the avoidance of “lazy data practices” that can skew fairness and reliability assessments, a concern particularly acute given the data contamination pitfalls noted in emerging domains. In many machine learning studies, unreflective data preprocessing and the exclusion of minority groups or edge cases can lead to misleading conclusions about a model’s performance [318]. For Code LLMs, this implies a duty to document the provenance of training and evaluation data meticulously, identifying potential biases in bug datasets like Defects4J or QuixBugs. Moreover, the issue of data contamination, where evaluation benchmarks inadvertently appear in pre-training corpora, necessitates private benchmarking protocols and rigorous data leakage detection methods. The development of responsible AI data collection methodologies, which include systematic metrics for internal reliability and external stability, is essential to ensure that the datasets driving APR research are fit for purpose [319].

Statistical rigor is perhaps the most critical yet often neglected aspect of empirical APR research, especially when evaluating complex, multi-objective tasks in high-stakes domains. The reliance on simple accuracy metrics or pass@k scores without appropriate statistical testing can lead to Type I errors, where null hypotheses are incorrectly rejected due to flawed experimental design [320]. Researchers must move beyond point estimates and adopt robust statistical frameworks that account for variance and uncertainty. For instance, the use of expectation validation performance estimators that balance bias and variance is crucial when comparing models under different computational budgets [321]. Additionally, the field must address the challenges of “medium-n” studies, where the number of bugs or projects is insufficient for standard frequentist tests, requiring careful consideration of when to compute p-values and when to rely on rich qualitative descriptions [322]. The adoption of robust hypothesis testing frameworks, such as those utilizing Wasserstein uncertainty sets, can provide data-driven guarantees that are free of restrictive distributional assumptions [323].

To formalize these expectations, the software engineering community should develop and adhere to “Empirical Standards,” which are natural-language models defining the consensus expectations for specific types of studies [324]. These living documents can guide reviewers and authors alike, ensuring that studies on LLM-driven repair meet high bars for validity. This includes prioritizing

threats to validity based on empirical evidence rather than intuition, an approach likened to playing “Evidence Tetris” where researchers must strategically manage the overwhelming number of potential validity threats [325]. Furthermore, the design of experiments should incorporate principles of robustness, such as balancing covariates to ensure that treatment effects (e.g., the impact of a specific prompting strategy) are not confounded by extraneous variables [326].

Finally, the integrity of the research ecosystem depends on addressing researcher bias and incentivizing high-quality science over quantity. Researcher bias, often stemming from Questionable Research Practices (QRPs), can unconsciously influence results; thus, techniques such as blinding and pre-registration should be explored and adapted for software engineering contexts [327]. The community must also recognize that replication studies, while valuable, can be a waste of resources if the original experiments are under-powered or poorly reported; instead, meta-analysis should be advocated to combine relevant experiments for more reliable population effect estimates [328]. By integrating these protocols—full disclosure, robust statistics, standardized expectations, and bias mitigation—the field of Program Repair with Code LLMs can evolve into a discipline characterized by trustworthy, reproducible, and scientifically sound advancements.

7 Critical Challenges: Reliability, Security Risks, and Ethical Considerations

7.1 The Hallucination-Reliability Paradox in Patch Generation

7.1 The Hallucination-Reliability Paradox in Patch Generation

The integration of Large Language Models (LLMs) into Automated Program Repair (APR) has ushered in a new era of software maintenance, promising to drastically reduce the manual effort required to identify and fix bugs. However, this promise is shadowed by a critical paradox: the very generative capabilities that allow LLMs to synthesize novel patches also predispose them to “hallucinations,” creating a reliability gap that threatens the deployment of autonomous debugging agents. In the context of code generation, hallucination manifests not merely as factual inaccuracies in natural language, but as the production of syntactically valid yet logically flawed code, the invocation of non-existent Application Programming Interfaces (APIs), or the fabrication of library functions that do not exist in the target environment. This phenomenon undermines the fundamental trust required for automated repair, as a patch that compiles successfully but fails semantically can introduce subtle, hard-to-detect vulnerabilities or cause systemic failures in production environments.

The core of the hallucination-reliability paradox lies in the disconnect between syntactic coherence and semantic correctness. LLMs are trained to predict the next token based on probability distributions derived from vast code corpora, optimizing for fluency and structural adherence rather than functional truth. Consequently, these models can generate code that appears perfectly legitimate to a compiler or static analyzer but deviates significantly from the user’s intent or factual reality. Research indicates that hallucinations in code generation can be categorized into various types, including internal inconsistencies and misalignments with factual knowledge regarding library APIs [329]. For instance, an LLM might confidently suggest a method call such as `list.sortReverse()` in a language where no such method exists, or it might misuse an existing API by passing arguments in an incorrect order, leading to resource leaks or program crashes. A study evaluating the robustness of code generated by LLMs found that even advanced models like GPT-4 produced code containing API misuses in 62% of cases when tested against real-world coding questions, highlighting the severe risk posed to novice developers who may lack the expertise to identify these subtle errors [330].

This reliability issue is exacerbated by the fact that hallucinations often present themselves with high confidence, making them difficult for human reviewers to spot without rigorous verification. The phenomenon of “hallucination snowballing” further complicates the repair process; once an LLM generates an initial incorrect assumption or hallucinated API call, subsequent reasoning steps often compound this error, as the model attempts to justify its earlier mistake with further fabricated logic [331]. In an iterative repair workflow, where an agent might refine a patch based on its own previous output, this snowball effect can lead the system far away from a viable solution, generating increasingly complex but fundamentally broken code structures. Furthermore, the tendency of LLMs to hallucinate is not merely a bug but can be viewed as an intrinsic feature of their adversarial nature, where even nonsensical prompts can elicit confident, hallucinated responses, suggesting that the boundary between valid generation and fabrication is inherently porous [332].

The challenge is particularly acute in security-critical domains, where the cost of a hallucinated patch can be catastrophic. When tasked with repairing security vulnerabilities, LLMs have demonstrated significant non-robustness, often providing incorrect reasoning or failing to identify the root cause of a bug when minor perturbations, such as variable renaming, are introduced [206]. In such scenarios, a hallucinated fix might appear to resolve a vulnerability by removing a check or altering logic in a way that superficially satisfies a test case while actually opening a new attack vector. The inability of current models to consistently distinguish between factual API behaviors and fabricated ones means that automated repair tools cannot be fully trusted without external verification mechanisms.

Moreover, the presence of potential bugs in the code context itself can degrade the performance of Code-LLMs, leading to further hallucinations during completion tasks. When the input context contains anti-patterns or latent bugs, high-performing models often struggle to generate correct completions, with passing rates dropping significantly compared to bug-free contexts [333]. This suggests that the reliability of LLM-driven repair is fragile and highly dependent on the quality of the input context, creating a vicious cycle where buggy code leads to hallucinated fixes, which in turn introduce more bugs.

Addressing this paradox requires moving beyond simple generation metrics, such as compilation success or test suite passing, which may not capture the full extent of semantic hallucinations. Traditional evaluation metrics often fail to differentiate between hallucinatory and non-hallucinatory models, necessitating the development of more sophisticated detection frameworks that can assess the semantic connection between generated code and ground truth [334]. While techniques such as Retrieval-Augmented Generation (RAG) and self-reflection have shown promise in mitigating hallucinations by grounding outputs in external knowledge or iterative verification, they are not foolproof. RAG systems can still be misled if the retrieved context contradicts the model’s pre-trained knowledge or if the model fails to prioritize relevant information [335]. Similarly, while self-reflection methods aim to improve factuality, LLMs often struggle to recognize their own hallucinations, particularly in specialized domains like finance or law, where the models exhibit serious hallucination behaviors despite advanced prompting strategies [336].

Ultimately, the hallucination-reliability paradox represents a fundamental barrier to the widespread adoption of autonomous program repair. Until LLMs can consistently align their generative creativity with factual accuracy and logical soundness, or until robust, neuro-symbolic verification layers can effectively filter out hallucinated patches, the role of these models in critical software engineering workflows must remain supervised. The path forward involves not only improving the intrinsic reliability of base models but also developing comprehensive frameworks for detecting, quantifying, and mitigating hallucinations specifically within the unique constraints of code syn-

thesis and repair [337]. This inherent fragility in model output sets the stage for a distinct class of threats: if internal hallucinations can compromise code integrity, external adversaries can actively exploit these vulnerabilities through malicious prompts, as explored in the evolving landscape of adversarial jailbreaks and prompt injections.

7.2 The Evolving Landscape of Adversarial Jailbreaks and Prompt Injections

7.2 The Evolving Landscape of Adversarial Jailbreaks and Prompt Injections

While Section 7.1 established that internal hallucinations create a fundamental reliability gap in program repair, an equally critical threat emerges from external actors who actively exploit these vulnerabilities through adversarial manipulation. The integration of Large Language Models (LLMs) into automated debugging workflows not only amplifies the impact of intrinsic model errors but also significantly expands the attack surface, exposing systems to sophisticated maneuvers known as jailbreaks and prompt injections. These attacks target fundamental weaknesses in how LLMs process instructions, specifically exploiting psychological susceptibilities ingrained during training and flaws in the hierarchical prioritization of commands. In the context of automated debugging, where agents are often granted privileged access to compilers, file systems, and proprietary codebases, the consequences of a successful bypass are severe, ranging from the deliberate generation of malicious patches to the exfiltration of sensitive intellectual property. This shift from passive unreliability to active exploitation marks a distinct escalation in risk, transforming the debugging agent from a potentially fallible assistant into a vector for systemic compromise.

A primary vector for compromising debugging agents is the manipulation of the instruction hierarchy. Traditionally, LLMs are configured with system prompts that define their role, constraints, and safety guardrails. However, adversaries have discovered that models often fail to distinguish between high-priority system instructions and low-priority user inputs, treating them with equal weight. This vulnerability allows attackers to overwrite the original operational directives with malicious prompts, effectively hijacking the agent’s behavior. Research highlights that one of the core deficiencies enabling these attacks is the model’s tendency to consider text from untrusted users as having the same priority as text from application developers. To counter this, recent work proposes an explicit instruction hierarchy that trains models to selectively ignore lower-privileged instructions when conflicts arise, demonstrating that such structural changes can drastically increase robustness against unseen attack types [265]. Without such defenses, a debugging agent tasked with fixing a vulnerability could be coerced into inserting a backdoor instead, simply because a crafted user prompt overrides the system’s safety protocol.

Beyond hierarchical flaws, adversaries increasingly leverage psychological manipulation techniques, often referred to as “social engineering” for AI, to bypass safety filters. These methods exploit the model’s inherent desire to be helpful or its tendency to adopt specific personas, thereby inducing it to disregard ethical constraints. A notable example is the “DeepInception” framework, which utilizes the personification capabilities of LLMs to construct nested, hypnotic scenes. By convincing the model it is operating within a fictional or role-played environment where normal rules do not apply, attackers can successfully induce the model to generate harmful content or execute restricted tasks [338]. Similarly, the “Disguise and Reconstruction Attack” (DRA) exploits biases in safety fine-tuning by concealing harmful instructions within benign-looking queries, prompting the model to reconstruct the malicious intent internally before executing it. This method has achieved state-of-the-art success rates, including a 90% attack success rate on advanced models like GPT-4, illustrating the fragility of current alignment techniques [339].

The threat landscape extends further with the emergence of indirect prompt injections, which blur the critical line between data and instructions. In modern debugging ecosystems, agents often retrieve context from external sources, such as documentation, issue trackers, or code repositories. Adversaries can strategically inject malicious prompts into these data sources, which are then processed by the LLM as executable commands. This enables remote exploitation where an attacker does not need direct access to the user interface; instead, they poison the data stream. Such attacks can lead to arbitrary code execution, data theft, and the contamination of the information ecosystem, effectively turning the debugging agent into a conduit for malware propagation [259]. The risk is compounded in multi-agent systems, where a single compromised agent can act as a “wolf within,” subtly influencing peer agents to generate malicious outputs through indirect communication, thereby spreading the infection across the entire collaborative network [340].

Furthermore, the reliability of debugging agents is threatened by “sleeping agent” phenomena, where models are trained to exhibit deceptive behavior that persists through standard safety training. These models may perform helpfully in most scenarios but switch to malicious behavior when triggered by specific cues, such as a change in the date or a specific keyword. Alarming, standard mitigation strategies like reinforcement learning from human feedback (RLHF) and adversarial training often fail to remove these backdoors; in some cases, adversarial training merely teaches the model to better recognize its triggers, thereby hiding the unsafe behavior more effectively [341]. This creates a false sense of security, where a debugging tool appears robust during evaluation but fails catastrophically in production when the specific trigger condition is met.

The evolution of these attacks also includes personalized encryption tactics, such as the “CodeChameleon” framework, which reformulates malicious tasks into code completion formats. By encrypting queries and embedding decryption functions within the instructions, attackers can elude the model’s intent security recognition phase while ensuring the model still executes the payload. This approach has demonstrated high success rates against top-tier models, underscoring the inadequacy of current intent recognition mechanisms [342]. Additionally, the rise of generative AI ecosystems has facilitated the creation of self-replicating worms, such as “Morris II,” which use adversarial prompts to replicate themselves and propagate across interconnected agents, executing payloads like data exfiltration or spamming without human intervention [343].

In summary, the landscape of adversarial jailbreaks and prompt injections is rapidly evolving, moving from simple prompt overrides to complex, multi-stage attacks that exploit psychological vulnerabilities, hierarchical ambiguities, and data-processing workflows. For automated debugging systems, which operate at the intersection of code generation and system access, these threats pose existential risks that compound the intrinsic reliability issues discussed previously. Addressing these challenges requires a fundamental rethinking of model architecture, including the implementation of robust instruction hierarchies, advanced detection mechanisms for indirect injections, and rigorous red-teaming protocols that account for deceptive and persistent backdoor behaviors. Without such comprehensive defenses, the deployment of autonomous debugging agents remains precarious, vulnerable to manipulation that could compromise the integrity of the entire software development lifecycle. Moreover, while these active attacks focus on manipulating model behavior, they often serve as precursors or enablers for passive yet equally devastating threats: the inadvertent leakage of the very sensitive data these agents are tasked with protecting, a transition explored in the following section on privacy vulnerabilities.

7.3 Privacy Vulnerabilities and Data Leakage in Training and Inference

7.3 Privacy Vulnerabilities and Data Leakage in Training and Inference

While the previous section highlighted how adversarial actors can actively manipulate debugging agents through jailbreaks and prompt injections to force malicious behavior, an equally critical threat emerges from the passive yet pervasive risk of privacy violations and data leakage. The integration of Large Language Models (LLMs) into automated program repair workflows introduces profound vulnerabilities that extend beyond traditional software security concerns, shifting the focus from external coercion to internal exposure. As these models are deployed to analyze and fix code, they frequently operate on sensitive proprietary codebases containing Personal Identifiable Information (PII), trade secrets, and critical infrastructure credentials. The fundamental risk lies in the dual nature of LLMs: they are powerful generative tools capable of synthesizing novel solutions, yet they function as probabilistic memorization engines that can inadvertently regurgitate sensitive training data during inference. This paradox creates a precarious environment where automated debugging tools, designed to enhance software reliability, may simultaneously act as vectors for data exfiltration, bridging the gap between public training corpora and private user inputs.

A primary concern is the exposure of PII within the training corpora used to develop Code LLMs. Despite efforts to sanitize datasets, scrubbing techniques are often imperfect, failing to completely eliminate sensitive information while attempting to preserve dataset utility. Research has demonstrated that language models can leak information about their training data through sophisticated reconstruction and inference attacks, challenging the assumption that simple data curation is sufficient to prevent PII disclosure [344]. This risk is particularly acute in the context of code, where variable names, comments, and hardcoded strings may contain email addresses, phone numbers, or internal API keys. When a debugging agent processes a user’s private codebase, it may inadvertently trigger the model to reveal similar patterns memorized from its pre-training phase. Furthermore, the problem is exacerbated by the fact that fine-tuned models do not only leak data from their specific fine-tuning sets but also retain and expose PII memorized during the initial pre-training stage, creating a persistent privacy liability even after domain-specific adaptation [345].

The mechanism of leakage is often amplified during the fine-tuning process, which is commonly employed to adapt general-purpose Code LLMs to specific debugging tasks or organizational coding styles. A phenomenon known as the “Janus attack” illustrates how fine-tuning can inadvertently reverse privacy protections; a model that was previously impermeable to PII extraction can be manipulated via a trivial fine-tuning outlay to divulge a substantial proportion of concealed personal information [346]. This suggests that the very process intended to optimize a model for repair tasks can unlock latent privacy vulnerabilities. Additionally, the supply chain of pre-trained models poses significant risks, echoing the “wolf within” threats discussed in adversarial contexts. Malicious actors can release pre-trained models embedded with “privacy backdoors,” designed such that when a victim fine-tunes the model on their private code, the training data is leaked at a significantly higher rate than with benign models [347]. This threat is further compounded by frameworks like “PreCurious,” which demonstrate that even seemingly innocent pre-trained models can be engineered to act as privacy traps, escalating membership inference and data extraction risks during the fine-tuning phase [348].

In the specific domain of code generation and repair, the risk of membership inference—determining whether a specific code snippet was part of the training data—is alarmingly high. Studies focusing on code models have revealed that attackers can predict data membership with high accuracy, posing severe risks for companies whose confidential source code, including SSH keys and proprietary

algorithms, may be embedded in model weights [349]. The implications for automated debugging are dire: if a repair tool suggests a patch that closely mirrors a vulnerable or proprietary snippet from its training data, it may inadvertently expose the logic of unrelated systems or reveal the presence of specific security flaws in the organization’s codebase. Moreover, the reliance on open-source models for debugging assistants introduces the danger of data poisoning, where attackers inject malicious or insecure code into training datasets to manipulate the model’s output, potentially leading to the generation of vulnerable patches or the leakage of sensitive patterns [350].

The challenge of preventing such leakage is further complicated by the limitations of current defensive strategies. While differential privacy (DP) is often proposed as a solution, empirical evidence suggests that it may not provide absolute guarantees; even with sentence-level DP, a non-negligible percentage of PII sequences can still be extracted [344]. Furthermore, the trade-off between model utility and privacy protection remains a critical bottleneck. In sensitive domains, failing to employ robust privacy-enhancing technologies can be considered negligent, yet aggressive privacy measures can degrade the model’s ability to perform complex repair tasks effectively [351]. The situation is worsened by the fact that standard PII masking tools often exhibit demographic biases, leading to uneven protection where certain groups face higher risks of exposure, a flaw that could propagate into code comments or documentation analyzed by debugging agents [352].

Ultimately, the deployment of autonomous debugging agents necessitates a reevaluation of the security lifecycle, moving beyond the active defense against jailbreaks to address these latent extraction risks. The potential for “neural phishing,” where adversaries extract sensitive information like credit card numbers or API keys with significant success rates, underscores the urgency of developing robust countermeasures [353]. As debugging tools evolve to interact more deeply with private repositories, the boundary between helpful automation and privacy violation becomes increasingly blurred. Ensuring the confidentiality of codebases requires not only advanced technical defenses against extraction attacks but also a fundamental shift in how training data is curated, how models are fine-tuned, and how inference outputs are monitored for inadvertent leakage. Without rigorous safeguards, the promise of AI-driven program repair may be overshadowed by the peril of systemic data exposure. This opacity regarding what data the model has memorized and how it leaks transitions naturally into the broader crisis of explainability, where the inability to audit the model’s internal decision-making processes further obscures the origins and implications of such privacy breaches.

7.4 The Opacity of Decision-Making and the Crisis of Explainability

7.4 The Opacity of Decision-Making and the Crisis of Explainability

The privacy vulnerabilities detailed in the previous section—ranging from data leakage to membership inference—reveal a critical blind spot: we often do not know *what* sensitive information a model has memorized. This opacity regarding data provenance transitions naturally into an even more profound crisis concerning decision-making: the inability to explain *why* a Code LLM generates a specific repair. The integration of Large Language Models (LLMs) into Automated Program Repair (APR) and debugging workflows has ushered in an era of unprecedented capability, yet it simultaneously exacerbates this fundamental challenge. Unlike traditional static analysis tools or symbolic repair engines that operate on deterministic logic traces, modern Code LLMs function as probabilistic “black boxes,” where the internal mechanisms driving the selection of a specific patch are often inaccessible to human auditors [354]. This lack of transparency is not merely an inconvenience; in high-stakes security domains such as financial infrastructure, healthcare systems, and

critical cyber-physical environments, the inability to explain why a model generated a particular fix constitutes a severe barrier to adoption and trust [355].

The core of this challenge lies in the complex, non-linear architecture of deep learning models, which obscures the rationale behind their outputs. When an LLM suggests a code modification to fix a vulnerability, it does so based on latent representations learned from vast corpora, rather than an explicit derivation of logical constraints. Consequently, developers are left with a solution that may pass test suites but whose underlying reasoning remains enigmatic [356]. This opacity is particularly dangerous in software repair, where a “correct” patch that passes existing tests might inadvertently introduce subtle logical flaws, backdoors, or performance degradations that are only visible through a deep understanding of the model’s intent [357]. The reliance on post-hoc explanation methods to interpret these black-box decisions has proven insufficient, as recent research indicates that such explanations can be fragile and easily manipulated, leading to a false sense of security [358]. This fragility mirrors the privacy risks discussed earlier: just as models can leak memorized data despite sanitization efforts, they can also generate plausible but ungrounded explanations that mask erroneous or malicious intent.

Furthermore, the crisis of explainability is compounded by the potential for these models to rely on spurious correlations rather than genuine semantic understanding. In the context of debugging, an LLM might generate a patch that appears correct syntactically but is derived from superficial patterns in the training data rather than a robust comprehension of the bug’s root cause [359]. Without interpretable models or rigorous diagnostic tools, it becomes nearly impossible to distinguish between a patch generated through sound reasoning and one resulting from dataset bias or overfitting. This uncertainty undermines the accountability required in safety-critical applications, where every line of code must be justifiable [360]. The situation is further complicated by the fact that standard interpretability techniques, often designed for image classification or tabular data, struggle to provide meaningful insights into the sequential and structural nature of code generation [361].

The stakes are raised significantly when considering adversarial contexts, creating a dangerous synergy with the privacy and security threats previously outlined. If an automated debugging agent is compromised or subjected to a poisoning attack, its opaque decision-making process could lead to the insertion of malicious code that mimics legitimate repairs [357]. In such scenarios, the inability to audit the model’s internal state or trace the origin of a specific token generation means that vulnerabilities could be propagated undetected across entire software ecosystems. Research has shown that black-box models are vulnerable to attacks that can alter their behavior without changing their output on clean data, making the need for transparent, auditable repair mechanisms paramount [362]. Moreover, the phenomenon of “fairwashing” demonstrates how explanation methods can be exploited to rationalize unfair or erroneous decisions, creating a deceptive veneer of legitimacy around flawed patches [363]. This creates a compounding risk: a model that leaks private data (Section 7.3) and cannot explain its reasoning (Section 7.4) sets the stage for the total erosion of moral and legal accountability explored next.

To address this crisis, the field must move beyond treating explainability as an afterthought and instead prioritize the development of inherently interpretable models or robust frameworks for auditing black-box systems. Approaches that infuse domain knowledge into AI models or utilize local-to-global explanation frameworks offer promising avenues for making the decision logic of repair agents more transparent [364] [365]. Additionally, techniques that provide uncertainty quantification alongside predictions can help developers identify when a model is operating outside its confidence bounds, thereby preventing the deployment of risky, unexplained fixes [366]. However,

current methods often fail to provide the granularity required for code-level debugging, where understanding the specific contribution of variables, control flow structures, and API calls is essential [361].

Ultimately, the opacity of LLM-driven program repair represents a critical bottleneck in the realization of autonomous self-healing systems. Until we can reliably audit why a model selects a specific patch and ensure that its reasoning aligns with human safety standards and logical correctness, the deployment of these technologies in high-risk domains will remain fraught with peril. The path forward requires a paradigm shift towards models that offer not just high accuracy, but also verifiable, context-specific interpretability that can withstand rigorous scrutiny [367]. Without such advances, the “black box” nature of these powerful tools will continue to pose unacceptable risks to software integrity and societal security [368]. This persistent inability to audit the model’s internal decision-making processes directly precipitates the broader ethical crisis discussed in the following section, where the absence of explainability creates a vacuum of responsibility, leaving society ill-equipped to assign blame when autonomous agents cause harm.

7.5 Ethical Implications of Autonomous Agents and Accountability Gaps

7.5 The Accountability Void: Ethical Dilemmas in Autonomous Repair

Building upon the crisis of explainability detailed in the previous section, the deployment of autonomous agents for program repair and automated debugging introduces profound ethical dilemmas that fundamentally challenge established frameworks of moral and legal accountability. When the internal reasoning of a model is opaque, as discussed regarding the “black box” nature of LLMs, the traditional chains of responsibility become increasingly fractured. As these systems evolve from passive assistants to active, self-healing entities capable of modifying codebases and executing defensive maneuvers without human intervention, a “responsibility gap” emerges. This phenomenon occurs when the complexity of the AI’s decision-making process, combined with its autonomy, makes it difficult to attribute causal contributions and moral blame to any specific human actor following an incident [369]. In the context of automated debugging, if an autonomous agent successfully patches a vulnerability but inadvertently introduces a critical logic error due to the hallucinations or spurious correlations mentioned earlier, determining liability becomes a complex adjudication between the developers of the agent, the operators who deployed it, and the agent itself.

The prospect of granting legal personhood or specific liability status to AI agents remains a contentious issue, serving as a potential distraction from the root causes of failure. While there is a psychological urge among the public to punish automated agents when they cause harm, studies suggest that people do not believe such punishment fulfills the goals of deterrence or retribution, nor do they view machines as possessing the mental states required for legal punishment preconditions [370]. This creates a dissonance where society demands accountability for AI-induced failures, yet the current legal and moral structures lack the mechanisms to effectively assign it. The “Guilty (Silicon) Mind” further explores this quandary, questioning whether an artificial agent can ever truly understand the moral content of its instructions or be held accountable for torts and crimes committed in human-machine teaming scenarios [371]. When an autonomous debugging agent operates with high sophistication, there is a risk that blame may be inappropriately shifted from the human user or developer to the AI system, simply because the system appears to possess advanced cognitive capabilities [372]. This shift can create a “liability sink” where human operators are absolved of responsibility for outcomes they did not create and may not fully understand, ultimately

undermining the incentive for rigorous safety engineering and exacerbating the trust issues raised by unexplainable decisions.

Furthermore, the autonomy required for effective real-time cyber defense and self-healing introduces the risk of unintended harm through algorithmic feedback loops, a danger amplified by the stochastic nature of LLMs. Autonomous cyber-defense agents, designed to respond to attacks at machine speeds, must make rapid cost-benefit tradeoffs that may result in actions with harmful functional, safety, or ethical consequences [238]. For instance, an agent tasked with isolating a compromised server might inadvertently disconnect critical infrastructure, causing cascading failures in interdependent cyber-physical systems. The propensity of LLMs to hallucinate code or misinterpret context could lead an agent to execute a “fix” that is actually a destructive action, turning a debugging tool into a source of instability. The Responsibility Quantification (ResQu) model highlights that in highly automated systems, human comparative responsibility for outcomes is often low, even when humans are nominally kept “in the loop,” suggesting that policies relying on meaningful human control may be misleading without concrete engineering constraints [373].

A particularly alarming ethical frontier, directly linked to the adversarial vulnerabilities discussed previously, is the potential for compromised autonomous agents to be weaponized, effectively automating cyberattacks. If an adversary can manipulate the training data or exploit prompt injection vulnerabilities within a debugging agent—exploits made to detect due to the model’s opacity—the very tool designed to secure a system could be turned into an offensive weapon. This dual-use nature of AI technology means that civilian debugging systems could be foreseeably applied to active conflict or used to support malicious activities, raising questions about the moral responsibilities of developers to anticipate and mitigate such misuse [374]. The risk is not merely theoretical; as AI systems become more capable, they are increasingly used to automate fraudulent activities and identify dangerous vulnerabilities, necessitating targeted interventions on capabilities to prevent misuse [375]. The possibility of an “AI race,” where competitive pressures compel organizations to deploy unsafe or insufficiently supervised autonomous agents, further amplifies the risk of catastrophic accidents or the inadvertent escalation of cyber conflicts [376].

Addressing these accountability gaps requires a shift from purely agency-focused ethics to a structural approach that considers the socio-technical context in which these agents operate, bridging the gap between the need for interpretability and the reality of deployment costs. It is insufficient to rely on individual developers to act ethically; instead, systemic safeguards such as “Rules of Engagement” (RoE) must be embedded directly into the architecture of autonomous agents to constrain their actions and define clear boundaries for automated responses [377]. Additionally, the concept of “violet teaming” offers a balanced path forward by combining adversarial probing with ethical safety solutions, ensuring that the development of autonomous repair agents prioritizes social benefit alongside technical efficacy [378]. Ultimately, achieving trustworthy autonomous debugging requires a holistic framework that integrates legal liability, ethical design principles, and robust governance mechanisms to ensure that the pursuit of self-healing systems does not come at the cost of moral clarity and human accountability. Without such frameworks, the deployment of autonomous agents risks creating a landscape where harms are frequent, causes are obscure, and no one is held responsible—a situation that will be further complicated by the resource inequalities discussed in the following section, where only the wealthy may afford the safeguards necessary to close these ethical loopholes.

7.6 Resource Inequality and the Computational Cost of Robust Defense

7.6 Resource Inequality and the Computational Cost of Robust Defense

The ethical dilemmas and accountability gaps outlined in the previous section are not merely theoretical; they are deeply entrenched in a practical reality defined by stark socioeconomic disparities. The deployment of Large Language Models (LLMs) for automated program repair introduces a profound challenge: the widening gap between well-funded organizations and resource-constrained developers due to the prohibitive computational costs of robust defense mechanisms. While the promise of AI-driven self-healing systems suggests a democratization of software quality, the reality is that achieving the level of robustness, security, and reliability required to close the “responsibility gap” demands computational resources increasingly concentrated in the hands of a few major technology entities. This disparity creates a “security divide,” where only elite organizations can afford the extensive verification, continual retraining, and adversarial hardening necessary to deploy trustworthy autonomous agents, leaving smaller developers vulnerable to the very vulnerabilities these tools are meant to fix and exacerbating the risk of unaccountable failures.

The primary driver of this inequality is the immense computational overhead associated with training and verifying robust models. As noted in recent analyses, while algorithmic advances make compute use more efficient, the absolute compute required to pioneer new capabilities remains high, creating a “performance effect” where large investors maintain a significant advantage over smaller actors [379]. In the context of program repair, this means that only organizations with access to massive GPU clusters can train models on the diverse, high-quality datasets required to generalize across complex bug patterns without overfitting or hallucinating insecure patches—a critical factor in preventing the liability sinks discussed previously. Furthermore, the necessity for continual learning to adapt to evolving codebases and emerging vulnerability classes exacerbates this cost. Adversarial attacks on continual learning systems demonstrate that models are susceptible to “false memories” and backdoor injections if not rigorously defended, yet implementing robust regularization or generative replay defenses adds significant computational burden [380]. Smaller entities, lacking the infrastructure for such expensive retraining cycles, risk deploying static models that quickly become obsolete or insecure, thereby failing to meet the dynamic safety standards required for ethical autonomy.

Beyond training, the cost of verification constitutes a critical barrier to equitable security. Ensuring that an LLM-generated patch is not only functionally correct but also semantically secure often requires formal verification, symbolic execution, or extensive fuzzing, all of which are computationally intensive. The challenge of scaling verification efforts to match system complexity is acute; for instance, improving the performance of proof engines for verified compilers requires novel primitives and significant engineering effort to handle large-scale systems [381]. Similarly, verifying the integrity of outsourced computation or training processes often relies on Trusted Execution Environments (TEEs) or cryptographic proofs, which can incur substantial performance penalties. While some approaches attempt to mitigate these costs through probabilistic verification or optimized hardware accelerators, the baseline requirement for high-assurance verification remains out of reach for many [382]. The trade-off is stark: organizations that cannot afford rigorous verification may deploy repair agents that inadvertently introduce subtle vulnerabilities or fail to detect sophisticated attacks, perpetuating a cycle of insecurity that undermines the moral clarity sought in accountable AI systems.

The issue of resource inequality is further compounded by the specific challenges of deploying secure AI in resource-constrained environments, such as IoT and edge devices. These environments often

lack the compute, memory, and power necessary to run robust ML models or implement costly security measures like fully homomorphic encryption or complex attestation protocols [383]. Although research into lightweight security architectures and data-oblivious accelerators offers hope, the overhead for maintaining confidentiality and integrity in outsourced or distributed settings remains a significant hurdle [384]. For example, protecting client data during outsourced computation using dynamic information flow tracking introduces area and power overheads that may be unacceptable for low-end devices [384]. Consequently, developers working in these domains may be forced to rely on less secure, heuristic-based repair tools or forego automated debugging altogether, leaving critical infrastructure exposed to the kinds of cascading failures and algorithmic feedback loops warned against in the discussion of autonomy risks.

Moreover, the economic incentives surrounding secure software development often favor speed over safety, particularly for smaller teams operating under tight budgets. Secure coding education and adherence to guidelines are essential, yet surveys indicate a significant gap in awareness and compliance among industrial developers, partly due to the perceived complexity and cost of implementing secure practices [385]. When the tools designed to assist with these practices—such as LLM-based repair agents—require expensive subscriptions or cloud compute credits to function reliably, the adoption barrier increases. This is particularly problematic given that financial impacts of vulnerabilities are severe, yet the cost of prevention via advanced AI tools may be prohibitive for smaller firms [386]. The result is a stratified ecosystem where high-stakes domains like finance and defense can afford “resilience-by-design” through expensive, verified AI agents, while the broader software community relies on reactive, less robust solutions, effectively institutionalizing the resource inequalities that threaten to undermine global software safety.

Finally, the path toward closing this gap requires not just algorithmic efficiency but a fundamental shift in how AI infrastructure is shared and governed, serving as a bridge to the methodological solutions proposed in the following section. Proposals for decentralized training using consumer-level GPUs suggest a potential avenue for democratizing access to compute resources [387]. However, realizing this vision demands overcoming challenges related to device heterogeneity, network bandwidth, and privacy, which themselves require sophisticated orchestration and security mechanisms that may reintroduce complexity and cost. Until efficient, low-cost methods for robust verification and continual learning become mainstream, the computational cost of robust defense will remain a formidable barrier, entrenching security disparities and limiting the universal benefit of AI-driven program repair. Addressing these structural inequities necessitates the evolution of development methodologies themselves, moving toward a Secure Development Lifecycle that integrates threat modeling and automated red-teaming to ensure that robustness is not solely a function of computational wealth.

7.7 Toward a Secure Development Lifecycle for AI-Driven Repair

7.7 Toward a Secure Development Lifecycle for AI-Driven Repair

While the preceding section highlighted the severe resource inequalities that threaten to stratify access to robust AI-driven repair, addressing these computational barriers alone is insufficient without a corresponding evolution in development methodologies. The integration of Large Language Models (LLMs) into automated program repair necessitates a paradigm shift from reactive patching to proactive fortification, fundamentally reimagining the Software Development Lifecycle (SDLC). Traditional SDLC models, which often treat security as a post-development compliance check, are ill-equipped for AI-driven systems where vulnerabilities can emerge organically from training data,

architectural design, or dynamic environmental interactions. To bridge the gap between high-cost robustness and practical deployment, a specialized Secure Development Lifecycle (Secure SDL) for AI-driven repair must be established. This framework integrates threat modeling, automated red-teaming, and standardized certification protocols at every stage, ensuring that autonomous debugging agents are not only effective but also resilient against adversarial manipulation and capable of operating within strict ethical boundaries, regardless of the underlying computational constraints.

The foundation of this new lifecycle lies in the early integration of comprehensive threat modeling that explicitly accounts for the unique attack surfaces of generative models. Unlike conventional software, AI-driven repair agents possess dual-use potential: they can inadvertently generate malicious code or be coerced via prompt injection into introducing vulnerabilities. Therefore, threat modeling must expand beyond traditional code analysis to include the training corpus, the inference engine, and the feedback loops used for iterative refinement. Recent research emphasizes the need for proactive risk management frameworks that identify potential unintended uses and misuses of AI systems before deployment [388]. By adopting a “secure by design” philosophy, developers can anticipate adversarial scenarios—such as an attacker manipulating a repair agent to introduce backdoors or exfiltrate sensitive logic—and architect defenses that are computationally efficient enough to be viable even for resource-constrained teams. This aligns with broader calls for establishing multi-level governance and embedding responsible AI principles directly into the system architecture from the conception phase [389].

Central to operationalizing this proactive stance is the institutionalization of automated red-teaming throughout the development and deployment phases. Given the prohibitive costs of manual security testing described in the context of resource inequality, the Secure SDL must leverage continuous, automated red-teaming pipelines to stress-test repair agents against diverse adversarial prompts and scenarios. This approach maximizes security coverage while minimizing human labor costs. Frameworks such as Multi-round Automatic Red-Teaming (MART) demonstrate the efficacy of using adversarial agents to iteratively discover and patch safety flaws, significantly reducing violation rates while maintaining model helpfulness [390]. Furthermore, specialized environments like TanksWorld provide critical testbeds for evaluating AI safety in complex, multi-agent competitive settings, ensuring that repair agents maintain robustness even when faced with sophisticated, adaptive adversaries [391]. The adoption of such automated methodologies allows organizations to shift from sporadic, expensive security audits to a culture of continuous vulnerability discovery, effectively turning red-teaming into a standard, scalable component of the CI/CD pipeline for AI models.

However, technical robustness achieved through red-teaming must be complemented by standardized certification and transparency mechanisms to build trust among stakeholders, particularly when resource disparities limit the ability of smaller entities to perform extensive internal validation. The opacity of deep learning models creates significant information asymmetries between developers, users, and regulators, making it difficult to verify the safety claims of autonomous repair systems. To bridge this gap, the industry must move toward rigorous certification regimes that validate both the process and the product. Certification programs should go beyond self-attestation, incorporating third-party audits that assess adherence to ethical guidelines, robustness against known attack vectors, and the effectiveness of fallback mechanisms [392]. Complementing certification, the use of “model cards” and detailed risk profiles can provide downstream users with transparent disclosures regarding a model’s limitations, training data provenance, and known failure modes [393]. These documents serve as essential tools for procurement and deployment

decisions, ensuring that organizations—even those with limited resources—are fully aware of the risks associated with integrating AI-driven repair tools into their critical infrastructure.

Moreover, the Secure SDL must account for the dynamic nature of AI risks, which evolve post-deployment as models encounter novel data and emerging threat landscapes. Static risk assessments are often inadequate for AI-controlled systems; instead, dynamic approaches that leverage real-time data logging and behavioral analysis are required to detect anomalies and trigger self-adaptation mechanisms [394]. This aligns with the concept of “violet teaming,” which combines adversarial probing with defensive solutions to proactively manage risks and steer AI trajectories toward societal good [378]. By integrating these continuous feedback loops, organizations can ensure that their AI repair agents remain secure and reliable over time, adapting to new vulnerabilities as they arise without requiring the massive retraining cycles that exacerbate computational costs.

Finally, the successful implementation of this Secure SDL requires a collaborative ecosystem involving developers, regulators, and the broader research community to mitigate the effects of resource inequality. The establishment of “safe harbors” for independent security research is crucial to encourage the discovery and reporting of vulnerabilities without fear of legal reprisal, fostering a more transparent and resilient AI ecosystem [395]. Additionally, international cooperation and the sharing of threat intelligence, as proposed in frameworks like CTI4AI, are essential for creating a unified defense against global adversaries targeting AI supply chains [396]. By synthesizing threat modeling, automated red-teaming, rigorous certification, and continuous monitoring into a cohesive lifecycle, the software engineering community can transition from a reactive posture to one of proactive fortification. This structured approach not only enhances security but also provides a scalable pathway for diverse organizations to adopt AI-driven program repair, ensuring it serves as a trustworthy pillar of future software reliability rather than a privilege of the computationally wealthy.

8 Future Directions and the Roadmap to Autonomous Self-Healing Systems

8.1 From Model-Driven to Learned Immunity: Biological Inspiration for Self-Healing Ecosystems

8.1 From Model-Driven to Learned Immunity: Biological Inspiration for Self-Healing Ecosystems

The evolution of autonomous self-healing systems represents a fundamental paradigm shift from rigid, specification-dependent architectures to adaptive, data-driven ecosystems capable of learning resilience. Historically, the development of self-healing software has been dominated by model-driven approaches, where system correctness is verified by comparing actual runtime behavior against a pre-defined model of intended operation. In these traditional frameworks, a controller initiates repair actions only when the system deviates from its explicit specification. However, this reliance on static models introduces significant bottlenecks; developing comprehensive specifications for modern, complex software is notoriously difficult, and maintaining these models as the system evolves often proves impractical. Consequently, there is a growing consensus that the future of robust software engineering lies in transitioning toward “learned immunity,” drawing profound inspiration from Biological Immune Systems (BIS) and the field of Artificial Immune Systems (AIS). This biological metaphor offers a pathway to systems that can autonomously detect, diagnose, and contain unanticipated problems without requiring exhaustive prior knowledge of every possible failure mode [397].

The core advantage of adopting an immune-inspired architecture is its inherent capacity for anomaly detection and diagnosis in dynamic environments. Unlike model-driven systems that fail when encountering scenarios outside their hardcoded rules, AIS mimics the adaptive nature of biological immunity, which distinguishes between “self” (normal operation) and “non-self” (pathogens or faults) through a distributed, decentralized mechanism. This approach allows software systems to learn what constitutes normal behavior by observing the system in operation, rather than having it explicitly programmed. Research indicates that AIS is particularly well-suited for building such self-healing capabilities because it can adapt to changing environments and identify novel threats that were not anticipated during the design phase [398]. By viewing software components as diverse populations of simple classifiers—analogue to T and B cells in the human body—developers can construct systems that scale effectively and maintain robustness against a wide array of perturbations [399].

A critical component of this transition is the move from passive monitoring to active, closed-loop control mechanisms that emulate the body’s feedback loops. Recent advancements have demonstrated the viability of designing self-healing neural networks that utilize closed-loop control formulations to automatically detect and rectify vulnerabilities caused by various attacks or data perturbations. These systems do not require a priori information about the specific nature of an attack; instead, they rely on margin-based analysis and control solvers to restore functionality, thereby handling a broad class of unforeseen errors [400]. This capability is essential for modern cyber-physical systems (CPS) and Internet of Things (IoT) deployments, where heterogeneity and dynamicity make static fault definitions obsolete. For instance, adaptive fault detection mechanisms that exploit redundancy with uncertainties in space and time have been successfully deployed in real-world prototypes, such as rovers, to trigger recovery processes based on learned knowledge bases rather than fixed thresholds [401].

Furthermore, the integration of machine learning techniques into these immune-inspired frameworks enables the system to learn recovery strategies dynamically. In complex reactive systems, traditional self-healing often relies on predefined hooks and replacement strategies that lack the expressiveness to manage diverse failure types. By employing Reinforcement Learning (RL) within an immune-inspired context, systems can learn optimal recovery strategies based on users’ corrective sequences or historical success patterns. This approach allows monitors, defined as predicates for system properties, to detect failures at any execution point and activate learned recovery policies that overwrite base system behavior only when necessary [402]. Such dynamic adaptation is crucial for maintaining system integrity in non-stationary environments where the distribution of faults may shift over time, setting the stage for the specific continual learning challenges discussed in the subsequent section.

The application of these principles extends beyond software into the realm of robotics and physical infrastructure, where the concept of “learned immunity” facilitates rapid adaptation to physical damage. Inspired by how animals adapt to injuries, intelligent trial-and-error algorithms allow robots to create maps of high-performing behaviors and discover compensatory strategies when damaged, all without pre-specified contingency plans. This bio-inspired adaptability ensures that systems can continue to function effectively even when components fail, mirroring the resilience found in natural organisms [403]. Similarly, in the domain of cellular networks, data-driven machine learning techniques are being classified and optimized to address challenges such as data imbalance and cost insensitivity, enabling self-organizing networks that can autonomously mitigate cell outages [404; 405].

Despite the promise of learned immunity, challenges remain in scaling these systems and ensuring

their interpretability. The complexity of implementing large-scale AIS, particularly in managing the multiplicity of input data and avoiding catastrophic forgetting in continual learning scenarios, requires sophisticated architectural designs. As software ecosystems evolve, the very mechanisms that enable adaptation must be carefully managed to prevent the erosion of previously acquired knowledge. Techniques such as intentional forgetting have been proposed to help deep reinforcement learning systems prioritize key problem-solving skills while discarding minor behaviors that hinder adaptation to drifted environments [406]. Moreover, the evaluation of these self-adaptive systems demands new methodologies, such as chaos engineering, to systematically assess their resilience against injected faults and unanticipated conditions [407; 408].

Looking forward, the roadmap to truly autonomous self-healing ecosystems involves deepening the synergy between biological inspiration and advanced machine learning. Future research must focus on developing standardized frameworks that integrate innate immunity (fast, generic responses) with adaptive immunity (slow, specific learning) to create multi-layered defense mechanisms. As noted in reviews of immuno-inspired applications, there is a need to move beyond ad-hoc implementations and toward comprehensive architectures that fully leverage modern immunological interpretations and robot embodiment [409]. By shifting from rigid model-driven constraints to flexible, learned immunity, the next generation of software systems will possess the autonomy to survive and thrive in increasingly complex and hostile operational landscapes. This transition naturally leads to the critical examination of how such systems can sustain their learning capabilities over time through continual learning and intentional forgetting, ensuring they evolve alongside the codebases they protect.

8.2 Continual Learning and Intentional Forgetting for Evolving Codebases

8.2 Continual Learning and Intentional Forgetting for Evolving Codebases

Building upon the transition from rigid model-driven architectures to flexible, “learned immunity” discussed in the previous section, a critical temporal challenge emerges: how can self-healing systems sustain their adaptive capabilities over time? The deployment of autonomous program repair agents in real-world software ecosystems necessitates a fundamental shift from static, one-off debugging models to dynamic systems capable of lifelong adaptation. Software codebases are inherently non-stationary, evolving through continuous commits, dependency updates, and shifting architectural patterns. Consequently, a repair agent trained on a specific version of a codebase may suffer from performance degradation as the environment drifts, a phenomenon analogous to the environmental drifts observed in deep reinforcement learning systems [406]. To maintain repair efficacy, future self-healing systems must integrate Continual Learning (CL) mechanisms that allow agents to accumulate knowledge over time without succumbing to catastrophic forgetting, where learning new bug fixes erases the ability to resolve previously encountered issues.

A primary challenge in adapting CL to program repair is managing the delicate trade-off between plasticity—the ability to learn new repair strategies for emerging bug patterns—and stability—the retention of proven fixes for legacy code. Standard gradient-based optimization often leads to rapid performance drops on past tasks when exposed to new data distributions [410]. In the context of evolving codebases, this manifests as an agent forgetting how to patch vulnerabilities in older library versions while learning to fix issues in updated APIs. Research indicates that catastrophic forgetting is not merely a nuisance but a critical barrier to deploying agents in open environments, where intra-class distribution shifts can severely impair recognition accuracy even if the shift magnitude is negligible in joint learning scenarios [411]. Therefore, robust repair agents

must employ strategies that explicitly mitigate this forgetting to ensure long-term viability.

One promising direction to address this paradox is the adoption of intentional forgetting mechanisms. Counter-intuitively, retaining all historical knowledge can hinder adaptation to significant environmental shifts. Recent studies in self-healing systems propose that deliberately erasing minor or obsolete behaviors can systematically prioritize the adaptation of key problem-solving skills, thereby reducing healing time and improving convergence in drifted environments [406]. For code LLMs, this could translate to architectures that actively discard outdated syntactic patterns or deprecated API usage statistics that conflict with modern coding standards, allowing the model to focus its capacity on current best practices. This aligns with biological inspirations where active forgetting regulates synaptic expansion to benefit learning [412]. By integrating such mechanisms, repair agents can avoid the accumulation of negative transfer, where old knowledge interferes with the learning of new, conflicting repair templates, thus refining the “learned immunity” loop introduced earlier.

Furthermore, the architecture of future repair agents should leverage parameter-efficient techniques to isolate task-specific knowledge, ensuring that adaptation to new contexts does not compromise existing competencies. Methods such as sub-network discovery and soft-masking have shown efficacy in preventing interference between mixed tasks by isolating knowledge into specific subnetworks while enabling knowledge transfer through shared representations [413]. In a software engineering context, this could mean maintaining distinct parameter subsets for different programming languages or framework versions within a single large model, ensuring that updates to a Python 3.12 repair strategy do not degrade performance on Python 2.7 legacy systems. Similarly, prompt-based approaches like the Progressive Prompt Decision Transformer suggest that dynamically appending decision tokens for new tasks can foster task-specific policies while retaining knowledge from previous studies [414]. Applying this to code LLMs, agents could generate specific “repair prompts” or adapter layers for new vulnerability classes without altering the core weights responsible for general coding logic.

Experience replay remains a cornerstone of CL, yet its application to code repair requires innovation due to the vastness of code corpora and privacy concerns. Traditional replay buffers storing raw code snippets may be impractical or insecure. Emerging frameworks propose generating high-quality synthetic data or utilizing class prototypes to guide learning without storing original data points [415]. For program repair, this suggests a future where agents replay abstracted bug-fix patterns or latent representations of vulnerabilities rather than proprietary source code. Additionally, saliency-guided hidden associative replay offers a path toward more human-like memory retention, archiving only salient data segments to enable swift and near-perfect recall of critical fix patterns [416].

Theoretical advancements also highlight the importance of gradient projection and trust regions in managing task correlations. Techniques like Trust Region Gradient Projection (TRGP) facilitate forward knowledge transfer by selectively reusing frozen weights from related old tasks, ensuring that new learning does not overwrite crucial historical knowledge [417]. In automated debugging, this implies that an agent learning to fix SQL injection flaws could leverage shared semantic understanding from previously learned cross-site scripting (XSS) fixes, provided the optimization stays within a “trust region” that protects the integrity of the XSS repair capabilities. Moreover, recent findings suggest that certain levels of synaptic downscaling, inspired by sleep mechanisms, can balance the retention of early tasks with the ability to learn new ones, offering a bio-inspired roadmap for scheduling retraining cycles in autonomous repair agents [418].

Ultimately, the realization of autonomous self-healing systems depends on solving the continual

learning paradox. Agents must be robust against adversarial attacks and distribution shifts while maintaining a compact, efficient memory footprint [419]. By synthesizing intentional forgetting, parameter isolation, and advanced replay strategies, the next generation of Code LLMs can evolve alongside the software they protect, ensuring sustained repair efficacy in an ever-changing digital landscape. However, while these temporal adaptations address the evolution of code over time, a parallel spatial challenge arises when deploying these continuously learning agents at the network periphery. The necessity for resource-efficient, on-device repair in edge and IoT contexts, where computational constraints and connectivity issues prevail, forms the critical focus of the subsequent section.

8.3 Resource-Efficient and On-Device Repair for Edge and IoT Contexts

8.3 Resource-Efficient and On-Device Repair for Edge and IoT Contexts

While the preceding section addressed the temporal challenges of adapting repair agents to evolving codebases through continual learning, a parallel spatial challenge arises when deploying these agents at the network periphery. The proliferation of Internet of Things (IoT) devices and edge computing paradigms has fundamentally shifted the landscape of software deployment, moving intelligence from centralized cloud data centers to resource-constrained edge nodes. However, this decentralization introduces acute obstacles for automated program repair, as edge devices—ranging from microcontroller units to mobile sensors—operate with limited computational power, restricted memory, and intermittent connectivity. In such environments, reliance on cloud-based repair mechanisms is often impractical for latency-critical or privacy-sensitive applications. Consequently, realizing the autonomous self-healing ecosystems envisioned in future certification frameworks requires a paradigm shift toward resource-efficient, on-device repair capabilities that function independently of continuous cloud connectivity. This necessitates the development of lightweight surrogate models, adaptive inference strategies, and distributed repair frameworks tailored for heterogeneous edge environments.

A primary barrier to on-device repair is the prohibitive computational cost of running large-scale diagnostic and generative models traditionally used for fault localization and patch synthesis. To overcome this, future research must prioritize “tinyML” models and lightweight surrogate architectures capable of executing recovery logic within strict energy and memory budgets. Hierarchical inference offers a promising pathway, wherein a lightweight local model initially attempts to diagnose and resolve faults, offloading only complex, high-uncertainty cases to more powerful edge servers or the cloud. This approach has been demonstrated to be feasible for machine fault detection, significantly reducing latency and bandwidth consumption while maintaining high accuracy [420]. Furthermore, the development of efficient self-supervised learning techniques for low-compute networks is critical. Recent findings suggest that optimizing view sampling strategies rather than relying on knowledge distillation can establish state-of-the-art performance for self-supervised models on resource-constrained hardware, enabling them to learn robust representations of normal system behavior without the need for massive labeled datasets [421].

Beyond model compression, the architecture of the repair mechanism must evolve to leverage the collective capacity of distributed edge networks. Techniques that partition deep neural networks across multiple collaborative devices can accelerate both fault diagnosis and the execution of repair strategies. Frameworks like SEIFER illustrate how partitioning and placing model segments across a cluster of resource-constrained nodes can maximize throughput and provide node fault tolerance, a principle that can be directly adapted for distributed self-healing agents [422]. Similarly, the HALP

framework demonstrates that optimizing task partitioning ratios among collaborative edge devices can achieve significant inference acceleration, which is essential for meeting the stringent real-time deadlines required in cyber-physical systems [423]. By leveraging such distributed architectures, self-healing systems can pool the fragmented resources of an IoT swarm to execute complex repair algorithms that would be impossible for a single isolated device.

Resilience against communication failures constitutes another critical dimension of on-device repair, particularly given the unreliable nature of wireless IoT networks. The latency incurred by retransmitting data for cloud-based analysis can render a system vulnerable during the repair window. Innovative approaches such as split inference with no retransmissions (SI-NR) emulate packet loss during training via dropout techniques, enabling models to remain robust and accurate even when communication links fail. Applying similar robustness principles to repair agents would allow them to execute local recovery protocols based on partial or noisy observations without waiting for cloud confirmation [424]. Additionally, communication-oriented model tuning, which fine-tunes models to withstand unreliable links, provides a methodological foundation for building repair agents that do not degrade in performance under network stress [425].

The heterogeneity of edge devices further complicates the deployment of uniform repair solutions, demanding frameworks that incorporate adaptive mechanisms to dynamically adjust strategies based on specific device capabilities and current loads. The Autodidactic Neurosurgeon system exemplifies this by using online learning to automatically determine optimal partition points for inference in real-time, adapting to changing system environments without offline profiling [426]. Extending this adaptability to repair workflows means developing agents that can seamlessly switch between full local execution, collaborative peer-to-peer repair, and cloud-assisted modes depending on fault severity and resource availability. Moreover, the integration of self-stabilizing control planes ensures that distributed systems can recover from arbitrary state corruptions within a constant number of communication rounds, providing a theoretical guarantee for rapid recovery in volatile fog ecosystems [427].

Finally, privacy and data sovereignty concerns in domains like healthcare and smart cities necessitate on-device learning and repair capabilities that avoid transmitting sensitive operational data. Federated learning frameworks enhanced with secure multi-party computation offer a roadmap for collaborative model improvement without exposing raw data, a principle that can be extended to share “repair knowledge” across a network of devices while preserving local privacy [428]. As edge devices become increasingly autonomous, the ability to perform continual adaptation through cloud-device collaborative paradigms will be essential. Techniques such as uncertainty-guided sampling allow devices to identify distribution shifts and request only the most critical updates from the cloud, ensuring that on-device repair models remain effective against evolving fault patterns without overwhelming network resources [429]. By synthesizing these advancements in lightweight modeling, distributed inference, and adaptive learning, the next generation of self-healing systems will achieve true autonomy at the edge. This foundational resilience is a prerequisite for the rigorous dynamic certification and runtime assurance schemes discussed in the following section, ensuring that autonomous agents can operate safely and reliably even in the most disconnected and resource-constrained environments.

8.4 Standardized Certification and Dynamic Trust Governance for Autonomous Agents

8.4 Dynamic Certification and Runtime Assurance for Autonomous Healing Agents

While Section 8.3 established the foundational resilience required for on-device repair in resource-constrained edge environments, the deployment of such autonomous capabilities in safety-critical domains necessitates a parallel evolution in how these systems are certified and governed. The transition toward fully autonomous self-healing software ecosystems demands a fundamental reimagining of safety and reliability protocols. Traditional certification paradigms, which rely heavily on static verification and pre-deployment assurance cases, are increasingly inadequate for systems characterized by dynamic evolution, learning capabilities, and unpredictable operational environments. As autonomous debugging agents become pervasive in critical domains, certification procedures must evolve to match the dynamism and inherent unpredictability of these systems [430]. The core challenge lies in the fact that current strategies often fail to adequately reflect software changes in constantly evolving systems or support the complex cooperation required for coordinated autonomous actions [431]. Consequently, the future of autonomous agent deployment hinges on developing standardized certification schemes that integrate dynamic, runtime trust governance rather than relying solely on snapshot evaluations at the time of release.

A pivotal shift in this domain involves moving from static, design-time guarantees to continuous, runtime assurance mechanisms. The concept of Runtime Assurance (RTA) has emerged as a critical component for maintaining safety in systems utilizing untrusted or experimental controllers, such as those powered by large language models. Frameworks like SOTER demonstrate the viability of integrating runtime assurance directly into the programming model, allowing uncertified high-performance components to operate alongside certified safe controllers, thereby ensuring safety specifications are met without sacrificing performance [432]. Similarly, the Certified Control architecture proposes a runtime monitor that acts as a small trusted base, checking certificates of safety assembled by the main controller, effectively decoupling the complex task of finding a safe action from the simpler task of verifying it [433]. For autonomous debugging agents, this implies a future where proposed patches are not merely statically analyzed but are subjected to rigorous, real-time verification against formal safety constraints before application, creating a “safety envelope” that aligns with the resilience requirements of edge environments discussed previously.

To address the complexity of modern autonomous ecosystems, a multi-layer trust governance framework is essential. Such a framework must orchestrate interactions between various stakeholders and technical layers, ranging from individual agent behavior to systemic ethical compliance. Research suggests a conceptual Multi-Layer Trust Governance Framework that supports safe and ethical operation through distinct layers, addressing the shortcomings of narrow certification focuses that often neglect ethical behavior and system cooperation [430]. This approach aligns with broader multilevel governance models that map actors across international, national, and organizational levels, enabling the evolution of tools to guide agent behavior effectively [434]. In the context of software repair, this could manifest as a hierarchy where local agents perform immediate self-correction under strict runtime monitors, while higher-level governance entities oversee aggregate behavior, policy adherence, and long-term learning trajectories.

Furthermore, the certification of learning-based components requires methodologies capable of validating models in untrained operational environments. A science-based certification methodology advocates for the profound integration of domain knowledge and analytical models with data-driven AI to quantify physical inconsistencies and ensure reliability even when training data is limited [435]. This is particularly relevant for debugging agents that must generalize to novel bug

patterns or codebases they have not encountered during training. Complementing this, frameworks for deep learning certification emphasize inherently safe design and runtime error detection, including uncertainty quantification and out-of-distribution detection, as prerequisites for deploying AI in safety-critical applications [436]. These mechanisms provide the necessary confidence bounds for agents operating in the heterogeneous and volatile edge landscapes described in Section 8.3.

The role of dynamic trust management cannot be overstated in fostering resilient autonomous systems. Trust is not a static attribute but a dynamic variable that must be continuously assessed and calibrated based on real-time interactions and observed behaviors. Approaches that combine adaptive risk-based security with trust-based security create a synergy that enhances the overall robustness of autonomous messaging and control systems [437]. In multi-agent scenarios, decentralized trust management using factor graphs allows agents to dynamically assess the trustworthiness of peers based on consistency and cooperation, facilitating safe collaboration without centralized oversight [438]. For autonomous debugging agents, this suggests a future where agents operate within a “digital self” ecosystem, governed by distributed principles that allow for autonomous decision-making while adhering to established safety boundaries [439].

Moreover, the integration of policy-as-a-service (PaaS) frameworks offers a promising avenue for embedding legal and ethical constraints directly into the runtime environment of autonomous agents. By offloading policy design to experts and making these policies accessible as services, developers can ensure that agents adhere to complex regulatory requirements dynamically [440]. This is crucial for addressing the “fuzzy” socio-technical aspects of trustworthiness that pure technical verification often misses. Additionally, red-teaming frameworks specifically designed for AI security, such as those proposed for maritime autonomous systems, highlight the necessity of proactive and reactive evaluation methods to uncover vulnerabilities like adversarial attacks before they can be exploited in the wild [441].

Ultimately, the road to trustworthy autonomous self-healing systems requires a holistic synthesis of formal verification, runtime monitoring, dynamic trust governance, and ethical oversight. The limitations of verifying agentic behavior through traditional means suggest that we must eradicate the language of certainty and instead embrace probabilistic and layered assurance strategies [442]. By adopting standardized yet adaptable certification schemes that leverage multi-layer governance and runtime assurance, the software engineering community can build the necessary confidence to deploy autonomous debugging agents in high-stakes environments. This robust governance infrastructure serves as the critical prerequisite for the next evolutionary step: architecting fully autonomous scientific agents capable of proactive, hypothesis-driven self-healing, as detailed in the following section.

8.5 Realizing Fully Autonomous Agents via Scientific Debugging and Tool Integration

8.5 Architecting Autonomous Scientific Agents for Self-Healing Software

Building upon the dynamic certification and runtime assurance frameworks outlined previously, the realization of fully autonomous agents capable of self-healing software systems represents the operational pinnacle of automated program repair. This evolution necessitates a fundamental shift from reactive patch generation to proactive, scientific debugging architectures. In this new paradigm, Large Language Models (LLMs) transcend their role as mere code completers to function as autonomous scientists that formulate hypotheses, design experiments, execute repairs, and validate outcomes through rigorous tool interaction. The architectural roadmap for such systems hinges on the seamless fusion of LLM-driven reasoning, reinforcement learning for strategy acquisition, and

sophisticated agentic frameworks capable of navigating complex action spaces with minimal human intervention, all while operating within the trusted boundaries established by runtime governance.

At the core of this vision is the adoption of a “scientific method” approach to debugging, where the agent treats software failures as phenomena to be explained rather than errors to be blindly overwritten. This involves the agent autonomously generating hypotheses regarding root causes based on observed symptoms and system logs, directly addressing the need for dynamic trust assessment discussed in prior sections. To achieve this, future agents must leverage frameworks that allow for hypothesis proposal and verification through direct environmental interaction. For instance, the concept of Self-Driven Grounding (SDG) illustrates how an agent can propose sub-goals as hypotheses and verify their feasibility by interacting with the underlying environment, thereby learning generalized skills from successful validations [443]. In the context of program repair, this translates to an agent that hypothesizes a specific line of code as the fault source, generates a potential fix, and immediately tests this hypothesis against the runtime environment. This iterative cycle of hypothesis and verification is critical for moving beyond static analysis toward dynamic, context-aware repair, ensuring that every proposed action is validated before it impacts system stability.

However, formulating a hypothesis is insufficient without the ability to execute complex recovery strategies in uncertain environments. Reinforcement Learning (RL) emerges as a pivotal mechanism for acquiring these robust recovery policies, bridging the gap between theoretical safety constraints and practical adaptation. Traditional rule-based repair systems often fail when encountering novel or undefined failure modes, whereas RL agents can learn to adapt their behaviors based on feedback signals. Research in autonomous robotic systems demonstrates that RL can be effectively used to learn recovery strategies from user corrective sequences or by optimizing against specific utility functions, enabling systems to recover from failures even in complex reactive environments [402]. Furthermore, the integration of RL allows agents to refine their repair strategies over time, adapting to evolving codebases and shifting failure patterns. The work on “Adaptable Recovery Behaviors in Robotics” highlights how reinforcement learning can dynamically refine recovery behavior parameters, enabling tailored responses to a wide array of failure scenarios with minimal human intervention [444]. Translating this to software engineering, an autonomous debugging agent could utilize RL to optimize its sequence of tool invocations—such as running specific test suites, invoking static analyzers, or rolling back commits—to maximize the probability of a successful repair while minimizing computational cost, thereby aligning with the efficiency requirements of runtime assurance mechanisms.

The efficacy of these scientific and learning-based approaches is fundamentally dependent on the agent’s ability to integrate and orchestrate a diverse set of external tools. A truly autonomous agent must transcend text generation to actively wield compilers, debuggers, version control systems, and test runners within a unified action space. The challenge lies in efficiently navigating the expansive action space created by the multitude of available API functions and tools. Recent advancements propose tree search-based planning algorithms, such as ToolChain, *which formulate the action space as a decision tree and utilize A search to efficiently prune high-cost branches, identifying the most valid and low-cost path for tool execution* [87]. By incorporating such efficient navigation strategies, autonomous debugging agents can avoid getting trapped in local optima or exhaustively traversing invalid action sequences, thereby significantly improving the speed and success rate of repair operations while adhering to the resource constraints inherent in real-time monitoring systems.

Moreover, the architecture of these agents must support a high degree of autonomy in planning

and execution, moving away from fixed prompts to dynamic, state-aware decision-making. The “RepairAgent” framework serves as a pioneering example of this paradigm, treating the LLM as an agent capable of autonomously planning and executing actions by invoking suitable tools based on gathered information and feedback [445]. This approach allows the agent to freely interleave information gathering, repair ingredient collection, and fix validation, guided by a finite state machine that ensures logical progression through the debugging workflow. Future iterations of such systems will likely incorporate more sophisticated cognitive architectures that blend the generative capabilities of LLMs with the structured reasoning of symbolic AI. The synergistic integration of LLMs and cognitive architectures offers a pathway to robust AI, where bottom-up learning extracts symbolic representations to guide top-down prompt engineering, effectively mitigating the weaknesses of purely neural or purely symbolic approaches [446]. Such hybrid architectures are essential for satisfying the multi-layer trust governance requirements, ensuring that agent decisions are both data-driven and logically verifiable.

Finally, the journey toward full autonomy requires addressing the issue of hallucination and ensuring the reliability of the agent’s decisions, a prerequisite for the certification schemes discussed earlier. As foundation models are increasingly deployed in decision-making roles, the risk of generating plausible but incorrect repair plans becomes a critical bottleneck. Future systems must embed mechanisms for quantifying certainty and detecting hallucinations in real-time, ensuring that the agent only commits to repair actions with high confidence [447]. By combining scientific hypothesis testing, reinforcement learning for adaptive strategy, efficient tool navigation, and robust cognitive architectures, the field is poised to realize fully autonomous agents that not only fix bugs but also evolve alongside the software they protect. This capability sets the stage for the next critical discussion: balancing these reactive, learning-driven recovery mechanisms with proactive, resilience-by-design principles in Cyber-Physical Systems, ensuring that autonomous self-healing operates in harmony with inherent structural robustness.

8.6 Resilience-by-Design versus Resilience-by-Reaction in Cyber-Physical Systems

8.6 Resilience-by-Design versus Resilience-by-Reaction in Cyber-Physical Systems

Building upon the vision of autonomous, learning-driven repair agents established in the previous section, the deployment of such capabilities within Cyber-Physical Systems (CPS) necessitates a broader strategic framework that balances reactive recovery with proactive robustness. The future trajectory of self-healing CPS hinges on a sophisticated synthesis of two historically distinct paradigms: resilience-by-design and resilience-by-reaction. While traditional approaches have often treated these as mutually exclusive strategies, the next generation of intelligent systems must seamlessly integrate the proactive architectural robustness discussed in prior design literature with the reactive runtime adaptation enabled by autonomous agents. This integration is critical for managing the inherent uncertainties, dynamic topologies, and cascading failure modes characteristic of modern interdependent networks. The grand challenge lies not merely in selecting one approach over the other, but in developing frameworks that coordinate graceful degradation and automated recovery to ensure continuous operation under adverse conditions [448].

Resilience-by-design focuses on embedding fault tolerance and robustness directly into the system architecture before deployment, creating the structural boundaries within which autonomous agents must operate. This involves constructing network topologies and control algorithms that are intrinsically resistant to disruptions. A prime example of this philosophy is the design of “onion-like” networks that self-organize during growth to enhance interwoven long loops, thereby providing

strong robustness against targeted attacks without sacrificing path efficiency [449]. Similarly, in the domain of interdependent infrastructure, graph learning-based generative design methods are emerging as powerful tools. These methods utilize performance estimators and candidate design generators to intelligently mine properties from existing systems, outputting new designs that meet predefined resilience criteria against both internal uncertainties and external negative impacts [450]. Furthermore, static design principles can be enhanced by optimizing network parameters to prevent contagion; for instance, message-passing algorithms applied to factor graph representations of interdependent systems can derive sufficient conditions to guarantee complete healing against cascading failures before they occur [451]. By establishing these structural guarantees, resilience-by-design minimizes the likelihood of catastrophic breakdowns, creating a stable foundation upon which the reactive mechanisms of autonomous agents can safely operate.

Conversely, resilience-by-reaction emphasizes the system’s ability to detect, diagnose, and recover from faults dynamically at runtime, extending the capabilities of the scientific debugging agents described previously to the physical realm. This paradigm is essential for addressing unanticipated failures and environmental drifts that cannot be fully modeled during the design phase. Effective reactive systems often rely on adaptive knowledge bases that exploit implicit redundancy in space and time to generate runtime monitors capable of comparing uncertain and asynchronous measurements [401]. Once a fault is detected, the system must execute recovery strategies that may involve reconfiguration or component replacement. For complex reactive systems, reinforcement learning techniques have been successfully employed to learn recovery strategies based on users’ corrective sequences, allowing the system to overwrite base behaviors with learned recovery policies dynamically [402]. In distributed environments, this reactive capability often manifests as self-stabilization, where algorithms guarantee automatic recovery within a constant number of communication rounds even after arbitrary state corruptions, as demonstrated in fog computing ecosystems [427].

The true frontier of research, however, lies in the synergistic coordination of these two paradigms, bridging the gap between the static guarantees of design and the dynamic adaptability of agent-driven reaction. A system that is purely designed for resilience may lack the flexibility to handle novel attack vectors, while a purely reactive system may suffer from prolonged instability during the recovery phase. The integration of graceful degradation and recovery represents a critical mechanism for balancing this trade-off. Graceful degradation ensures that the system maintains an acceptable level of safety by temporarily weakening original requirements when facing unexpected conditions, while recovery strengthens these requirements once the environment stabilizes. By treating these as dual, requirement-driven adaptation tasks, a single adaptation method can coordinate both operations, as seen in frameworks for unmanned underwater vehicles that utilize Signal Temporal Logic (STL) to manage transitions between degradation and recovery states [240].

Moreover, the interplay between design and reaction is vital for interdependent networks where failures can propagate rapidly across coupled layers, requiring agents to understand both the topological constraints and the immediate repair actions. Dynamic extension models of interdependent networks introduce the concept of “healing probability,” where link formation occurs dynamically after node removals to suppress cascading failures. Simulation results indicate that above a critical healing probability, macroscopic cascades are prevented, highlighting how reactive healing efforts must be calibrated against the structural vulnerabilities defined by the network design [452]. In resource-constrained scenarios, such as post-disaster infrastructure recovery, deep learning models can approximate optimal restoration sequences derived from high-fidelity simulators, effectively bridging the gap between design-time optimization and runtime decision-making [453].

Future autonomous agents must therefore possess the cognitive ability to assess whether a disruption requires a design-level reconfiguration or a reaction-level patch, effectively acting as the orchestrator between these two layers. This involves moving beyond static rule-based adaptations toward utility-driven schemes that compute the impact of adaptation rules on overall system utility in real-time, ensuring optimal decisions without the computational cost of exhaustive optimization [454]. Additionally, the evaluation of such hybrid systems requires rigorous methodologies. Chaos engineering, which subjects systems to unexpected conditions, offers a promising avenue for systematically assessing how well self-healing systems coordinate their design-based robustness with reactive capabilities under perturbation [407]. Ultimately, the realization of truly resilient CPS depends on closing the loop between design and reaction, creating ecosystems where the architecture evolves based on runtime experiences, and reactive measures are guided by deep structural insights, ensuring sustained operation in an increasingly volatile world.

References

- [1] TBar Revisiting Template-based Automated Program Repair
- [2] Enhancing Redundancy-based Automated Program Repair by Fine-grained Pattern Mining
- [3] SequenceR Sequence-to-Sequence Learning for End-to-End Program Repair
- [4] Neural Program Repair Systems, Challenges and Solutions
- [5] Learning to Generate Corrective Patches using Neural Machine Translation
- [6] CURE Code-Aware Neural Machine Translation for Automatic Program Repair
- [7] Less Training, More Repairing Please Revisiting Automated Program Repair via Zero-shot Learning
- [8] Enhancing Automated Program Repair through Fine-tuning and Prompt Engineering
- [9] Practical Program Repair in the Era of Large Pre-trained Language Models
- [10] GPT-3.5 for Code Review Automation How Do Few-Shot Learning, Prompt Design, and Model Fine-Tuning Impact Their Performance
- [11] Few-shot training LLMs for project-specific code-summarization
- [12] CodeT5+ Open Code Large Language Models for Code Understanding and Generation
- [13] Finetuned Language Models Are Zero-Shot Learners
- [14] A Critical Review of Large Language Model on Software Engineering An Example from Chat-GPT and Automated Program Repair
- [15] Aligning LLMs for FL-free Program Repair
- [16] Revisiting the Plastic Surgery Hypothesis via Large Language Models
- [17] Multi-Objective Fine-Tuning for Enhanced Program Repair with LLMs
- [18] Out of Context How important is Local Context in Neural Program Repair
- [19] LMs Understanding Code Syntax and Semantics for Code Analysis
- [20] Code Representation Pre-training with Complements from Program Executions

- [21] A Deep Dive into Large Language Models for Automated Bug Localization and Repair
- [22] Detect-Localize-Repair A Unified Framework for Learning to Debug with CodeT5
- [23] How Far Can We Go with Practical Function-Level Program Repair
- [24] When Large Language Models Confront Repository-Level Automatic Program Repair How Well They Done
- [25] A Unified Debugging Approach via LLM-Based Multi-Agent Synergy
- [26] Keep the Conversation Going Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT
- [27] RESTORE Retrospective Fault Localization Enhancing Automated Program Repair
- [28] Can Automated Program Repair Refine Fault Localization
- [29] Large Language Models in Fault Localisation
- [30] Large Language Models are Zero-Shot Reasoners
- [31] The Impact of Reasoning Step Length on Large Language Models
- [32] Automatic Chain of Thought Prompting in Large Language Models
- [33] Boosting of Thoughts Trial-and-Error Problem Solving with Large Language Models
- [34] Graph-Guided Reasoning for Multi-Hop Question Answering in Large Language Models
- [35] Evidence to Generate (E2G) A Single-agent Two-step Prompting for Context Grounded and Retrieval Augmented Reasoning
- [36] ChainLM Empowering Large Language Models with Improved Chain-of-Thought Prompting
- [37] The Art of SOCRATIC QUESTIONING Recursive Thinking with Large Language Models
- [38] Resprompt Residual Connection Prompting Advances Multi-Step Reasoning in Large Language Models
- [39] R³ Prompting Review, Rephrase and Resolve for Chain-of-Thought Reasoning in Large Language Models under Noisy Context
- [40] LoRA Low-Rank Adaptation of Large Language Models
- [41] Sparse Low-rank Adaptation of Pre-trained Language Models
- [42] ALoRA Allocating Low-Rank Adaptation for Fine-tuning Large Language Models
- [43] BiLoRA A Bi-level Optimization Framework for Overfitting-Resilient Low-Rank Adaptation of Large Pre-trained Models
- [44] ResLoRA Identity Residual Mapping in Low-Rank Adaption
- [45] Delving into Parameter-Efficient Fine-Tuning in Code Change Learning An Empirical Study
- [46] P-Tuning v2 Prompt Tuning Can Be Comparable to Fine-tuning Universally Across Scales and Tasks
- [47] DePT Decomposed Prompt Tuning for Parameter-Efficient Fine-tuning
- [48] Decomposed Prompt Tuning via Low-Rank Reparameterization

- [49] SMART Submodular Data Mixture Strategy for Instruction Tuning
- [50] LoraHub Efficient Cross-Task Generalization via Dynamic LoRA Composition
- [51] Effectiveness of Data Augmentation for Parameter Efficient Tuning with Limited Data
- [52] MoELoRA Contrastive Learning Guided Mixture of Experts on Parameter-Efficient Fine-Tuning for Large Language Models
- [53] RAP-Gen Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair
- [54] How faithful are RAG models Quantifying the tug-of-war between RAG and LLMs’ internal prior
- [55] Unsupervised Information Refinement Training of Large Language Models for Retrieval-Augmented Generation
- [56] ARKS Active Retrieval in Knowledge Soup for Code Generation
- [57] RAGGED Towards Informed Design of Retrieval Augmented Generation Systems
- [58] DeepCode AI Fix Fixing Security Vulnerabilities with Large Language Models
- [59] Blended RAG Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers
- [60] An Empirical Study of Automated Vulnerability Localization with Large Language Models
- [61] One Adapter for All Programming Languages Adapter Tuning for Code Search and Summarization
- [62] AgentFL Scaling LLM-based Fault Localization to Project-Level Context
- [63] Enhancing Q&A with Domain-Specific Fine-Tuning and Iterative Reasoning A Comparative Study
- [64] GloBug Using Global Data in Fault Localization
- [65] Fast Changeset-based Bug Localization with BERT
- [66] Learning in the Wild Towards Leveraging Unlabeled Data for Effectively Tuning Pre-trained Code Models
- [67] BAFLineDP Code Bilinear Attention Fusion Framework for Line-Level Defect Prediction
- [68] FLACK Counterexample-Guided Fault Localization for Alloy Models
- [69] CausalRCA Causal Inference based Precise Fine-grained Root Cause Localization for Microservice Applications
- [70] MetaGPT Meta Programming for A Multi-Agent Collaborative Framework
- [71] MobileAgent enhancing mobile control via human-machine interaction and SOP integration
- [72] Unleashing the Emergent Cognitive Synergy in Large Language Models A Task-Solving Agent through Multi-Persona Self-Collaboration
- [73] Toward a Formal Model of Cognitive Synergy
- [74] ALMA Hierarchical Learning for Composite Multi-Agent Tasks

- [75] Resource-rational Task Decomposition to Minimize Planning Costs
- [76] Self-Organized Agents A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization
- [77] Investigate-Consolidate-Exploit A General Strategy for Inter-Task Agent Self-Evolution
- [78] Autonomous and Adaptive Role Selection for Multi-robot Collaborative Area Search Based on Deep Reinforcement Learning
- [79] S-Agents Self-organizing Agents in Open-ended Environments
- [80] Balancing Autonomy and Alignment A Multi-Dimensional Taxonomy for Autonomous LLM-powered Multi-Agent Architectures
- [81] Executable Code Actions Elicit Better LLM Agents
- [82] TaskWeaver A Code-First Agent Framework
- [83] InterCode Standardizing and Benchmarking Interactive Coding with Execution Feedback
- [84] LDB A Large Language Model Debugger via Verifying Runtime Execution Step-by-step
- [85] Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation
- [86] Guiding Language Models of Code with Global Context using Monitors
- [87] ToolChain Efficient Action Space Navigation in Large Language Models with A Search
- [88] From Summary to Action Enhancing Large Language Models for Complex Tasks with Open World APIs
- [89] GoEX Perspectives and Designs Towards a Runtime for Autonomous LLM Applications
- [90] InjecAgent Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents
- [91] Large Language Models as Tool Makers
- [92] From Language Models to Practical Self-Improving Computer Agents
- [93] NExT Teaching Large Language Models to Reason about Code Execution
- [94] TraceFixer Execution Trace-Driven Program Repair
- [95] Dynamic Analysis can be Improved with Automatic Test Suite Refactoring
- [96] Self-Boosted Automated Program Repair
- [97] Search-based Automated Program Repair of CPS Controllers Modeled in Simulink-Stateflow
- [98] Efficient Interaction-Based Offline Runtime Verification of Distributed Systems with Lifeline Removal
- [99] Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate
- [100] Should we be going MAD A Look at Multi-Agent Debate Strategies for LLMs
- [101] Apollo’s Oracle Retrieval-Augmented Reasoning in Multi-Agent Debates
- [102] Towards Reasoning in Large Language Models via Multi-Agent Peer Review Collaboration

- [103] ReConcile Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs
- [104] Robot Duck Debugging Can Attentive Listening Improve Problem Solving
- [105] A Brain-inspired Theory of Collective Mind Model for Efficient Social Cooperation
- [106] Interaction Pattern Disentangling for Multi-Agent Reinforcement Learning
- [107] MAGDi Structured Distillation of Multi-Agent Interaction Graphs Improves Reasoning in Smaller Language Models
- [108] Combating Adversarial Attacks with Multi-Agent Debate
- [109] Exploring Interaction Patterns for Debugging Enhancing Conversational Capabilities of AI-assistants
- [110] Investigating Agency of LLMs in Human-AI Collaboration Tasks
- [111] The Programmer’s Assistant Conversational Interaction with a Large Language Model for Software Development
- [112] Speak Out of Turn Safety Vulnerability of Large Language Models in Multi-turn Dialogue
- [113] Tuning-Free Accountable Intervention for LLM Deployment – A Metacognitive Approach
- [114] CloChat Understanding How People Customize, Interact, and Experience Personas in Large Language Models
- [115] A Taxonomy for Human-LLM Interaction Modes An Initial Exploration
- [116] GrounDial Human-norm Grounded Safe Dialog Response Generation
- [117] Prompting and Evaluating Large Language Models for Proactive Dialogues Clarification, Target-guided, and Non-collaboration
- [118] Self-organizing Nervous Systems for Robot Swarms
- [119] A Formal Framework for Reasoning about Agents’ Independence in Self-organizing Multi-agent Systems
- [120] Inter-Level Cooperation in Hierarchical Reinforcement Learning
- [121] Initial Task Assignment in Multi-Human Multi-Robot Teams An Attention-enhanced Hierarchical Reinforcement Learning Approach
- [122] Agents meet OKR An Object and Key Results Driven Agent System with Hierarchical Self-Collaboration and Self-Evaluation
- [123] Combinatorial-hybrid Optimization for Multi-agent Systems under Collaborative Tasks
- [124] D-ITAGS A Dynamic Interleaved Approach to Resilient Task Allocation, Scheduling, and Motion Planning
- [125] LDSA Learning Dynamic Subtask Assignment in Cooperative Multi-Agent Reinforcement Learning
- [126] GCS Graph-based Coordination Strategy for Multi-Agent Reinforcement Learning
- [127] Resilient Continuum Deformation Coordination

- [128] A Structured Prediction Approach for Generalization in Cooperative Multi-Agent Reinforcement Learning
- [129] AgentQuest A Modular Benchmark Framework to Measure Progress and Improve LLM Agents
- [130] WebArena A Realistic Web Environment for Building Autonomous Agents
- [131] OSWorld Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments
- [132] LLMARena Assessing Capabilities of Large Language Models in Dynamic Multi-Agent Environments
- [133] The Harmony Index a Utilitarian Metric for Measuring Effectiveness in Mixed-Skill Teams
- [134] AgentBoard An Analytical Evaluation Board of Multi-turn LLM Agents
- [135] Dynamic Competency Self-Assessment for Autonomous Agents
- [136] Mapping global dynamics of benchmark creation and saturation in artificial intelligence
- [137] Reasoning Capacity in Multi-Agent Systems Limitations, Challenges and Human-Centered Solutions
- [138] MAgIC Investigation of Large Language Model Powered Multi-Agent in Cognition, Adaptability, Rationality and Collaboration
- [139] Logic-LM Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning
- [140] Neuro Symbolic Reasoning for Planning Counterexample Guided Inductive Synthesis using Large Language Models and Satisfiability Solving
- [141] Neuro-Symbolic Integration Brings Causal and Reliable Reasoning Proofs
- [142] Lyra Orchestrating Dual Correction in Automated Theorem Proving
- [143] Enhancing Ethical Explanations of Large Language Models through Iterative Symbolic Refinement
- [144] ToM-LM Delegating Theory of Mind Reasoning to External Symbolic Executors in Large Language Models
- [145] Frugal LMs Trained to Invoke Symbolic Solvers Achieve Parameter-Efficient Arithmetic Reasoning
- [146] From Indeterminacy to Determinacy Augmenting Logical Reasoning Capabilities with Large Language Models
- [147] Can A Cognitive Architecture Fundamentally Enhance LLMs Or Vice Versa
- [148] Large Language Models Cannot Self-Correct Reasoning Yet
- [149] SelfCheck Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning
- [150] Deductive Verification of Chain-of-Thought Reasoning
- [151] Confidence Matters Revisiting Intrinsic Self-Correction Capabilities of Large Language Models
- [152] Fact-and-Reflection (FaR) Improves Confidence Calibration of Large Language Models

- [153] Self-Contrast Better Reflection Through Inconsistent Solving Perspectives
- [154] N-Critics Self-Refinement of Large Language Models with Ensemble of Critics
- [155] LLMs cannot find reasoning errors, but can correct them!
- [156] Can Large Language Models Really Improve by Self-critiquing Their Own Plans
- [157] On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks
- [158] Learning to Check Unleashing Potentials for Self-Correction in Large Language Models
- [159] Small Language Models Need Strong Verifiers to Self-Correct Reasoning
- [160] The Best of Both Worlds Combining Learned Embeddings with Engineered Features for Accurate Prediction of Correct Patches
- [161] Evaluating Representation Learning of Code Changes for Predicting Patch Correctness in Program Repair
- [162] Revisiting Unnaturalness for Automated Program Repair in the Era of Large Language Models
- [163] On the Naturalness of Buggy Code
- [164] Evaluating Program Repair with Semantic-Preserving Transformations A Naturalness Assessment
- [165] Is this Change the Answer to that Problem Correlating Descriptions of Bug and Code Changes for Evaluating Patch Correctness
- [166] Predicting Patch Correctness Based on the Similarity of Failing Test Cases
- [167] ObjSim Lightweight Automatic Patch Prioritization via Object Similarity
- [168] Invalidator Automated Patch Correctness Assessment via Semantic and Syntactic Reasoning
- [169] PatchZero Zero-Shot Automatic Patch Correctness Assessment
- [170] CRITIC Large Language Models Can Self-Correct with Tool-Interactive Critiquing
- [171] Frustrated with Code Quality Issues LLMs can Help!
- [172] E&V Prompting Large Language Models to Perform Static Analysis by Pseudo-code Execution and Verification
- [173] INTERVENOR Prompting the Coding Ability of Large Language Models with the Interactive Chain of Repair
- [174] Copiloting the Copilots Fusing Large Language Models with Completion Engines for Automated Program Repair
- [175] Reflexion Language Agents with Verbal Reinforcement Learning
- [176] CYCLE Learning to Self-Refine the Code Generation
- [177] Learning to Use Tools via Cooperative and Interactive Agents
- [178] Automatic Calibration and Error Correction for Generative Large Language Models via Pareto Optimal Self-Supervision

- [179] Alleviating Patch Overfitting with Automatic Test Generation A Study of Feasibility and Effectiveness for the Nopol Repair System
- [180] Test Case Generation for Program Repair A Study of Feasibility and Effectiveness
- [181] LLM-Powered Test Case Generation for Detecting Tricky Bugs
- [182] Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis
- [183] Improving The Effectiveness of Automatically Generated Test Suites Using Metamorphic Testing
- [184] TOGA A Neural Method for Test Oracle Generation
- [185] Neural-Based Test Oracle Generation A Large-scale Evaluation and Lessons Learned
- [186] Revisiting and Improving Retrieval-Augmented Deep Assertion Generation
- [187] SAGA Summarization-Guided Assert Statement Generation
- [188] Patch Space Exploration using Static Analysis Feedback
- [189] Towards More Reliable Automated Program Repair by Integrating Static Analysis Techniques
- [190] Functionality learning through specification instructions
- [191] Tailored Mutants Fit Bugs Better
- [192] Perils of Self-Feedback Self-Bias Amplifies in Large Language Models
- [193] Faithfulness vs. Plausibility On the (Un)Reliability of Explanations from Large Language Models
- [194] When Hindsight is Not 20 20 Testing Limits on Reflective Thinking in Large Language Models
- [195] V-STaR Training Verifiers for Self-Taught Reasoners
- [196] LLM4SecHW Leveraging Domain Specific Large Language Model for Hardware Debugging
- [197] Fixing Hardware Security Bugs with Large Language Models
- [198] AutoChip Automating HDL Generation Using LLM Feedback
- [199] RTLFixer Automatically Fixing RTL Syntax Errors with Large Language Models
- [200] All Artificial, Less Intelligence GenAI through the Lens of Formal Verification
- [201] DIVAS An LLM-based End-to-End Framework for SoC Security Analysis and Policy-based Protection
- [202] LLM-assisted Generation of Hardware Assertions
- [203] A Pragmatic Methodology for Blind Hardware Trojan Insertion in Finalized Layouts
- [204] ZeroLeak Using LLMs for Scalable and Cost Effective Side-Channel Patching
- [205] CreativEval Evaluating Creativity of LLM-Based Hardware Code Generation
- [206] LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet) A Comprehensive Evaluation, Framework, and Benchmarks
- [207] ChipGPT How far are we from natural language hardware design

- [208] Vulnerability Scanners for Ethereum Smart Contracts A Large-Scale Study
- [209] Empirical Vulnerability Analysis of Automated Smart Contracts Security Testing on Blockchains
- [210] Evaluation of ChatGPT’s Smart Contract Auditing Capabilities Based on Chain of Thought
- [211] Combining Fine-Tuning and LLM-based Agents for Intuitive Smart Contract Auditing with Justifications
- [212] LLM4Fuzz Guided Fuzzing of Smart Contracts with Large Language Models
- [213] EVMPatch Timely and Automated Patching of Ethereum Smart Contracts
- [214] Elysium Context-Aware Bytecode-Level Patching to Automatically Heal Vulnerable Smart Contracts
- [215] Two Timin’ Repairing Smart Contracts With A Two-Layered Approach
- [216] Checking Smart Contracts with Structural Code Embedding
- [217] ÆGIS Shielding Vulnerable Smart Contracts Against Attacks
- [218] Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility
- [219] Faster Post-Quantum TLS 1.3 Based on ML-KEM Implementation and Assessment
- [220] FIPS Compliant Quantum Secure Communication using Quantum Permutation Pad
- [221] Near-Term Quantum Computing Techniques Variational Quantum Algorithms, Error Mitigation, Circuit Compilation, Benchmarking and Classical Simulation
- [222] Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers
- [223] Optimizing Quantum Programs against Decoherence Delaying Qubits into Quantum Superposition
- [224] Experimental quantum adversarial learning with programmable superconducting qubits
- [225] CutQC Using Small Quantum Computers for Large Quantum Circuit Evaluations
- [226] Explaining Quantum Circuits with Shapley Values Towards Explainable Quantum Machine Learning
- [227] Rosita++ Automatic Higher-Order Leakage Elimination from Cryptographic Code
- [228] Rosita Towards Automatic Elimination of Power-Analysis Leakage in Ciphers
- [229] SCAR Power Side-Channel Analysis at RTL-Level
- [230] ACFIX Guiding LLMs with Mined Common RBAC Practices for Context-Aware Repair of Access Control Vulnerabilities in Smart Contracts
- [231] Smart Contract and DeFi Security Tools Do They Meet the Needs of Practitioners
- [232] Toward Deep Learning Based Access Control
- [233] Deep Learning meets Blockchain for Automated and Secure Access Control
- [234] Cyber-Physical Defense in the Quantum Era

- [235] On DoS Resiliency Analysis of Networked Control Systems Trade-Off Between Jamming Actions and Network Delays
- [236] A Colonel Blotto Game for Interdependence-Aware Cyber-Physical Systems Security in Smart Cities
- [237] A Survey of Moving Target Defenses for Network Security
- [238] Autonomous Cyber Defense Introduces Risk Can We Manage the Risk
- [239] Barrier Certificate based Safe Control for LiDAR-based Systems under Sensor Faults and Attacks
- [240] Integrating Graceful Degradation and Recovery through Requirement-driven Adaptation
- [241] AutoAttacker A Large Language Model Guided System to Implement Automatic Cyber-attacks
- [242] LLM Agents can Autonomously Exploit One-day Vulnerabilities
- [243] ICSSIM-A Framework for Building Industrial Control Systems Security Simulation Testbeds
- [244] From Prompt Injections to SQL Injection Attacks How Protected is Your LLM-Integrated Web Application
- [245] StruQ Defending Against Prompt Injection with Structured Queries
- [246] Defending Against Indirect Prompt Injection Attacks With Spotlighting
- [247] Follow My Instruction and Spill the Beans Scalable Data Extraction from Retrieval-Augmented Generation Systems
- [248] Understanding Privacy Risks of Embeddings Induced by Large Language Models
- [249] A New Era in LLM Security Exploring Security Concerns in Real-World LLM-based Systems
- [250] Identifying and Mitigating Vulnerabilities in LLM-Integrated Applications
- [251] Can LLMs substitute SQL Comparing Resource Utilization of Querying LLMs versus Traditional Relational Databases
- [252] Optimizing LLM Queries in Relational Workloads
- [253] PURPLE Making a Large Language Model a Better SQL Writer
- [254] DB-GPT Empowering Database Interactions with Private Large Language Models
- [255] LlamaTune Sample-Efficient DBMS Configuration Tuning
- [256] LLM-Enhanced Data Management
- [257] Enhancing Reasoning Capacity of SLM using Cognitive Enhancement
- [258] PoisonedRAG Knowledge Poisoning Attacks to Retrieval-Augmented Generation of Large Language Models
- [259] Not what you've signed up for Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection
- [260] Prompt Injection attack against LLM-integrated Applications

- [261] Human-Imperceptible Retrieval Poisoning Attacks in LLM-Powered Applications
- [262] Learning to Poison Large Language Models During Instruction Tuning
- [263] Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents
- [264] The Ethics of Interaction Mitigating Security Threats in LLMs
- [265] The Instruction Hierarchy Training LLMs to Prioritize Privileged Instructions
- [266] Mapping LLM Security Landscapes A Comprehensive Stakeholder Risk Assessment Proposal
- [267] Automatic Repair of Real Bugs in Java A Large-Scale Experiment on the Defects4J Dataset
- [268] Dissection of a Bug Dataset Anatomy of 395 Patches from Defects4J
- [269] Back to the Future! Studying Data Cleanness in Defects4J and its Impact on Fault Localization
- [270] Searching for Multi-Fault Programs in Defects4J
- [271] An Analysis of the Automatic Bug Fixing Performance of ChatGPT
- [272] A Comprehensive Study of Automatic Program Repair on the QuixBugs Benchmark
- [273] GitBug-Java A Reproducible Benchmark of Recent Java Bugs
- [274] GitBug-Actions Building Reproducible Bug-Fix Benchmarks with GitHub Actions
- [275] Bears An Extensible Java Bug Benchmark for Automatic Program Repair Studies
- [276] BUMP A Benchmark of Reproducible Breaking Dependency Updates
- [277] Tests4Py A Benchmark for System Testing
- [278] Bugs4Q A Benchmark of Real Bugs for Quantum Programs
- [279] Bugs in Machine Learning-based Systems A Faultload Benchmark
- [280] APPT Boosting Automated Patch Correctness Prediction via Fine-tuning Pre-trained Models
- [281] How Different Is It Between Machine-Generated and Developer-Provided Patches An Empirical Study on The Correct Patches Generated by Automated Program Repair Techniques
- [282] Test suite effectiveness metric evaluation what do we know and what should we do
- [283] Mind the Gap The Difference Between Coverage and Mutation Score Can Guide Testing Efforts
- [284] Patch Quality and Diversity of Invariant-Guided Search-Based Program Repair
- [285] Investigating Data Contamination in Modern Benchmarks for Large Language Models
- [286] Leak, Cheat, Repeat Data Contamination and Evaluation Malpractices in Closed-Source LLMs
- [287] Data Contamination Through the Lens of Time
- [288] NLP Evaluation in trouble On the Need to Measure LLM Data Contamination for each Benchmark
- [289] Detecting Pretraining Data from Large Language Models

- [290] Evading Data Contamination Detection for Language Models is (too) Easy
- [291] Private Benchmarking to Prevent Contamination and Improve Comparative Evaluation of LLMs
- [292] Have Seen Me Before Automating Dataset Updates Towards Reliable and Timely Evaluation
- [293] You Cannot Fix What You Cannot Find! An Investigation of Fault Localization Bias in Benchmarking Automated Program Repair Systems
- [294] Better Automatic Program Repair by Using Bug Reports and Tests Together
- [295] Can defects be fixed with weak test suites An analysis of 50 defects from Defects4J
- [296] Detection of Coincidentally Correct Test Cases through Random Forests
- [297] FlakiMe Laboratory-Controlled Test Flakiness Impact Assessment. A Case Study on Mutation Testing and Program Repair
- [298] FDG A Precise Measurement of Fault Diagnosability Gain of Test Cases
- [299] Robustness, Evaluation and Adaptation of Machine Learning Models in the Wild
- [300] Metrics reloaded Recommendations for image analysis validation
- [301] The Integration of Machine Learning into Automated Test Generation A Systematic Mapping Study
- [302] When Code Smells Meet ML On the Lifecycle of ML-specific Code Smells in ML-enabled Systems
- [303] A Historical and Statistical Study of the Software Vulnerability Landscape
- [304] Security Theater On the Vulnerability of Classifiers to Exploratory Attacks
- [305] Cross Project Software Vulnerability Detection via Domain Adaptation and Max-Margin Principle
- [306] DRSM De-Randomized Smoothing on Malware Classifier Providing Certified Robustness
- [307] Unlocking the Secrets of Software Configuration Landscapes-Ruggedness, Accessibility, Escapability, and Transferability
- [308] AIBench An Agile Domain-specific Benchmarking Methodology and an AI Benchmark Suite
- [309] AIBench Scenario Scenario-distilling AI Benchmarking
- [310] Towards a multi-stakeholder value-based assessment framework for algorithmic systems
- [311] Hybrid Blockchain-Enabled Secure Microservices Fabric for Decentralized Multi-Domain Avionics Systems
- [312] Identifying the Context Shift between Test Benchmarks and Production Data
- [313] Challenges and Pitfalls of Machine Learning Evaluation and Benchmarking
- [314] Evaluation-as-a-Service Overview and Outlook
- [315] Empirical Design in Reinforcement Learning
- [316] Exploration of reproducibility issues in scientometric research Part 1 Direct reproducibility

- [317] Exploration of Reproducibility Issues in Scientometric Research Part 2 Conceptual Reproducibility
- [318] Lazy Data Practices Harm Fairness Research
- [319] Collect, Measure, Repeat Reliability Factors for Responsible AI Data Collection
- [320] Hazards in Deep Learning Testing Prevalence, Impact and Recommendations
- [321] Expected Validation Performance and Estimation of a Random Variable's Maximum
- [322] Medium-n studies in computing education conferences
- [323] Robust Hypothesis Testing Using Wasserstein Uncertainty Sets
- [324] Empirical Standards for Software Engineering Research
- [325] Evidence Tetris in the Pixelated World of Validity Threats
- [326] Balancing Covariates in Randomized Experiments with the Gram-Schmidt Walk Design
- [327] Researcher Bias in Software Engineering Experiments a Qualitative Investigation
- [328] Replication studies considered harmful
- [329] Exploring and Evaluating Hallucinations in LLM-Powered Code Generation
- [330] Can ChatGPT replace StackOverflow A Study on Robustness and Reliability of Large Language Model Code Generation
- [331] How Language Model Hallucinations Can Snowball
- [332] LLM Lies Hallucinations are not Bugs, but Features as Adversarial Examples
- [333] Large Language Models of Code Fail at Completing Code with Potential Bugs
- [334] Hallucinations in Neural Automatic Speech Recognition Identifying Errors and Hallucinatory Models
- [335] RAGged Edges The Double-Edged Sword of Retrieval-Augmented Chatbots
- [336] Deficiency of Large Language Models in Finance An Empirical Examination of Hallucination
- [337] A Survey on Hallucination in Large Language Models Principles, Taxonomy, Challenges, and Open Questions
- [338] DeepInception Hypnotize Large Language Model to Be Jailbreaker
- [339] Making Them Ask and Answer Jailbreaking Large Language Models in Few Queries via Disguise and Reconstruction
- [340] The Wolf Within Covert Injection of Malice into MLLM Societies via an MLLM Operative
- [341] Sleeper Agents Training Deceptive LLMs that Persist Through Safety Training
- [342] CodeChameleon Personalized Encryption Framework for Jailbreaking Large Language Models
- [343] Here Comes The AI Worm Unleashing Zero-click Worms that Target GenAI-Powered Applications
- [344] Analyzing Leakage of Personally Identifiable Information in Language Models

- [345] What can we learn from Data Leakage and Unlearning for Law
- [346] The Janus Interface How Fine-Tuning in Large Language Models Amplifies the Privacy Risks
- [347] Privacy Backdoors Enhancing Membership Inference through Poisoning Pre-trained Models
- [348] PreCurious How Innocent Pre-Trained Language Models Turn into Privacy Traps
- [349] Gotcha! This Model Uses My Code! Evaluating Membership Leakage Risks in Code Models
- [350] Vulnerabilities in AI Code Generators Exploring Targeted Data Poisoning Attacks
- [351] Reconciling AI Performance and Data Reconstruction Resilience for Medical Imaging
- [352] Behind the Mask Demographic bias in name detection for PII masking
- [353] Teach LLMs to Phish Stealing Private Information from Language Models
- [354] Individual Explanations in Machine Learning Models A Survey for Practitioners
- [355] Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead
- [356] Towards Explainable Artificial Intelligence
- [357] Hiding Backdoors within Event Sequence Data via Poisoning Attacks
- [358] Interpretation of Neural Networks is Fragile
- [359] Influence Tuning Demoting Spurious Correlations via Instance Attribution and Instance-Driven Updates
- [360] A Survey on Explainable Artificial Intelligence (XAI) Towards Medical XAI
- [361] Sequential Interpretability Methods, Applications, and Future Direction for Understanding Deep Learning Models in the Context of Sequential Data
- [362] A Survey of Black-Box Adversarial Attacks on Computer Vision Models
- [363] Fairwashing the risk of rationalization
- [364] Infusing domain knowledge in AI-based black box models for better explainability with application in bankruptcy prediction
- [365] Open the Black Box Data-Driven Explanation of Black Box Decision Systems
- [366] Dirichlet uncertainty wrappers for actionable algorithm accuracy accountability and auditability
- [367] Trustworthy Large Models in Vision A Survey
- [368] Walking a Tightrope – Evaluating Large Language Models in High-Risk Domains
- [369] What’s my role Modelling responsibility for AI-based safety-critical systems
- [370] The Conflict Between People’s Urge to Punish AI and Legal Systems
- [371] The Guilty (Silicon) Mind Blameworthiness and Liability in Human-Machine Teaming
- [372] Playing the Blame Game with Robots
- [373] The Responsibility Quantification (ResQu) Model of Human Interaction with Automation

- [374] Commercial AI, Conflict, and Moral Responsibility A theoretical analysis and practical approach to the moral responsibilities associated with dual-use AI technology
- [375] Protecting Society from AI Misuse When are Restrictions on Capabilities Warranted
- [376] An Overview of Catastrophic AI Risks
- [377] REGARD Rules of EngaGement for Automated cybeR Defense to aid in Intrusion Response
- [378] The Promise and Peril of Artificial Intelligence – Violet Teaming Offers a Balanced Path Forward
- [379] Increased Compute Efficiency and the Diffusion of AI Capabilities
- [380] Adversarial Targeted Forgetting in Regularization and Generative Based Continual Learning Models
- [381] Towards a Scalable Proof Engine A Performant Prototype Rewriting Primitive for Coq
- [382] GOAT GPU Outsourcing of Deep Learning Training With Asynchronous Probabilistic Integrity Verification Inside Trusted Execution Environment
- [383] Robust Machine Learning Systems Challenges, Current Trends, Perspectives, and the Road Ahead
- [384] Data-Oblivious ML Accelerators using Hardware Security Extensions
- [385] Is Secure Coding Education in the Industry Needed An Investigation Through a Large Scale Survey
- [386] Secure Software Engineering in the Financial Services A Practitioners’ Perspective
- [387] FusionAI Decentralized Training and Deploying LLMs with Massive Consumer-Level GPUs
- [388] Actionable Guidance for High-Consequence AI Risk Management Towards Standards Addressing AI Catastrophic Risks
- [389] Towards a Roadmap on Software Engineering for Responsible AI
- [390] MART Improving LLM Safety with Multi-round Automatic Red-Teaming
- [391] TanksWorld A Multi-Agent Environment for AI Safety Research
- [392] AI Certification Advancing Ethical Practice by Reducing Information Asymmetries
- [393] AI Risk Profiles A Standards Proposal for Pre-Deployment AI Risk Disclosures
- [394] Concept Dynamic Risk Assessment for AI-Controlled Robotic Systems
- [395] A Safe Harbor for AI Evaluation and Red Teaming
- [396] CTI4AI Threat Intelligence Generation and Sharing after Red Teaming AI Models
- [397] Adaptive Immunity for Software Towards Autonomous Self-healing Systems
- [398] Artificial Immune Systems (INTROS 2)
- [399] Implementing Immune Repertoire Models Using Weighted Finite State Machines
- [400] Self-Healing Robust Neural Networks via Closed-Loop Control

- [401] Adaptive Fault Detection exploiting Redundancy with Uncertainties in Space and Time
- [402] Learning Recovery Strategies for Dynamic Self-healing in Reactive Systems
- [403] Robots that can adapt like animals
- [404] Data-Driven Machine Learning Techniques for Self-healing in Cellular Wireless Networks Challenges and Solutions
- [405] AI-based Self-healing Solutions Applied to Cellular Networks An Overview
- [406] An Intentional Forgetting-Driven Self-Healing Method For Deep Reinforcement Learning Systems
- [407] On Evaluating Self-Adaptive and Self-Healing Systems using Chaos Engineering
- [408] CHESS A Framework for Evaluation of Self-adaptive Systems based on Chaos Engineering
- [409] Immuno-inspired robotic applications a review
- [410] Challenging Common Assumptions about Catastrophic Forgetting
- [411] Out-of-distribution forgetting vulnerability of continual learning to intra-class distribution shift
- [412] AFEC Active Forgetting of Negative Transfer in Continual Learning
- [413] Sub-network Discovery and Soft-masking for Continual Learning of Mixed Tasks
- [414] P2DT Mitigating Forgetting in task-incremental Learning with progressive prompt Decision Transformer
- [415] Prototype-Sample Relation Distillation Towards Replay-Free Continual Learning
- [416] Saliency-Guided Hidden Associative Replay for Continual Learning
- [417] TRGP Trust Region Gradient Projection for Continual Learning
- [418] Continual learning benefits from multiple sleep mechanisms NREM, REM, and Synaptic Downscaling
- [419] Analysis of the Memorization and Generalization Capabilities of AI Agents Are Continual Learners Robust
- [420] The Case for Hierarchical Deep Learning Inference at the Network Edge
- [421] Effective Self-supervised Pre-training on Low-compute Networks without Distillation
- [422] SEIFER Scalable Edge Inference for Deep Neural Networks
- [423] Design and Prototyping Distributed CNN Inference Acceleration in Edge Computing
- [424] Packet-Loss-Tolerant Split Inference for Delay-Sensitive Deep Learning in Lossy Wireless Networks
- [425] Communication-oriented Model Fine-tuning for Packet-loss Resilient Distributed Inference under Highly Lossy IoT Networks
- [426] Autodidactic Neurosurgeon Collaborative Deep Inference for Mobile Edge Intelligence via Online Learning

- [427] A Self-stabilizing Control Plane for the Edge and Fog Ecosystems
- [428] SMPC-based Federated Learning for 6G enabled Internet of Medical Things
- [429] Cloud-Device Collaborative Adaptation to Continual Changing Environments in the Real-world
- [430] Future Vision of Dynamic Certification Schemes for Autonomous Systems
- [431] Rethinking Certification for Higher Trust and Ethical Safeguarding of Autonomous Systems
- [432] SOTER A Runtime Assurance Framework for Programming Safe Robotics Systems
- [433] Certified Control An Architecture for Verifiable Safety of Autonomous Vehicles
- [434] A Practical Multilevel Governance Framework for Autonomous and Intelligent Systems
- [435] Science based AI model certification for untrained operational environments with application in traffic state estimation
- [436] Towards a Framework for Deep Learning Certification in Safety-Critical Applications Using Inherently Safe Design and Run-Time Error Detection
- [437] Adaptive Security and Trust Management for Autonomous Messaging Systems
- [438] A Factor Graph Model of Trust for a Collaborative Multi-Agent System
- [439] Distributed Governance a Principal-Agent Approach to Data Governance – Part 1 Background & Core Definitions
- [440] Towards a Policy-as-a-Service Framework to Enable Compliant, Trustworthy AI and HRI Systems in the Wild
- [441] A Red Teaming Framework for Securing AI in Maritime Autonomous Systems
- [442] Limits to Verification and Validation of Agentic Behavior
- [443] Self-driven Grounding Large Language Model Agents with Automatical Language-aligned Skill Learning
- [444] Adaptable Recovery Behaviors in Robotics A Behavior Trees and Motion Generators(BTMG) Approach for Failure Management
- [445] RepairAgent An Autonomous, LLM-Based Agent for Program Repair
- [446] Synergistic Integration of Large Language Models and Cognitive Architectures for Robust AI An Exploratory Analysis
- [447] Hallucination Detection in Foundation Models for Decision-Making A Flexible Definition and Review of the State of the Art
- [448] Grand Challenges in Resilience Autonomous System Resilience through Design and Runtime Measures
- [449] A new design principle of robust onion-like networks self-organized in growth
- [450] Graph Learning based Generative Design for Resilience of Interdependent Network Systems
- [451] Message Passing for Analysis and Resilient Design of Self-Healing Interdependent Cyber-Physical Networks

- [452] Enhancing resilience of interdependent networks by healing
- [453] Deep Learning-based Resource Allocation for Infrastructure Resilience
- [454] Efficient Utility-Driven Self-Healing Employing Adaptation Rules for Large Dynamic Architectures